

LAPORAN PRAKTIKUM BIG DATA
PRAKTIKUM 3

PENGUMPULAN DAN PRA-PEMROSESAN DATA
UNTUK BIG DATA



Disusun oleh:

Habibi Putra Rizqullah

NIM 2411531001

Kelas A

Dosen Pengampu:

Afdhal Dinilhak, M.Kom., S.Kom.

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
UNIVERSITAS ANDALAS
2026

DAFTAR ISI

BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
BAB II DASAR TEORI.....	3
2.1 Akuisisi dan Pra-pemrosesan Data	3
2.2 Pola Akuisisi Data	3
2.3 Enam Dimensi Kualitas Data.....	3
2.4 Nilai Hilang	4
2.5 Outlier dengan IQR dan Z-score.....	4
2.6 Penggabungan Data dan Granularitas	4
2.7 Encoding, Scaling, dan Data Leakage	5
2.8 Idempotensi, Lineage, dan Kontrak Data.....	5
2.9 Parquet Berpartisi dan PySpark	5
BAB III METODOLOGI PRAKTIKUM.....	6
3.1 Waktu dan Lingkungan Praktikum.....	6
3.2 Perangkat dan Perangkat Lunak.....	6
3.3 Dataset dan Sumber Data.....	7
3.4 Akses Repository dan Artefak.....	7
3.5 Tahapan Praktikum.....	8
BAB IV HASIL DAN PEMBAHASAN.....	9
4.1 Menyiapkan Lingkungan Kerja - K-1.....	9
4.2 Akuisisi Berkas Berkala yang Tahan Gagal - K-2	13
4.3 Akuisisi API JSON - K-3	19
4.4 Jalur Cadangan ketika Internet Diblokir - K-4.....	23
4.5 SQLite dan Pemuatan Bertahap - K-5	24
4.6 Profiling Kualitas Data - K-6.....	28
4.7 Nilai Hilang dan Duplikat - K-7	33
4.8 Outlier dan Penggabungan Data - K-8.....	38
4.9 Rekayasa Fitur, Encoding, dan Scaling - K-9.....	47
4.10 Pipeline, Validasi, dan Parquet Berpartisi - K-10.....	50
4.11 Pra-pemrosesan Setara dengan PySpark - K-11.....	56
4.12 Analisis Hasil - Bagian O	62
4.12.1 Peta Kualitas Data	62
4.12.2 Dampak Strategi Pengisian	63
4.12.3 IQR dan Z-score.....	64
4.12.4 Biaya Pembacaan dan Penggabungan.....	65
4.12.5 Cakupan Penggabungan.....	68
4.12.6 Susut Data Kumulatif	70
4.12.7 Perbandingan pandas dan Spark	72
4.12.8 Distribusi total_amount Sebelum dan Sesudah Pra-pemrosesan.....	73
4.13 Studi Kasus Penetapan Harga Dinamis - Bagian P.....	75
4.13.1 Pembentukan Tabel Analitik	75
4.13.2 Perbandingan Hujan dan Tidak Hujan	77
4.13.3 Batas Kepercayaan Hasil	78
4.13.4 Rencana Pembaruan Otomatis Bulanan	79
4.14 Latihan Praktikum - Bagian Q.....	80
4.14.1 Kecepatan Unduh dan Cache	80
4.14.2 Dua Aturan Kualitas Tambahan	82
4.14.3 Join Lokasi Tujuan	83
4.14.4 Perbandingan MinMaxScaler.....	84
4.14.5 Pembuktian Validasi many-to-one.....	85
4.14.6 Pipeline Inkremental.....	86

BAB V TUGAS PRAKTIKUM DAN EVALUASI.....	89
5.1 Tugas Praktikum - Bagian R.....	89
5.1.1 Persiapan Notebook Tugas.....	89
5.1.2 R.1 Fungsi Umum, Cache, dan Pencatatan	92
5.1.3 R.2 Akuisisi Tabel Referensi dan Sumber Keempat	95
5.1.4 R.3 Akuisisi Cuaca dan Laporan Kualitas.....	97
5.1.5 R.4 Pipeline Dua Bulan	100
5.1.6 R.5 Menjalankan Januari dan Juli 2023	108
5.1.7 R.6 Tidak Ada Duplikasi Antarbulan	109
5.1.8 R.7 Laporan Kualitas Komparatif.....	111
5.1.9 R.8 Integrasi Sumber Keempat.....	113
5.1.10 R.9 Kamus Data	114
5.1.11 R.10 Lineage dan Diagram Alur.....	118
5.1.12 R.11 Idempotensi dan Reproducibility.....	121
5.1.13 R.12 Artefak Akhir dan Arsip ZIP.....	122
5.1.14 R.13 Ringkasan Hasil Aktual.....	124
5.2 Jawaban Pertanyaan Evaluasi - Bagian S	125
5.2.1 ETL dan ELT.....	125
5.2.2 Risiko Akuisisi melalui API.....	125
5.2.3 MCAR, MAR, dan MNAR.....	125
5.2.4 Z-score pada Distribusi Miring	125
5.2.5 Kunci Ganda pada Join Many-to-one	125
5.2.6 Data Leakage pada Scaling.....	126
5.2.7 Pipeline Idempoten.....	126
5.2.8 Partisi Berkardinalitas Tinggi.....	126
5.2.9 Pemeriksaan Sebelum Scraping	126
BAB VI PENUTUP.....	127
6.1 Kesimpulan.....	127
DAFTAR PUSTAKA.....	128
LAMPIRAN.....	129
Lampiran 1 Tautan Akses Praktikum	129
Lampiran 2 Dokumentasi Struktur Repository	130

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data untuk analisis Big Data umumnya tidak berasal dari satu sumber yang sudah rapi. Data dapat diperoleh dari berkas berkala, layanan API, basis data operasional, serta tabel referensi yang mempunyai skema dan tingkat rincian berbeda. Kondisi tersebut menimbulkan risiko nilai hilang, duplikasi, anomali, perubahan skema, dan kegagalan penggabungan yang dapat memengaruhi kesimpulan analisis.

Praktikum ini membahas pengumpulan dan pra-pemrosesan data secara terukur menggunakan NYC TLC Yellow Taxi Trip Records Januari 2023 sebagai sumber transaksi utama. Data perjalanan digabungkan dengan tabel zona, data cuaca per jam dari Open-Meteo, dan tabel referensi pembayaran pada SQLite. Pada tugas praktikum, pipeline diperluas ke dua bulan dan ditambah sumber hari libur nasional dari Nager.Date.

Setiap transformasi perlu dapat dijelaskan, diulang, dan ditelusuri. Oleh sebab itu, proses akuisisi menerapkan cache, retry, dan penulisan atomik; proses pembersihan disertai laporan kualitas; sedangkan hasil kurasi disimpan sebagai Parquet berpartisi. Waktu proses dan perubahan resident set size (RSS) juga dicatat agar biaya komputasi tidak hanya dinilai berdasarkan perkiraan.

Kegiatan ini berkaitan dengan konsep Big Data karena kelayakan alat ditentukan oleh hubungan antara ukuran data, keragaman sumber, kualitas data, serta sumber daya runtime. Pandas digunakan untuk pipeline satu mesin, sedangkan PySpark digunakan sebagai pembanding untuk menunjukkan manfaat dan overhead pemrosesan berbasis Spark pada dataset yang masih dapat ditangani di Google Colab.

1.2 Tujuan Praktikum

1. Mengunduh data berkala secara terprogram dengan mekanisme retry, cache, dan penulisan atomik.
2. Memanggil API JSON, memvalidasi struktur respons, dan menangani galat sementara secara tepat.
3. Membaca data dari SQLite menggunakan SQL dan pemuatan bertahap.
4. Menyusun laporan kualitas data berdasarkan kelengkapan, keunikan, validitas, konsistensi, ketepatan waktu, dan kesesuaian referensi.
5. Membandingkan strategi penanganan nilai hilang, duplikat, dan outlier beserta dampaknya terhadap distribusi.
6. Menggabungkan data transaksi dengan data referensi tanpa ledakan atau kehilangan baris yang tidak disengaja.
7. Melakukan encoding dan scaling tanpa data leakage.
8. Membangun pipeline yang idempoten, tervalidasi, terdokumentasi melalui lineage, serta menyimpan keluaran sebagai Parquet berpartisi.
9. Membandingkan implementasi pandas dan PySpark pada runtime yang sama.
10. Memperluas pipeline ke dua bulan dan menambahkan sumber data publik keempat pada notebook tugas terpisah.

BAB II

DASAR TEORI

2.1 Akuisisi dan Pra-pemrosesan Data

Akuisisi data merupakan tahap pengambilan data dari sumber menuju lapisan penyimpanan mentah. Pra-pemrosesan mengubah data mentah menjadi data terkurasi yang dapat digunakan untuk analisis atau pemodelan. Pemisahan lapisan mentah dan terkurasi menjaga sumber asli tetap tersedia ketika aturan transformasi perlu diperbaiki atau dijalankan ulang.

Dalam pola ETL, data diekstrak, ditransformasi, lalu dimuat ke sistem tujuan. Dalam pola ELT, data mentah dimuat terlebih dahulu dan transformasi dilakukan setelahnya. ELT sesuai untuk lingkungan Big Data karena data mentah tetap tersimpan dan dapat diproses ulang ketika skema atau kebutuhan analisis berubah.

2.2 Pola Akuisisi Data

Berkas berkala berisiko terputus atau berubah tanpa pemberitahuan sehingga memerlukan cache, pemeriksaan ukuran atau struktur, dan penulisan ke berkas sementara. API web berisiko mengalami rate limit, gangguan server, dan perubahan skema; permintaan perlu memiliki timeout, validasi respons, dan retry hanya untuk galat sementara. Basis data operasional berisiko membebani sistem dan menghasilkan data terlalu besar sehingga kolom perlu dibatasi serta hasil kueri dapat dibaca secara bertahap.

2.3 Enam Dimensi Kualitas Data

Tabel 2.1 Dimensi Kualitas Data

Dimensi	Pertanyaan Utama	Pemeriksaan pada Praktikum
Kelengkapan	Apakah nilai yang dibutuhkan tersedia?	Persentase nilai kosong per kolom
Keunikan	Apakah baris atau kunci tercatat lebih dari satu kali?	Duplikat penuh dan duplikat berdasarkan kunci
Validitas	Apakah nilai berada pada domain yang diperbolehkan?	Rentang jarak, tarif, dan kode referensi

Dimensi	Pertanyaan Utama	Pemeriksaan pada Praktikum
Konsistensi	Apakah hubungan antarkolom masuk akal?	Durasi perjalanan dan total terhadap tarif
Ketepatan waktu	Apakah data berada pada periode yang diharapkan?	Rentang Januari 2023
Akurasi	Apakah nilai sesuai kenyataan?	Pemeriksaan terbatas melalui tabel referensi

Kualitas data dinilai menggunakan aturan yang eksplisit. Ambang 0,01-100 mil dan 1-180 menit pada praktikum merupakan keputusan analitis, bukan kebenaran mutlak. Nilai yang gagal terhadap aturan tersebut perlu diperiksa konteksnya sebelum dinyatakan sebagai kesalahan.

2.4 Nilai Hilang

Missing Completely at Random (MCAR) berarti kehilangan tidak berhubungan dengan variabel yang diamati maupun nilai yang hilang. Missing at Random (MAR) berarti kehilangan dapat dijelaskan oleh variabel lain yang teramati. Missing Not at Random (MNAR) berarti peluang hilang berhubungan dengan nilai yang tidak tercatat. Pemilihan drop, imputasi konstan, imputasi kelompok, atau indikator kehilangan harus mempertimbangkan mekanisme tersebut.

2.5 Outlier dengan IQR dan Z-score

Metode IQR menandai nilai di luar $Q1 - 1,5 \text{ IQR}$ dan $Q3 + 1,5 \text{ IQR}$. Metode ini relatif tahan terhadap distribusi miring. Z-score menilai jarak nilai dari rerata dalam satuan simpangan baku dan lebih sesuai ketika distribusi mendekati normal. Outlier tidak selalu merupakan galat; perjalanan bandara atau tarif tinggi dapat menjadi kejadian bisnis yang sah.

2.6 Penggabungan Data dan Granularitas

Join many-to-one mengizinkan banyak transaksi mengarah ke satu baris referensi, tetapi kunci pada tabel referensi wajib unik. Parameter validate pada pandas mengubah pelanggaran asumsi menjadi error yang jelas. Untuk penggabungan berbasis waktu, kedua sumber harus disamakan granularitasnya,

misalnya waktu transaksi dibulatkan ke tingkat jam sebelum digabungkan dengan cuaca per jam.

2.7 Encoding, Scaling, dan Data Leakage

One-hot encoding mengubah kategori menjadi kolom biner dan sesuai untuk kategori dengan kardinalitas rendah. StandardScaler memusatkan data terhadap rerata dan menskalakannya berdasarkan simpangan baku, sedangkan MinMaxScaler memetakan rentang data latih ke 0-1. Scaler dan encoder harus difit hanya pada data latih, kemudian digunakan untuk mentransformasikan data latih dan uji agar informasi dari data uji tidak bocor.

2.8 Idempotensi, Lineage, dan Kontrak Data

Pipeline disebut idempoten apabila input dan parameter yang sama menghasilkan keadaan akhir yang sama ketika dijalankan berulang. Lineage mencatat sumber, waktu pengambilan, jumlah baris, serta transformasi yang diterapkan. Kontrak data memeriksa keberadaan kolom, tipe, keadaan DataFrame, dan keunikan kunci sehingga kegagalan berhenti sebelum data yang salah masuk ke tahap berikutnya.

2.9 Parquet Berpartisi dan PySpark

Parquet merupakan format kolumnar yang mendukung kompresi dan pembacaan kolom terpilih. Partisi berdasarkan borrough memungkinkan partition pruning ketika analisis hanya membaca satu kategori. Pada Spark, transformation dievaluasi secara lazy dan baru dijalankan ketika action seperti count dipanggil. Broadcast join dapat menghindari shuffle untuk tabel kecil, tetapi inisialisasi JVM dan perencanaan eksekusi tetap menimbulkan overhead pada data yang masih dapat diproses efisien oleh pandas.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Lingkungan Praktikum

Praktikum dilaksanakan menggunakan Google Colab dengan runtime CPU.
Tanggal pelaksanaan pada tanggal 24 September 2026.

3.2 Perangkat dan Perangkat Lunak

Tabel 3.1 Perangkat dan Perangkat Lunak

Perangkat atau Software	Versi atau Spesifikasi	Kegunaan
Google Colab	Runtime CPU	Menjalankan notebook dan mengakses penyimpanan sementara
Python	3.13.15	Bahasa pemrograman utama
pandas	2.2.3	Pengolahan data tabular dan pipeline satu mesin
NumPy	2.1.3	Operasi numerik dan array
PyArrow	23.0.1	Membaca dan menulis Parquet
requests	2.32.4	Akuisisi berkas dan API
scikit-learn	1.6.1	Encoding, pembagian data, dan scaling
SQLAlchemy	2.0.52	Koneksi SQLite melalui engine
PySpark	4.0.4	Pra-pemrosesan pembandingan berbasis Spark
Java	-	Menjalankan JVM untuk Spark
matplotlib	-	Membuat histogram dan diagram lineage
psutil	-	Mengukur RSS proses Python
Google Drive	Layanan penyimpanan daring	Menyimpan artefak yang tidak boleh hilang saat runtime berakhir
GitHub	Repository daring	Menyimpan notebook, artefak, dan dokumentasi

3.3 Dataset dan Sumber Data

Dataset utama adalah NYC TLC Yellow Taxi Trip Records Januari 2023. Notebook tugas memperluas pipeline ke Januari dan Juli 2023. Semua sumber dan tingkat rincian dirangkum pada tabel berikut.

Tabel 3.2 Sumber Data Praktikum dan Tugas

Sumber	Bentuk	Periode atau Granularitas	Peran
NYC TLC Yellow Taxi Trip Records	Parquet	Januari 2023; tugas menambah Juli 2023	Sumber transaksi utama
NYC TLC Taxi Zone Lookup	CSV	265 baris referensi	Nama zona dan borough
Open-Meteo Archive API	JSON	Cuaca per jam	Suhu dan presipitasi
SQLite lokal	Basis data	300.000 baris latihan	Simulasi sumber operasional dan tabel pembayaran
Nager.Date	JSON	Hari libur nasional AS tahun 2023	Sumber keempat pada tugas praktikum

Berkas Januari yang digunakan berukuran 45,4649 MB atau 47.673.370 byte dan memuat 3.066.766 baris pada 10 kolom terpilih. Tabel zona berukuran 12.331 byte dengan 265 baris. Data cuaca memuat 744 jam dari 1 sampai 31 Januari 2023 tanpa nilai kosong pada tiga kolomnya.

3.4 Akses Repository dan Artefak

Kode program, notebook tugas, dan artefak praktikum ditempatkan pada repository GitHub dan Google Drive. Tautan ditampilkan dalam bentuk URL lengkap agar alamat tujuan dapat diperiksa langsung dan tetap dapat diklik.

Repository Praktikum 3:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum03

Google Drive Praktikum 3:

https://drive.google.com/drive/folders/108V3OLKvsBIJCvmeTSSrfkQqZrdgvnWu?usp=drive_link

3.5 Tahapan Praktikum

1. K-1 menyiapkan lingkungan, folder kerja, pengukuran waktu dan RSS, serta lineage.
2. K-2 mengunduh Parquet perjalanan dan CSV zona dengan cache, retry, serta penulisan atomik.
3. K-3 mengambil cuaca Open-Meteo per jam dan memvalidasi struktur JSON.
4. K-4 menyediakan data cuaca sintetis sebagai jalur cadangan, tetapi tidak digunakan.
5. K-5 membangun SQLite dan membaca tabel secara bertahap.
6. K-6 menyusun profil kolom dan delapan aturan kualitas data.
7. K-7 membandingkan strategi imputasi serta menangani duplikat.
8. K-8 membandingkan IQR dan Z-score, menyaring nilai tidak layak, dan menjalankan tiga join.
9. K-9 membentuk fitur, membagi data, melakukan scaling, dan one-hot encoding tanpa leakage.
10. K-10 membungkus transformasi menjadi pipeline, memvalidasi, dan menyimpan Parquet berpartisi.
11. K-11 menjalankan aturan setara menggunakan PySpark dan memeriksa BroadcastHashJoin.
12. Bagian O, P, dan Q membahas analisis hasil, studi kasus pricing, dan enam latihan.
13. Notebook tugas R-S memperluas pipeline ke dua bulan, menambah Nager.Date, dan menjawab evaluasi.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Menyiapkan Lingkungan Kerja - K-1

A. Tujuan Langkah

Memastikan pustaka tersedia, menghubungkan Google Drive, menyiapkan lapisan data, serta membentuk fungsi pengukuran waktu, RSS, dan lineage. Data mentah, data terkurasi, dan artefak Drive dipisahkan. Pemisahan ini mencegah data asli tertimpa dan menjaga hasil penting tetap tersedia setelah runtime Colab berhenti.

B. Kode Program



```
K-1 Instalasi SQLAlchemy 1/1
1 !pip install sqlalchemy
```

Gambar 4.1 Kode Program K-1 (baris 1-1)


```
K-1 Menyiapkan Lingkungan Kerja 1/3

1 # K-1: Menyiapkan lingkungan kerja
2
3 import sys
4 import time
5 import os
6 import json
7 import sqlite3
8 import gc
9
10 import pandas as pd
11 import numpy as np
12 import psutil
13
14 # Menampilkan versi Python
15 print("Python      :", sys.version.split()[0])
16
17 # Memeriksa versi pustaka yang diperlukan
18 for nama in [
19     "pandas",
20     "numpy",
21     "pyarrow",
22     "requests",
23     "sklearn",
24     "sqlalchemy"
25 ]:
26     try:
27         modul = __import__(nama)
28         print(f"{nama:11s}: {modul.__version__}")
29     except ImportError:
30         print(f"{nama:11s}: BELUM TERPASANG")
31
32 # Menghubungkan Google Colab dengan Google Drive
33 from google.colab import drive
34 drive.mount("/content/drive")
35
36 # Menentukan direktori kerja
37 DIR_MENTAH = "/content/lapisan_mentah"
38 DIR_KURASI = "/content/lapisan_terkurasi"
39 DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum3"
40
41 # Membuat direktori jika belum tersedia
42 for direktori in (DIR_MENTAH, DIR_KURASI, DIR_SIMPAN):
43     os.makedirs(direktori, exist_ok=True)
44
```

Gambar 4.2 Kode Program K-1 Bagian 1 dari 3 (baris 1-44)

```
K-1 Menyiapkan Lingkungan Kerja 2/3

45 print("\nFolder kerja:")
46 print("Data mentah :", DIR_MENTAH, os.path.exists(DIR_MENTAH))
47 print("Data kurasi :", DIR_KURASI, os.path.exists(DIR_KURASI))
48 print("Penyimpanan :", DIR_SIMPAN, os.path.exists(DIR_SIMPAN))
49
50 # Menyiapkan pengukuran waktu dan penggunaan memori
51 proses = psutil.Process(os.getpid())
52 catatan = []
53
54 def rss_mb():
55     return proses.memory_info().rss / 1024**2
56
57 def ukur(label, fungsi):
58     memori_awal = rss_mb()
59     waktu_awal = time.perf_counter()
60
61     hasil = fungsi()
62
63     detik = time.perf_counter() - waktu_awal
64     delta_memori = rss_mb() - memori_awal
65
66     catatan.append({
67         "langkah": label,
68         "detik": round(detik, 2),
69         "delta_rss_mb": round(delta_memori, 1)
70     })
71
72     print(
73         f"[{label}] {detik:.2f} detik | "
74         f"delta RSS: {delta_memori:.1f} MB"
75     )
76
77     return hasil
78
79 # Menyiapkan pencatatan asal-usul data
80 lineage = []
81
82 def catat_sumber(nama, asal, jumlah_baris, keterangan=""):
83     lineage.append({
84         "sumber": nama,
85         "asal": asal,
86         "diambil_pada": pd.Timestamp.now("UTC").isoformat(),
87         "jumlah_baris": jumlah_baris,
88         "keterangan": keterangan
```

Gambar 4.3 Kode Program K-1 Bagian 2 dari 3 (baris 45-88)

```
K-1 Menyiapkan Lingkungan Kerja 3/3

89     })
90
91     print(f"[lineage] {nama}: {jumlah_baris:,} baris")
92
93 print("\nK-1 selesai: lingkungan kerja telah disiapkan.")
```

Gambar 4.4 Kode Program K-1 Bagian 3 dari 3 (baris 89-93)

C. Output Program

```
*** Python      : 3.13.15
    pandas      : 2.2.3
    numpy       : 2.1.3
    pyarrow     : 23.0.1
    requests    : 2.32.4
    sklearn     : 1.6.1
    sqlalchemy  : 2.0.52
    Mounted at  /content/drive

Folder kerja:
Data mentah : /content/lapisan_mentah True
Data kurasi : /content/lapisan_terkurasi True
Penyimpanan : /content/drive/MyDrive/BigData/Praktikum3 True

K-1 selesai: lingkungan kerja telah disiapkan.
```

Gambar 4.5 Output Program K-1

D. Pembahasan

Runtime menggunakan Python 3.13.15, pandas 2.2.3, NumPy 2.1.3, PyArrow 23.0.1, requests 2.32.4, scikit-learn 1.6.1, dan SQLAlchemy 2.0.52. Angka ini menggambarkan lingkungan yang benar-benar dipakai sehingga hasil performa hanya berlaku pada runtime tersebut.

Ketiga folder kerja berhasil dibuat. Folder /content bersifat sementara, sedangkan /content/drive/MyDrive/BigData/Praktikum3 digunakan untuk menyimpan laporan kualitas, lineage, pengukuran kinerja, dan arsip data bersih.

Fungsi ukur mencatat durasi serta perubahan RSS proses Python. Nilai RSS berguna untuk membandingkan perubahan memori proses, tetapi tidak selalu mencakup memori proses lain seperti JVM Spark.

Tabel 4.1 Ringkasan Lingkungan Kerja

Komponen	Hasil Aktual	Keterangan
Python	3.13.15	Runtime utama
pandas	2.2.3	Pipeline satu mesin
PyArrow	23.0.1	Parquet
scikit-learn	1.6.1	Scaling dan encoding
SQLAlchemy	2.0.52	SQLite engine
PySpark	4.0.4	Spark local mode pada K-11
CPU, RAM, dan disk	-	Belum dicetak pada K-1

4.2 Akuisisi Berkas Berkala yang Tahan Gagal - K-2

A. Tujuan Langkah

Mengunduh data perjalanan dan tabel zona dengan cache, retry, validasi sederhana, serta penulisan atomik. Cache hanya dipakai apabila berkas dapat dibaca. Unduhan ditulis ke ekstensi .part lalu dipindahkan ke nama akhir menggunakan os.replace setelah isinya dinyatakan valid.

B. Kode Program



```
1 # K-2: Akuisisi data dengan cache, retry, dan penulisan atomik
2
3 import requests
4 import pyarrow.parquet as pq
5
6 # Mengurangi penggunaan memori jika K-2 sebelumnya pernah dijalankan
7 if "trip" in globals():
8     del trip
9     gc.collect()
10
11
12 def cache_valid(path):
13     """
14     Memeriksa apakah berkas cache tersedia dan dapat dibaca.
15     """
16
17     if not os.path.exists(path):
18         return False
19
20     if os.path.getsize(path) == 0:
21         return False
22
23     try:
24         if path.endswith(".csv"):
25             pd.read_csv(path, nrows=2)
26
27         elif path.endswith(".parquet"):
28             pq.ParquetFile(path).metadata
29
30         return True
31
32     except Exception as error:
33         print(
34             "Cache tidak valid:",
35             os.path.basename(path),
36             f"({error})"
37         )
38         return False
39
40
41 def unduh_aman(url, tujuan, percobaan=3, jeda_awal=2):
42     """
43     Mengunduh berkas dengan cache, retry,
44     dan penulisan atomik.
```

Gambar 4.6 Kode Program K-2 Bagian 1 dari 6 (baris 1-44)

```
K-2 Akuisisi Berkas Berkala 2/6

45     """
46
47     # Memakai cache jika berkas tersedia dan dapat dibaca
48     if cache_valid(tujuan):
49         print("Cache ditemukan:", os.path.basename(tujuan))
50         return tujuan
51
52     sementara = tujuan + ".part"
53
54     for i in range(percobaan):
55         try:
56             with requests.get(
57                 url,
58                 stream=True,
59                 timeout=60
60             ) as respons:
61
62                 respons.raise_for_status()
63
64                 # Menyimpan data sedikit demi sedikit
65                 with open(sementara, "wb") as berkas:
66                     for potongan in respons.iter_content(
67                         chunk_size=1024 * 1024
68                     ):
69                         if potongan:
70                             berkas.write(potongan)
71
72                 # Memastikan hasil unduhan tidak kosong
73                 if os.path.getsize(sementara) == 0:
74                     raise IOError("Berkas hasil unduhan kosong")
75
76                 # Mengganti cache lama dengan berkas baru
77                 os.replace(sementara, tujuan)
78
79                 # Memastikan berkas baru dapat dibaca
80                 if not cache_valid(tujuan):
81                     raise IOError(
82                         "Berkas selesai diunduh, tetapi tidak dapat dibaca"
83                     )
84
85                 print(
86                     "Unduhan selesai:",
87                     os.path.basename(tujuan)
88                 )
```

Gambar 4.7 Kode Program K-2 Bagian 2 dari 6 (baris 45-88)

```
K-2 Akuisisi Berkas Berkala 3/6

89
90         return tujuan
91
92     except Exception as error:
93         # Menghapus berkas sementara jika proses gagal
94         if os.path.exists(sembanta):
95             os.remove(sembanta)
96
97         jeda = jeda_awal * (2 ** i)
98
99         print(
100             f"Percobaan {i + 1} gagal ({error}); "
101             f"menunggu {jeda} detik"
102         )
103
104         if i < percobaan - 1:
105             time.sleep(jeda)
106
107     raise RuntimeError(
108         f"Gagal mengunduh setelah {percobaan} percobaan: {url}"
109     )
110
111
112 # URL sumber resmi NYC TLC
113 URL_TRIP = (
114     "https://d37ci6vzurychx.cloudfront.net/"
115     "trip-data/yellow_tripdata_2023-01.parquet"
116 )
117
118 URL_ZONA = (
119     "https://d37ci6vzurychx.cloudfront.net/"
120     "misc/taxi_zone_lookup.csv"
121 )
122
123
124 # Lokasi penyimpanan data mentah
125 PATH_TRIP = os.path.join(
126     DIR_MENTAH,
127     "yellow_tripdata_2023-01.parquet"
128 )
129
130 PATH_ZONA = os.path.join(
131     DIR_MENTAH,
132     "taxi_zone_lookup.csv"
```

Gambar 4.8 Kode Program K-2 Bagian 3 dari 6 (baris 89-132)

```
K-2 Akuisisi Berkas Berkala 4/6

133 )
134
135
136 # Mengunduh dataset perjalanan dan tabel zona
137 ukur(
138     "unduh trip",
139     lambda: unduh_aman(URL_TRIP, PATH_TRIP)
140 )
141
142 ukur(
143     "unduh zona",
144     lambda: unduh_aman(URL_ZONA, PATH_ZONA)
145 )
146
147
148 # Menampilkan ukuran berkas
149 print("\nUkuran berkas:")
150
151 for path in (PATH_TRIP, PATH_ZONA):
152     ukuran_byte = os.path.getsize(path)
153     ukuran_mb = ukuran_byte / 1024**2
154
155     print(
156         f"{os.path.basename(path)} -> "
157         f"{ukuran_mb:.4f} MB "
158         f"({ukuran_byte:,} byte)"
159     )
160
161
162 # Daftar kolom yang diperlukan
163 KOLOM = [
164     "tpep_pickup_datetime",
165     "tpep_dropoff_datetime",
166     "passenger_count",
167     "trip_distance",
168     "PULocationID",
169     "DOLocationID",
170     "payment_type",
171     "fare_amount",
172     "tip_amount",
173     "total_amount"
174 ]
175
176
```

Gambar 4.9 Kode Program K-2 Bagian 4 dari 6 (baris 133-176)

```
K-2 Akuisisi Berkas Berkala 5/6

177 # Membaca dataset perjalanan
178 trip = ukur(
179     "baca trip",
180     lambda: pd.read_parquet(
181         PATH_TRIP,
182         columns=KOLOM
183     )
184 )
185
186
187 # Membaca tabel zona
188 zona = pd.read_csv(PATH_ZONA)
189
190
191 # Menghindari duplikasi lineage jika sel dijalankan ulang
192 lineage[:] = [
193     item for item in lineage
194     if item["sumber"] not in ("trip", "zona")
195 ]
196
197
198 # Mencatat asal-usul data
199 catat_sumber(
200     "trip",
201     URL_TRIP,
202     len(trip),
203     "Parquet bulanan NYC TLC"
204 )
205
206 catat_sumber(
207     "zona",
208     URL_ZONA,
209     len(zona),
210     "Tabel referensi zona NYC TLC"
211 )
212
213
214 # Menampilkan informasi dataset
215 print("\nInformasi dataset:")
216
217 print(
218     f"Dataset trip : {trip.shape[0]:,} baris "
219     f"x {trip.shape[1]} kolom"
220 )
```

Gambar 4.10 Kode Program K-2 Bagian 5 dari 6 (baris 177-220)

```
K-2 Akuisisi Berkas Berkala 6/6

221
222 print(
223     f"Tabel zona : {zona.shape[0]:,} baris "
224     f"x {zona.shape[1]} kolom"
225 )
226
227 print("\nKolom dataset trip:")
228 print(trip.columns.tolist())
229
230 print("\nLima baris pertama tabel zona:")
231 display(zona.head())
```

Gambar 4.11 Kode Program K-2 Bagian 6 dari 6 (baris 221-231)

C. Output Program

```
Unduhan selesai: yellow_tripdata_2023-01.parquet
[unduh trip] 0.44 detik | delta RSS: 4.9 MB
Unduhan selesai: taxi_zone_lookup.csv
[unduh zona] 0.06 detik | delta RSS: 0.9 MB

Ukuran berkas:
yellow_tripdata_2023-01.parquet -> 45.4649 MB (47,673,378 byte)
taxi_zone_lookup.csv -> 0.0118 MB (12,331 byte)
[baris trip] 1.66 detik | delta RSS: 266.2 MB
[lineage] trip: 3,066,766 baris
[lineage] zona: 265 baris

Informasi dataset:
Dataset trip : 3,066,766 baris x 10 kolom
Tabel zona : 265 baris x 4 kolom

Kolom dataset trip:
['tpep_pickup_datetime', 'tpep_dropoff_datetime', 'passenger_count', 'trip_distance', 'PULocationID', 'DOLocationID', 'payment_type', 'fare_amount', 'tip_amount', 'total_amount']

Lima baris pertama tabel zona:
```

	locationID	Borough	Zone	service_zone
0	1	EWB	Newark Airport	EWB
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

Gambar 4.12 Output Program K-2

D. Pembahasan

Berkas perjalanan berukuran 45,4649 MB dan tabel zona berukuran 0,0118 MB. Pembacaan 3.066.766 baris pada 10 kolom memerlukan 1,06 detik dengan kenaikan RSS 251,2 MB.

Penulisan atomik mencegah berkas setengah unduh memiliki nama akhir yang seolah-olah valid. Cache menghindari unduhan berulang, sedangkan retry berjeda menangani kegagalan jaringan yang bersifat sementara.

Daftar kolom mencakup PULocationID dan DOLocationID karena keduanya menjadi kunci penggabungan dengan tabel zona.

4.3 Akuisisi API JSON - K-3

A. Tujuan Langkah

Mengambil suhu dan curah hujan per jam di New York City melalui Open-Meteo Archive API. Zona waktu America/New_York dipakai agar timestamp cuaca konsisten dengan waktu perjalanan. Respons wajib memiliki struktur hourly dan kolom yang dibutuhkan.

B. Kode Program



```
1 # K-3: Mengambil data cuaca melalui API
2
3 API_CUACA = "https://archive-api.open-meteo.com/v1/archive"
4
5 def ambil_cuaca(
6     tanggal_mulai,
7     tanggal_selesai,
8     latitude=40.7128,
9     longitude=-74.0060,
10    maksimum_percobaan=3
11 ):
12     parameter = {
13         "latitude": latitude,
14         "longitude": longitude,
15         "start_date": tanggal_mulai,
16         "end_date": tanggal_selesai,
17         "hourly": "temperature_2m,precipitation",
18         "timezone": "America/New_York"
19     }
20
21     for percobaan_ke in range(1, maksimum_percobaan + 1):
22         try:
23             respons = requests.get(
24                 API_CUACA,
25                 params=parameter,
26                 timeout=60
27             )
28
29             # Respons berhasil
30             if respons.status_code == 200:
31                 data_json = respons.json()
32
33                 if "hourly" not in data_json:
34                     raise ValueError(
35                         "Bagian 'hourly' tidak ditemukan pada respons API."
36                     )
37
38                 data_cuaca = pd.DataFrame(data_json["hourly"])
39
40                 kolom_wajib = {
41                     "time",
42                     "temperature_2m",
43                     "precipitation"
44                 }
```

Gambar 4.13 Kode Program K-3 Bagian 1 dari 4 (baris 1-44)

```
K-3 Akuisisi API Cuaca 2/4

45     kolom_hilang = kolom_wajib - set(data_cuaca.columns)
46
47     if kolom_hilang:
48         raise ValueError(
49             f"Kolom cuaca tidak lengkap: {kolom_hilang}"
50         )
51
52     return data_cuaca
53
54 # Status sementara yang masih layak dicoba ulang
55 if respons.status_code in (429, 500, 502, 503, 504):
56     waktu_tunggu = 2 ** (percobaan_ke - 1)
57
58     print(
59         f"Percobaan {percobaan_ke} gagal "
60         f"dengan status {respons.status_code}. "
61         f"Mencoba kembali dalam {waktu_tunggu} detik."
62     )
63
64     time.sleep(waktu_tunggu)
65     continue
66
67 # Memunculkan error untuk status selain yang ditangani
68 respons.raise_for_status()
69
70 except requests.RequestException as error:
71     if percobaan_ke == maksimum_percobaan:
72         raise RuntimeError(
73             "Data cuaca gagal diambil setelah "
74             f"{maksimum_percobaan} percobaan."
75         ) from error
76
77     waktu_tunggu = 2 ** (percobaan_ke - 1)
78
79     print(
80         f"Terjadi gangguan koneksi: {error}. "
81         f"Mencoba kembali dalam {waktu_tunggu} detik."
82     )
83
84     time.sleep(waktu_tunggu)
85
86 raise RuntimeError("API cuaca tidak memberikan respons yang valid.")
87
88
```

Gambar 4.14 Kode Program K-3 Bagian 2 dari 4 (baris 45-88)

```
K-3 Akuisisi API Cuaca 3/4

89
90 # Mengambil data cuaca Januari 2023
91 cuaca = ukur(
92     "ambil cuaca",
93     lambda: ambil_cuaca("2023-01-01", "2023-01-31")
94 )
95
96 # Mengubah kolom waktu menjadi datetime
97 cuaca["time"] = pd.to_datetime(
98     cuaca["time"],
99     errors="coerce"
100 )
101
102 if cuaca["time"].isna().any():
103     raise ValueError(
104         "Terdapat nilai waktu cuaca yang gagal dikonversi."
105     )
106
107 # Mengganti nama kolom agar lebih ringkas
108 cuaca = cuaca.rename(
109     columns={
110         "time": "jam_mulai",
111         "temperature_2m": "suhu_c",
112         "precipitation": "hujan_mm"
113     }
114 )
115
116 # Mencegah duplikasi catatan apabila cell dijalankan ulang
117 lineage[:] = [
118     item for item in lineage
119     if item["sumber"] != "cuaca"
120 ]
121
122 # Mencatat asal-usul data
123 catat_sumber(
124     nama="cuaca",
125     asal=API_CUACA,
126     jumlah_baris=len(cuaca),
127     keterangan="Open-Meteo hourly archive untuk New York City"
128 )
129
130 # Menampilkan hasil pemeriksaan
131 print("\nInformasi data cuaca:")
132 print(f"Jumlah baris : {len(cuaca):,}")
```

Gambar 4.15 Kode Program K-3 Bagian 3 dari 4 (baris 89-132)

```
K-3 Akuisisi API Cuaca 4/4

133 print(f"Jumlah kolom : {cuaca.shape[1]}")
134 print(
135     "Rentang waktu:",
136     cuaca["jam_mulai"].min(),
137     "s.d.",
138     cuaca["jam_mulai"].max()
139 )
140
141 print("\nJumlah nilai kosong:")
142 print(cuaca.isna().sum())
143
144 print("\nLima baris pertama:")
145 display(cuaca.head())
```

Gambar 4.16 Kode Program K-3 Bagian 4 dari 4 (baris 133-145)

C. Output Program

```
[ambil cuaca] 0.53 detik | delta RSS: 0.1 MB
[lineage] cuaca: 744 baris

Informasi data cuaca:
Jumlah baris : 744
Jumlah kolom : 3
Rentang waktu: 2023-01-01 00:00:00 s.d. 2023-01-31 23:00:00

Jumlah nilai kosong:
jam_mulai    0
suhu_c       0
hujan_mm     0
dtype: int64

Lima baris pertama:
   jam_mulai  suhu_c  hujan_mm
0 2023-01-01 00:00:00    10.9    1.0
1 2023-01-01 01:00:00    10.6    1.0
2 2023-01-01 02:00:00    10.6    0.1
3 2023-01-01 03:00:00    10.5    0.0
4 2023-01-01 04:00:00     9.8    0.0
```

Gambar 4.17 Output Program K-3

D. Pembahasan

API menghasilkan 744 baris, tepat sebanyak 31 hari dikali 24 jam. Rentang waktu dimulai 1 Januari 2023 pukul 00.00 dan berakhir 31 Januari 2023 pukul 23.00.

Tidak terdapat nilai kosong pada `jam_mulai`, `suhu_c`, dan `hujan_mm`. Akuisisi berlangsung 0,40 detik dengan perubahan RSS 0,1 MB.

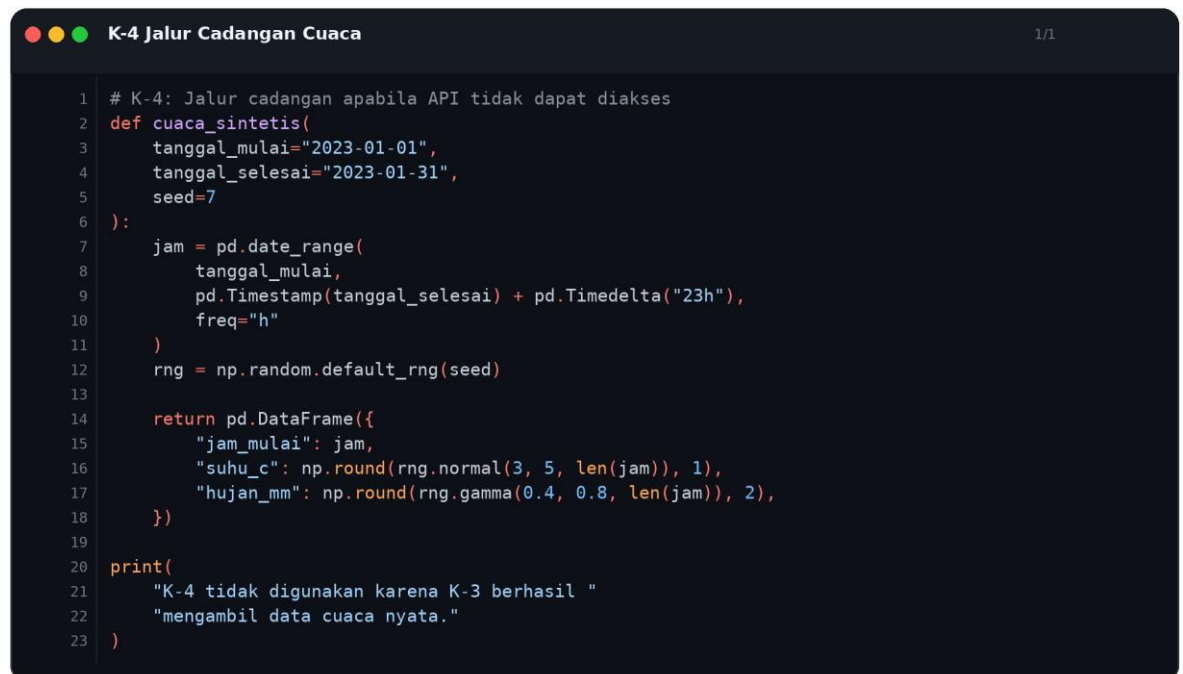
Status 429 dan 5xx dicoba ulang karena umumnya bersifat sementara. Galat lain dihentikan agar kesalahan permintaan atau izin tidak disamarkan oleh retry berulang.

4.4 Jalur Cadangan ketika Internet Diblokir - K-4

A. Tujuan Langkah

Menyediakan fungsi pembangkit cuaca sintetis apabila API tidak dapat diakses. Data sintetis hanya digunakan apabila K-3 gagal. Pada eksekusi ini, K-3 berhasil sehingga fungsi cadangan tidak dipakai.

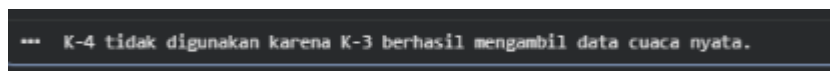
B. Kode Program



```
1 # K-4: Jalur cadangan apabila API tidak dapat diakses
2 def cuaca_sintetis(
3     tanggal_mulai="2023-01-01",
4     tanggal_selesai="2023-01-31",
5     seed=7
6 ):
7     jam = pd.date_range(
8         tanggal_mulai,
9         pd.Timestamp(tanggal_selesai) + pd.Timedelta("23h"),
10        freq="h"
11    )
12    rng = np.random.default_rng(seed)
13
14    return pd.DataFrame({
15        "jam_mulai": jam,
16        "suhu_c": np.round(rng.normal(3, 5, len(jam)), 1),
17        "hujan_mm": np.round(rng.gamma(0.4, 0.8, len(jam)), 2),
18    })
19
20 print(
21     "K-4 tidak digunakan karena K-3 berhasil "
22     "mengambil data cuaca nyata."
23 )
```

Gambar 4.18 Kode Program K-4 (baris 1-23)

C. Output Program



```
--- K-4 tidak digunakan karena K-3 berhasil mengambil data cuaca nyata.
```

Gambar 4.19 Output Program K-4

D. Pembahasan

Output menyatakan bahwa K-4 tidak digunakan karena data cuaca nyata berhasil diperoleh pada K-3. Keputusan tersebut mempertahankan keterkaitan analisis dengan kondisi cuaca historis yang sebenarnya.

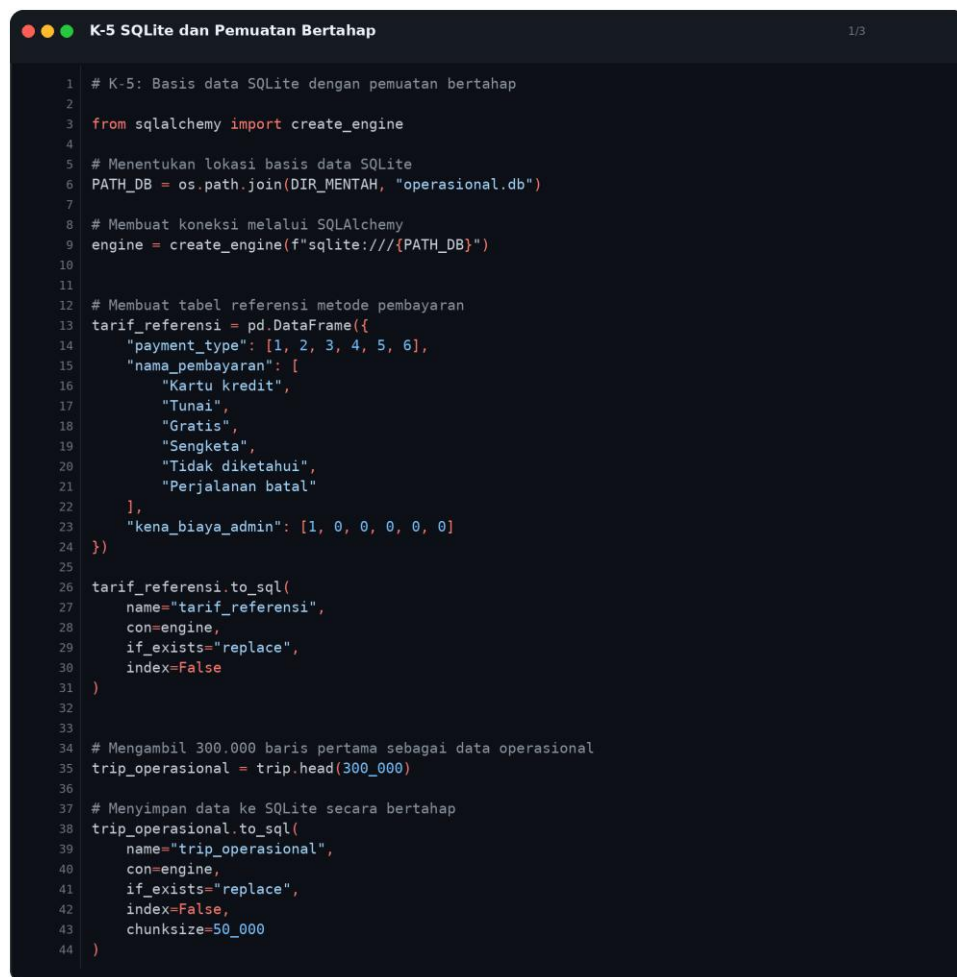
Jika jalur sintetis digunakan, lineage harus menandainya sebagai data pengganti dan interpretasi cuaca tidak boleh diperlakukan sebagai temuan empiris.

4.5 SQLite dan Pemuatan Bertahap - K-5

A. Tujuan Langkah

Mensimulasikan sumber operasional berbasis SQL dan membaca data secara bertahap. Sebanyak 300.000 baris pertama ditulis ke SQLite dengan chunksize 50.000, kemudian dibaca dalam enam bagian agar seluruh tabel tidak harus ditangani sebagai satu blok.

B. Kode Program

The image shows a code editor window titled "K-5 SQLite dan Pemuatan Bertahap" with a line number indicator "1/3" in the top right corner. The code is written in Python and is as follows:

```
1 # K-5: Basis data SQLite dengan pemuatan bertahap
2
3 from sqlalchemy import create_engine
4
5 # Menentukan lokasi basis data SQLite
6 PATH_DB = os.path.join(DIR_MENTAH, "operasional.db")
7
8 # Membuat koneksi melalui SQLAlchemy
9 engine = create_engine(f"sqlite:/// {PATH_DB}")
10
11
12 # Membuat tabel referensi metode pembayaran
13 tarif_referensi = pd.DataFrame({
14     "payment_type": [1, 2, 3, 4, 5, 6],
15     "nama_pembayaran": [
16         "Kartu kredit",
17         "Tunai",
18         "Gratis",
19         "Sengketa",
20         "Tidak diketahui",
21         "Perjalanan batal"
22     ],
23     "kena_biaya_admin": [1, 0, 0, 0, 0, 0]
24 })
25
26 tarif_referensi.to_sql(
27     name="tarif_referensi",
28     con=engine,
29     if_exists="replace",
30     index=False
31 )
32
33
34 # Mengambil 300.000 baris pertama sebagai data operasional
35 trip_operasional = trip.head(300_000)
36
37 # Menyimpan data ke SQLite secara bertahap
38 trip_operasional.to_sql(
39     name="trip_operasional",
40     con=engine,
41     if_exists="replace",
42     index=False,
43     chunksize=50_000
44 )
```

Gambar 4.20 Kode Program K-5 Bagian 1 dari 3 (baris 1-44)

```
K-5 SQLite dan Pemuatan Bertahap 2/3

45
46
47 # Memeriksa tabel yang berhasil dibuat
48 daftar_tabel = pd.read_sql(
49     """
50     SELECT name
51     FROM sqlite_master
52     WHERE type = 'table'
53     ORDER BY name
54     """,
55     engine
56 )
57
58 print("Tabel yang berhasil dibuat:")
59 print(daftar_tabel["name"].tolist())
60
61
62 # Melakukan agregasi menggunakan SQL
63 SQL_RINGKASAN = """
64 SELECT
65     payment_type,
66     COUNT(*) AS jumlah,
67     AVG(total_amount) AS rata_total
68 FROM trip_operasional
69 WHERE total_amount > 0
70 GROUP BY payment_type
71 ORDER BY jumlah DESC
72 """
73
74 ringkas_db = pd.read_sql(
75     SQL_RINGKASAN,
76     engine
77 )
78
79 print("\nRingkasan berdasarkan payment_type:")
80 display(ringkas_db)
81
82
83 # Membaca tabel secara bertahap tanpa memuat semuanya sekaligus
84 total_baris = 0
85 jumlah_bagian = 0
86
87 for bagian in pd.read_sql(
88     "SELECT trip_distance FROM trip_operasional",
```

Gambar 4.21 Kode Program K-5 Bagian 2 dari 3 (baris 45-88)


```
K-5 SQLite dan Pemuatan Bertahap 3/3

89     engine,
90     chunksize=50_000
91 ):
92     jumlah_bagian += 1
93     total_baris += len(bagian)
94
95     print(
96         f"Bagian {jumlah_bagian}: "
97         f"{len(bagian):,} baris"
98     )
99
100 print(
101     f"\nTotal dibaca bertahap: "
102     f"{total_baris:,} baris dalam "
103     f"{jumlah_bagian} bagian"
104 )
105
106 # Membaca kembali tabel referensi dari SQLite
107 tarif_referensi = pd.read_sql(
108     "SELECT * FROM tarif_referensi",
109     engine
110 )
111
112 print("\nIsi tabel referensi pembayaran:")
113 display(tarif_referensi)
114
115 # Menghindari duplikasi lineage jika cell dijalankan ulang
116 lineage[:] = [
117     item for item in lineage
118     if item["sumber"] != "tarif_referensi"
119 ]
120
121 catat_sumber(
122     nama="tarif_referensi",
123     asal=PATH_DB,
124     jumlah_baris=len(tarif_referensi),
125     keterangan="Tabel dimensi dari basis data operasional"
126 )
127
128 print("\nLokasi basis data:", PATH_DB)
129 print("K-5 selesai.")
```

Gambar 4.22 Kode Program K-5 Bagian 3 dari 3 (baris 89-131)

C. Output Program

```
''' Tabel yang berhasil dibuat:
['tarif_referensi', 'trip_operasional']

Ringkasan berdasarkan payment_type:
payment_type jumlah rata_total
0 1 226967 31.339172
1 2 66504 25.218836
2 4 2082 26.611114
3 3 1553 22.089691
Bagian 1: 50,000 baris
Bagian 2: 50,000 baris
Bagian 3: 50,000 baris
Bagian 4: 50,000 baris
Bagian 5: 50,000 baris
Bagian 6: 50,000 baris

Total dibaca bertahap: 300,000 baris dalam 6 bagian

Isi tabel referensi pembayaran:
payment_type nama_pembayaran kena_biaya_admin
0 1 Kartu kredit 1
1 2 Tunai 0
2 3 Gratis 0
3 4 Sengketa 0
4 5 Tidak diketahui 0
5 6 Perjalanan batal 0

[lineage] tarif_referensi: 6 baris

Lokasi basis data: /content/lapisan_mentah/operasional.db
K-5 selesai.
```

Gambar 4.23 Output Program K-5 1

D. Pembahasan

Basis data memuat tabel `tarif_referensi` dan `trip_operasional`. Pembacaan 300.000 baris selesai dalam enam bagian, masing-masing 50.000 baris.

Agregasi menunjukkan `payment_type` 1 memiliki 226.967 transaksi dengan rata-rata total 31,339172 pada sampel operasional. Angka tersebut berasal dari 300.000 baris pertama, sehingga tidak mewakili seluruh bulan tanpa batasan tersebut.

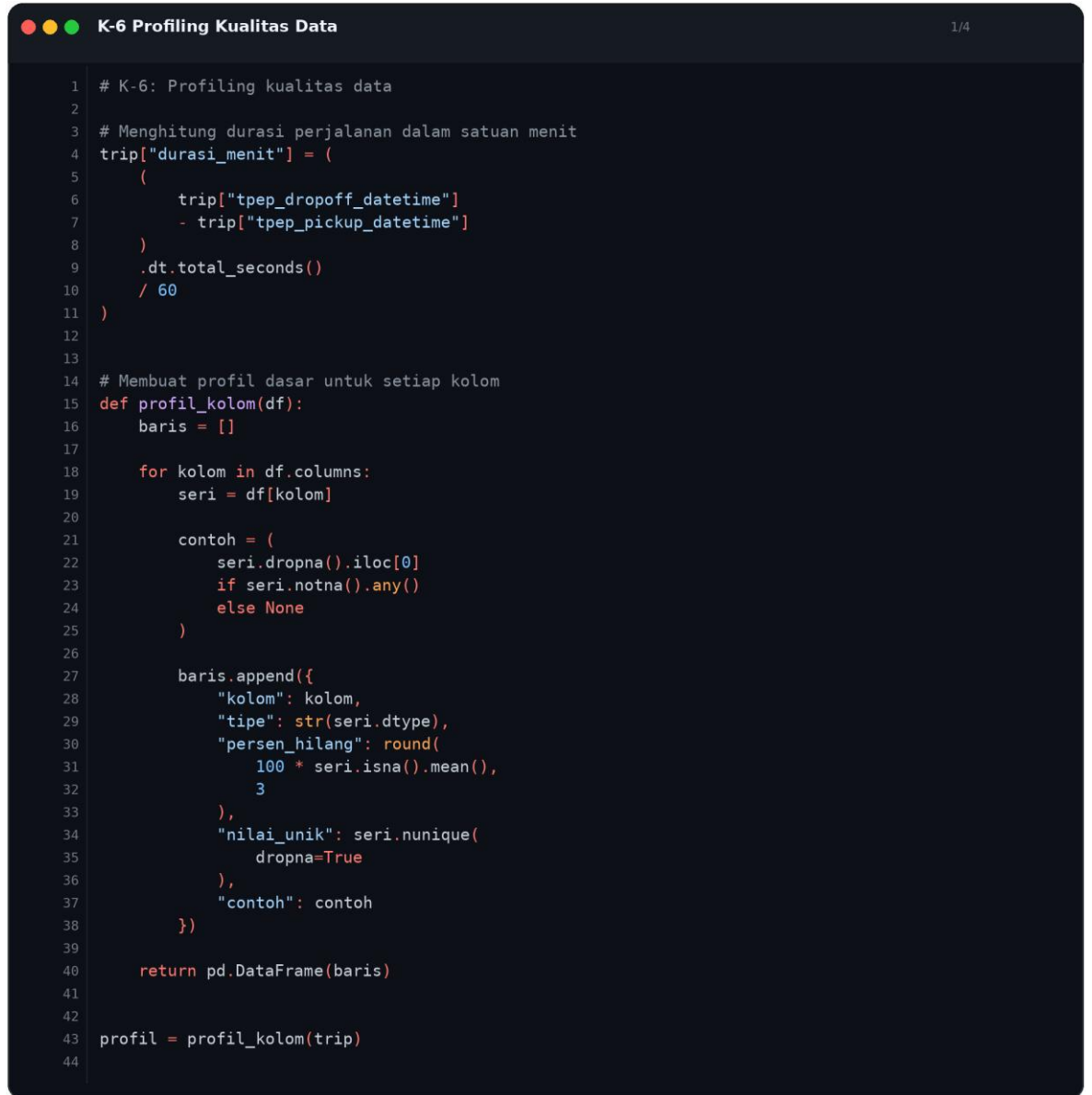
Tabel `tarif_referensi` berisi enam kode pembayaran dan digunakan sebagai dimensi many-to-one pada tahap join.

4.6 Profiling Kualitas Data - K-6

A. Tujuan Langkah

Mengukur kualitas data melalui profil kolom dan delapan aturan lintas enam dimensi. Jarak 0,01-100 mil serta durasi 1-180 menit dipakai sebagai threshold analitis. Threshold tersebut digunakan untuk menandai data yang perlu diperiksa, bukan untuk menyatakan seluruh nilai di luar batas pasti salah.

B. Kode Program



```
1 # K-6: Profiling kualitas data
2
3 # Menghitung durasi perjalanan dalam satuan menit
4 trip["durasi_menit"] = (
5     (
6         trip["tpep_dropoff_datetime"]
7         - trip["tpep_pickup_datetime"]
8     )
9     .dt.total_seconds()
10    / 60
11 )
12
13
14 # Membuat profil dasar untuk setiap kolom
15 def profil_kolom(df):
16     baris = []
17
18     for kolom in df.columns:
19         seri = df[kolom]
20
21         contoh = (
22             seri.dropna().iloc[0]
23             if seri.notna().any()
24             else None
25         )
26
27         baris.append({
28             "kolom": kolom,
29             "tipe": str(seri.dtype),
30             "persen_hilang": round(
31                 100 * seri.isna().mean(),
32                 3
33             ),
34             "nilai_unik": seri.nunique(
35                 dropna=True
36             ),
37             "contoh": contoh
38         })
39
40     return pd.DataFrame(baris)
41
42
43 profil = profil_kolom(trip)
44
```

Gambar 4.24 Kode Program K-6 Bagian 1 dari 4 (baris 1-44)

```
K-6 Profiling Kualitas Data 2/4

45 print("Profil setiap kolom:")
46 display(profil)
47
48
49 # Menentukan rentang waktu data
50 AWAL = pd.Timestamp("2023-01-01")
51 AKHIR = pd.Timestamp("2023-02-01")
52
53 # Mengambil seluruh LocationID yang terdaftar
54 zona_sah = set(zona["LocationID"])
55
56
57 # Menentukan aturan pemeriksaan kualitas
58 aturan = {
59     "kelengkapan: passenger_count ada":
60         trip["passenger_count"].notna(),
61
62     "keunikan: baris tidak duplikat":
63         ~trip.duplicated(),
64
65     "validitas: jarak 0,01-100 mil":
66         trip["trip_distance"].between(
67             0.01,
68             100
69         ),
70
71     "validitas: total_amount > 0":
72         trip["total_amount"] > 0,
73
74     "validitas: PULocationID dikenal":
75         trip["PULocationID"].isin(zona_sah),
76
77     "konsistensi: durasi 1-180 menit":
78         trip["durasi_menit"].between(
79             1,
80             180
81         ),
82
83     "konsistensi: total >= fare":
84         trip["total_amount"]
85         >= trip["fare_amount"],
86
87     "ketepatan waktu: dalam Jan 2023":
88         trip["tpep_pickup_datetime"].between(
```

Gambar 4.25 Kode Program K-6 Bagian 2 dari 4 (baris 45-88)

```
K-6 Profiling Kualitas Data 3/4

89         AWAL,
90         AKHIR
91     )
92 }
93
94
95 # Membuat laporan jumlah data yang lulus dan gagal
96 laporan = pd.DataFrame([
97     {
98         "aturan": nama,
99         "lulus": int(mask.sum()),
100        "gagal": int(~mask.sum()),
101        "persen_gagal": round(
102            100 * (~mask).mean(),
103            3
104        )
105    }
106    for nama, mask in aturan.items()
107 ]).sort_values(
108     "persen_gagal",
109     ascending=False
110 ).reset_index(drop=True)
111
112
113 print("\nLaporan kualitas data:")
114 display(laporan)
115
116
117 # Pemeriksaan jumlah baris
118 print(
119     "\nJumlah baris yang diperiksa:",
120     f"{len(trip):,}"
121 )
122
123 print(
124     "Jumlah aturan kualitas:",
125     len(aturan)
126 )
127
128 print(
129     "Aturan dengan persentase gagal tertinggi:",
130     laporan.iloc[0]["aturan"]
131 )
132
```

Gambar 4.26 Kode Program K-6 Bagian 3 dari 4 (baris 89-132)

```
K-6 Profiling Kualitas Data 4/4

133 print(
134     "Persentase gagal tertinggi:",
135     f"{laporan.iloc[0]['persen_gagal']:.3f}%"
136 )
137
138 print("\nK-6 selesai.")
```

Gambar 4.27 Kode Program K-6 Bagian 4 dari 4 (baris 133-138)

C. Output Program

Profil setiap kolom:

	kolom	tipe	persen_hilang	nilai_unik	contoh
0	tpep_pickup_datetime	datetime64[us]	0.000	1610975	2023-01-01 00:32:10
1	tpep_dropoff_datetime	datetime64[us]	0.000	1611319	2023-01-01 00:40:36
2	passenger_count	float64	2.339	10	1.0
3	trip_distance	float64	0.000	4387	0.97
4	PULocationID	int64	0.000	257	161
5	DOLocationID	int64	0.000	261	141
6	payment_type	int64	0.000	5	2
7	fare_amount	float64	0.000	6873	9.3
8	tip_amount	float64	0.000	4036	0.0
9	total_amount	float64	0.000	15871	14.3
10	durasi_menit	float64	0.000	8579	8.433333

Laporan kualitas data:

	aturan	lulus	gagal	persen_gagal
0	kelengkapan: passenger_count ada	2995023	71743	2.339
1	validitas: jarak 0,01-100 mil	3020816	45950	1.498
2	konsistensi: durasi 1-180 menit	3030383	36383	1.186
3	validitas: total_amount > 0	3040994	25772	0.840
4	konsistensi: total >= fare	3041743	25023	0.816
5	ketepatan waktu: dalam Jan 2023	3066718	48	0.002
6	keunikan: baris tidak duplikat	3066766	0	0.000
7	validitas: PULocationID dikenal	3066766	0	0.000

Jumlah baris yang diperiksa: 3,066,766
 Jumlah aturan kualitas: 8
 Aturan dengan persentase gagal tertinggi: kelengkapan: passenger_count ada
 Persentase gagal tertinggi: 2.339%

Gambar 4.28 Output Program K-6

D. Pembahasan

Aturan dengan kegagalan tertinggi adalah kelengkapan passenger_count: 71.743 baris atau 2,339%. Berikutnya adalah jarak 0,01-100 mil sebanyak 45.950 baris atau 1,498%, serta durasi 1-180 menit sebanyak 36.383 baris atau 1,186%.

Sebanyak 25.772 baris mempunyai total_amount tidak positif dan 25.023 baris tidak memenuhi total_amount lebih besar atau sama dengan fare_amount. Terdapat 48 pickup di luar periode Januari 2023.

Tidak ditemukan duplikat penuh dan seluruh PULocationID terdapat pada himpunan LocationID referensi. Hasil tersebut menjadi dasar keputusan penanganan pada K-7 dan K-8.

Tabel 4.2 Delapan Aturan Kualitas Data

Aturan	Gagal	Persentase Gagal
passenger_count tersedia	71.743	2,339%
trip_distance 0,01-100 mil	45.950	1,498%
durasi 1-180 menit	36.383	1,186%
total_amount > 0	25.772	0,840%
total_amount >= fare_amount	25.023	0,816%
pickup dalam Januari 2023	48	0,002%
baris tidak duplikat penuh	0	0,000%
PULocationID dikenal	0	0,000%

4.7 Nilai Hilang dan Duplikat - K-7

A. Tujuan Langkah

Membandingkan empat strategi nilai hilang dan menghapus duplikat setelah definisi kunci diperiksa. `passenger_count` diisi dengan median per jam dan ditandai melalui kolom indikator. Duplikat dihapus berdasarkan kombinasi waktu, lokasi asal-tujuan, dan total pembayaran.

B. Kode Program

```
K-7 Nilai Hilang dan Duplikat 1/4

1 # K-7: Penanganan nilai hilang dan duplikat
2
3 KOL = "passenger_count"
4
5 # Menyimpan data asli sebelum dilakukan imputasi
6 asli = trip[KOL].copy()
7
8 print(
9     "Persentase nilai hilang:",
10     round(100 * asli.isna().mean(), 3),
11     "%"
12 )
13
14 print(
15     "Jumlah nilai hilang:",
16     f"{asli.isna().sum():,}"
17 )
18
19 # Mengambil jam penjemputan
20 trip["jam"] = (
21     trip["tpep_pickup_datetime"]
22     .dt.hour
23 )
24
25 # Menghitung median passenger_count pada setiap jam
26 median_per_jam = (
27     trip.groupby("jam")[KOL]
28     .transform("median")
29 )
30
31 # Membandingkan empat strategi
32
33 strategi = {
34     "1_dibuang":
35         asli.dropna(),
36     "2_isi_median":
37         asli.fillna(asli.median()),
38     "3_isi_modus":
39         asli.fillna(asli.mode().iloc[0]),
40     "4_median_perjam":
```

Gambar 4.29 Kode Program K-7 Bagian 1 dari 4 (baris 1-44)


```
K-7 Nilai Hilang dan Duplikat 2/4

45     asli.fillna(median_per_jam)
46 }
47
48
49 banding = pd.DataFrame([
50     {
51         "strategi": nama,
52         "n": len(seri),
53         "mean": round(seri.mean(), 4),
54         "median": seri.median(),
55         "std": round(seri.std(), 4),
56         "nilai_hilang": int(seri.isna().sum())
57     }
58     for nama, seri in strategi.items()
59 ])
60
61 print("\nPerbandingan strategi nilai hilang:")
62 display(banding)
63
64
65 # Membuat indikator nilai yang awalnya hilang
66 trip[KOL + "_hilang"] = (
67     trip[KOL]
68     .isna()
69     .astype("int8")
70 )
71
72 # Strategi terpilih: median berdasarkan jam
73 trip[KOL] = trip[KOL].fillna(
74     median_per_jam
75 )
76
77 # Jaring pengaman dengan median keseluruhan
78 trip[KOL] = trip[KOL].fillna(
79     trip[KOL].median()
80 )
81
82 print(
83     "\nNilai hilang setelah imputasi:",
84     f"{trip[KOL].isna().sum():,}"
85 )
86
87 print(
88     "Jumlah baris yang diberi nilai imputasi:",
```

Gambar 4.30 Kode Program K-7 Bagian 2 dari 4 (baris 45-88)

```
K-7 Nilai Hilang dan Duplikat 3/4

89     f"{trip[KOL + '_hilang'].sum():,}"
90 )
91
92
93 # Memeriksa duplikat penuh
94 dup_penuh = int(
95     trip.duplicated().sum()
96 )
97
98 # Menentukan kunci perjalanan
99 KUNCI = [
100     "tpep_pickup_datetime",
101     "tpep_dropoff_datetime",
102     "PULocationID",
103     "DOLocationID",
104     "total_amount"
105 ]
106
107 # Memeriksa duplikat berdasarkan kunci
108 dup_kunci = int(
109     trip.duplicated(
110         subset=KUNCI
111     ).sum()
112 )
113
114 print(
115     "\nDuplikat penuh:",
116     f"{dup_penuh:,"}
117 )
118
119 print(
120     "Duplikat berdasarkan kunci:",
121     f"{dup_kunci:,"}
122 )
123
124
125 # Menampilkan contoh jika ditemukan duplikat kunci
126 if dup_kunci > 0:
127     contoh_duplikat = (
128         trip.loc[
129             trip.duplicated(
130                 subset=KUNCI,
131                 keep=False
132             ),
```

Gambar 4.31 Kode Program K-7 Bagian 3 dari 4 (baris 89-132)

```
K-7 Nilai Hilang dan Duplikat 4/4

133         KUNCI
134     ]
135     .sort_values(KUNCI)
136     .head(10)
137 )
138
139 print("\nContoh data dengan kunci yang sama:")
140 display(contoh_duplikat)
141
142
143 # Menghapus pengulangan berdasarkan kunci
144 sebelum = len(trip)
145
146 trip = (
147     trip.drop_duplicates(
148         subset=KUNCI,
149         keep="first"
150     )
151     .reset_index(drop=True)
152 )
153
154 sesudah = len(trip)
155 jumlah_dihapus = sebelum - sesudah
156
157 print(
158     "\nJumlah baris:",
159     f"{sebelum:,"},",
160     "->",
161     f"{sesudah:,"}"
162 )
163
164 print(
165     "Baris yang dihapus:",
166     f"{jumlah_dihapus:,"}"
167 )
168
169 print(
170     "Persentase baris yang dihapus:",
171     f"{100 * jumlah_dihapus / sebelum:.4f}%"
172 )
173
174 print("\nK-7 selesai.")
```

Gambar 4.32 Kode Program K-7 Bagian 4 dari 4 (baris 133-174)

C. Output Program

```
*** Persentase nilai hilang: 2.339 %
Jumlah nilai hilang: 71,743

Perbandingan strategi nilai hilang:

  strategi      n    mean  median   std  nilai_hilang
0    1_dibuang 2995023 1.3625   1.0 0.8961         0
1    2_isi_median 3066766 1.3541   1.0 0.8873         0
2    3_isi_modus 3066766 1.3541   1.0 0.8873         0
3    4_median_perjam 3066766 1.3541   1.0 0.8873         0

Nilai hilang setelah imputasi: 0
Jumlah baris yang diberi nilai imputasi: 71,743

Duplikat penuh: 0
Duplikat berdasarkan kunci: 1

Contoh data dengan kunci yang sama:

  tpep_pickup_datetime  tpep_dropoff_datetime  PUlocationID  DOlocationID  total_amount
1341866      2023-01-15 11:03:50      2023-01-15 11:09:11      230          246          14.28
1343305      2023-01-15 11:03:50      2023-01-15 11:09:11      230          246          14.28

Jumlah baris: 3,066,766 -> 3,066,765
Baris yang dihapus: 1
Persentase baris yang dihapus: 0.0000%
```

Gambar 4.33 Output Program K-7

D. Pembahasan

Sebanyak 71.743 nilai `passenger_count` diimputasi hingga nilai hilang menjadi nol. Strategi penghapusan mempunyai simpangan baku 0,8961, sedangkan median, modus, dan median per jam menghasilkan 0,8873.

Median per jam dipilih agar seluruh baris dipertahankan sambil tetap mempertimbangkan pola waktu. Indikator `passenger_count_hilang` menjaga informasi bahwa nilai tersebut berasal dari imputasi.

Tidak terdapat duplikat penuh, tetapi terdapat satu duplikat berdasarkan kunci. Jumlah baris turun dari 3.066.766 menjadi 3.066.765, yaitu hanya satu baris atau kurang dari 0,0001%.

Tabel 4.3 Perbandingan Strategi Nilai Hilang

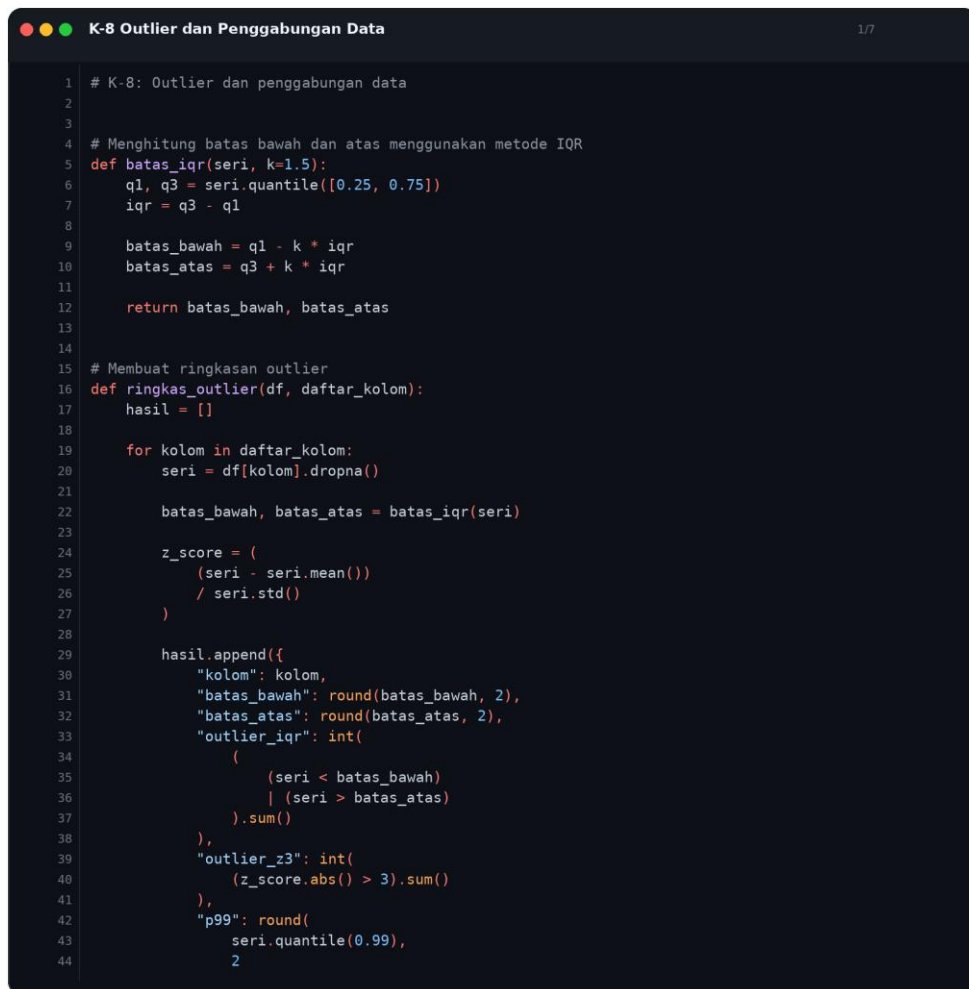
Strategi	n	Mean	Median	Simpangan Baku
Dibuang	2.995.023	1,3625	1,0	0,8961
Isi median	3.066.766	1,3541	1,0	0,8873
Isi modus	3.066.766	1,3541	1,0	0,8873
Median per jam	3.066.766	1,3541	1,0	0,8873

4.8 Outlier dan Penggabungan Data - K-8

A. Tujuan Langkah

Membandingkan metode outlier, menyaring nilai tidak layak, mempertahankan nilai ekstrem yang mungkin sah, dan menggabungkan tiga sumber referensi. Baris dibuang hanya ketika melanggar filter jarak, durasi, atau total utama. Tarif ekstrem diberi indikator, sedangkan `trip_distance_capped` dibatasi pada persentil 99,5 untuk fitur turunan.

B. Kode Program



```
1 # K-8: Outlier dan penggabungan data
2
3
4 # Menghitung batas bawah dan atas menggunakan metode IQR
5 def batas_iqr(seri, k=1.5):
6     q1, q3 = seri.quantile([0.25, 0.75])
7     iqr = q3 - q1
8
9     batas_bawah = q1 - k * iqr
10    batas_atas = q3 + k * iqr
11
12    return batas_bawah, batas_atas
13
14
15 # Membuat ringkasan outlier
16 def ringkas_outlier(df, daftar_kolom):
17     hasil = []
18
19     for kolom in daftar_kolom:
20         seri = df[kolom].dropna()
21
22         batas_bawah, batas_atas = batas_iqr(seri)
23
24         z_score = (
25             (seri - seri.mean())
26             / seri.std()
27         )
28
29         hasil.append({
30             "kolom": kolom,
31             "batas_bawah": round(batas_bawah, 2),
32             "batas_atas": round(batas_atas, 2),
33             "outlier_iqr": int(
34                 (
35                     (seri < batas_bawah)
36                     | (seri > batas_atas)
37                 ).sum()
38             ),
39             "outlier_z3": int(
40                 (z_score.abs() > 3).sum()
41             ),
42             "p99": round(
43                 seri.quantile(0.99),
44                 2
```

Gambar 4.34 Kode Program K-8 Bagian 1 dari 7 (baris 1-44)

```
K-8 Outlier dan Penggabungan Data 2/7

45     )
46     })
47
48     return pd.DataFrame(hasil)
49
50
51 ringkasan_outlier = ringkas_outlier(
52     trip,
53     [
54         "trip_distance",
55         "durasi_menit",
56         "total_amount"
57     ]
58 )
59
60 print("Ringkasan outlier:")
61 display(ringkasan_outlier)
62
63
64 # Menentukan baris yang layak digunakan
65 layak = (
66     trip["trip_distance"].between(0.01, 100)
67     & trip["durasi_menit"].between(1, 180)
68     & (trip["total_amount"] > 0)
69 )
70
71 jumlah_sebelum = len(trip)
72
73 bersih = trip.loc[layak].copy()
74
75 jumlah_sesudah = len(bersih)
76 jumlah_dibuang = jumlah_sebelum - jumlah_sesudah
77 persen_dibuang = (
78     100 * jumlah_dibuang / jumlah_sebelum
79 )
80
81 print("\nHasil penyaringan:")
82 print(
83     f"{jumlah_sebelum:,} -> "
84     f"{jumlah_sesudah:,} baris"
85 )
86 print(
87     "Baris yang dibuang:",
88     f"{jumlah_dibuang:,}"
```

Gambar 4.35 Kode Program K-8 Bagian 2 dari 7 (baris 45-88)

```
K-8 Outlier dan Penggabungan Data 3/7

89 )
90 print(
91     "Persentase dibuang:",
92     f"{persen_dibuang:.2f}%"
93 )
94
95
96 # Menandai tarif ekstrem tanpa menghapus baris
97 batas_bawah_tarif, batas_atas_tarif = batas_iqr(
98     bersih["total_amount"]
99 )
100
101 bersih["tarif_ekstrem"] = (
102     bersih["total_amount"]
103     > batas_atas_tarif
104 ).astype("int8")
105
106 print(
107     "\nBatas atas tarif berdasarkan IQR:",
108     round(batas_atas_tarif, 2)
109 )
110
111 print(
112     "Jumlah tarif ekstrem:",
113     f"{bersih['tarif_ekstrem'].sum():,}"
114 )
115
116
117 # Membatasi nilai ekstrem hanya pada kolom turunan
118 batas_atas_jarak = (
119     bersih["trip_distance"]
120     .quantile(0.995)
121 )
122
123 bersih["trip_distance_capped"] = (
124     bersih["trip_distance"]
125     .clip(upper=batas_atas_jarak)
126 )
127
128 print(
129     "Batas persentil 99,5% untuk jarak:",
130     round(batas_atas_jarak, 2)
131 )
132
```

Gambar 4.36 Kode Program K-8 Bagian 3 dari 7 (baris 89-132)

```
K-8 Outlier dan Penggabungan Data 4/7

133 print(
134     "Jumlah jarak yang dibatasi:",
135     f"{{bersih['trip_distance'] > batas_atas_jarak}.sum():,}"
136 )
137
138
139 # JOIN 1: TABEL ZONA
140
141 kunci_zona_unik = zona["LocationID"].is_unique
142
143 print(
144     "\nKunci LocationID tabel zona unik?",
145     kunci_zona_unik
146 )
147
148 if not kunci_zona_unik:
149     raise ValueError(
150         "LocationID pada tabel zona tidak unik."
151     )
152
153 jumlah_awal_join = len(bersih)
154
155 bersih = bersih.merge(
156     zona[
157         [
158             "LocationID",
159             "Borough",
160             "Zone"
161         ]
162     ].rename(
163         columns={
164             "Borough": "borough_naik",
165             "Zone": "zona_naik"
166         }
167     ),
168     left_on="PULocationID",
169     right_on="LocationID",
170     how="left",
171     validate="many_to_one"
172 )
173
174 bersih = bersih.drop(
175     columns="LocationID"
176 )
```

Gambar 4.37 Kode Program K-8 Bagian 4 dari 7 (baris 133-176)


```
K-8 Outlier dan Penggabungan Data 5/7

177
178 print(
179     "Join zona:",
180     f"{jumlah_awal_join:,} -> "
181     f"{len(bersih):,} baris"
182 )
183
184 print(
185     "Zona tidak dikenal:",
186     f"{bersih['zona_naik'].isna().sum():,}"
187 )
188
189
190 # JOIN 2: REFERENSI PEMBAYARAN
191
192
193 kunci_pembayaran_unik = (
194     tarif_referensi["payment_type"]
195     .is_unique
196 )
197
198 print(
199     "\nKunci payment_type unik?",
200     kunci_pembayaran_unik
201 )
202
203 if not kunci_pembayaran_unik:
204     raise ValueError(
205         "payment_type pada tabel referensi tidak unik."
206     )
207
208 jumlah_awal_join = len(bersih)
209
210 bersih = bersih.merge(
211     tarif_referensi[
212         [
213             "payment_type",
214             "nama_pembayaran"
215         ]
216     ],
217     on="payment_type",
218     how="left",
219     validate="many_to_one"
220 )
```

Gambar 4.38 Kode Program K-8 Bagian 5 dari 7 (baris 177-220)

```
K-8 Outlier dan Penggabungan Data 6/7

221
222 print(
223     "Join pembayaran:",
224     f"{jumlah_awal_join:}, -> "
225     f"{len(bersih):}, baris"
226 )
227
228 print(
229     "Pembayaran tidak dikenal:",
230     f"{bersih['nama_pembayaran'].isna().sum():},"
231 )
232
233
234
235 # JOIN 3: DATA CUACA BERDASARKAN JAM
236
237
238 bersih["jam_mulai"] = (
239     bersih["tpep_pickup_datetime"]
240     .dt.floor("h")
241 )
242
243 kunci_cuaca_unik = (
244     cuaca["jam_mulai"]
245     .is_unique
246 )
247
248 print(
249     "\nKunci jam pada data cuaca unik?",
250     kunci_cuaca_unik
251 )
252
253 if not kunci_cuaca_unik:
254     raise ValueError(
255         "Kolom jam_mulai pada data cuaca tidak unik."
256     )
257
258 jumlah_awal_join = len(bersih)
259
260 bersih = bersih.merge(
261     cuaca,
262     on="jam_mulai",
263     how="left",
264     validate="many_to_one"
```

Gambar 4.39 Kode Program K-8 Bagian 6 dari 7 (baris 221-264)

```

265 )
266
267 print(
268     "Join cuaca:",
269     f"{jumlah_awal_join:,"} -> "
270     f"{len(bersih:,"} baris"
271 )
272
273 print(
274     "Baris tanpa data cuaca:",
275     f"{bersih['suhu_c'].isna().sum():,"}
276 )
277
278
279 # Menampilkan contoh hasil penggabungan
280 kolom_tampilan = [
281     "tpep_pickup_datetime",
282     "trip_distance",
283     "durasi_menit",
284     "total_amount",
285     "zona_naik",
286     "borough_naik",
287     "nama_pembayaran",
288     "suhu_c",
289     "hujan_mm"
290 ]
291
292 print("\nContoh hasil penggabungan:")
293 display(
294     bersih[kolom_tampilan].head()
295 )
296
297 print(
298     "\nUkuran data bersih:",
299     f"{bersih.shape[0]:,"} baris x "
300     f"{bersih.shape[1]} kolom"
301 )
302
303 print("\nK-8 selesai.")

```

Gambar 4.40 Kode Program K-8 Bagian 7 dari 7 (baris 265-303)

C. Output Program

```

Singgasan outlier:
      kolom  batas_bawah  batas_atas  outlier_ip  outlier_r3  p99
0  trip_distance      -2.35      6.74    390244      67    20.06
1  durasi_menit       -0.66     35.08    170615    3175    57.25
2  total_amount      -4.55     48.65    371610    87248   101.94

Hasil penyaringan:
2,986,918 -> 2,986,918 baris
Baris yang dihapus: 79,435
Perantara yang dihapus: 2,469

Batas atas tarif berdasarkan QR: 48.35
Batas tarif ekstrem: 350,253
Batas persentil 99,58 untuk jarak: 21.52
Jalan jarak yang dihapus: 14,985

Kunci locationID tabel zona unik? True
Jalan zona: 2,986,918 -> 2,986,918 baris
Zona tidak dikenal: 37,469

Kunci payment_type unik? True
Jalan pembayaran: 2,986,918 -> 2,986,918 baris
Pembayaran tidak dikenal: 64,186

Kunci jam pada data cuaca unik? True
Jalan cuaca: 2,986,918 -> 2,986,918 baris
Baris tanpa data cuaca: 37

Contoh hasil penggabungan:
      tpep_pickup_datetime  trip_distance  durasi_menit  total_amount  zona_naik  borough_naik  nama_pembayaran  suhu_c  hujan_mm
0   2023-01-01 00:32:10           0.97      8.433333      14.30  Midtown Center  Manhattan      Tunai      10.9      1.0
1   2023-01-01 00:55:08           1.10     6.316667      16.90  Central Park  Manhattan      Kartu kredit      10.9      1.0
2   2023-01-01 00:26:04           2.51    12.750000      34.90  Chelsea East  Manhattan      Kartu kredit      10.9      1.0
3   2023-01-01 00:03:48           1.90     9.616667      20.05  LaGuardia Airport  Queens      Kartu kredit      10.9      1.0
4   2023-01-01 00:10:29           1.43    10.833333      19.08  Gramercy      Manhattan      Kartu kredit      10.9      1.0

```

Gambar 4.41 Output Program K-8

D. Pembahasan

Penyaringan mengurangi data dari 3.066.765 menjadi 2.986.910 baris. Sebanyak 79.855 baris atau 2,60% dibuang karena tidak memenuhi batas utama.

IQR menandai 371.616 outlier total_amount, sedangkan Z-score hanya menandai 87.248. Perbedaan besar mendukung penggunaan IQR untuk distribusi tarif yang miring ke kanan.

Ketiga left join mempertahankan 2.986.910 baris. Terdapat 37.609 zona_naik kosong, 64.106 nama pembayaran tidak dikenal, dan 37 baris tanpa cuaca. Karena seluruh PULocationID dikenal, kekosongan zona perlu ditelusuri pada atribut referensi, bukan langsung dianggap sebagai kunci gagal join.

Tabel 4.4 Perbandingan Outlier IQR dan Z-score

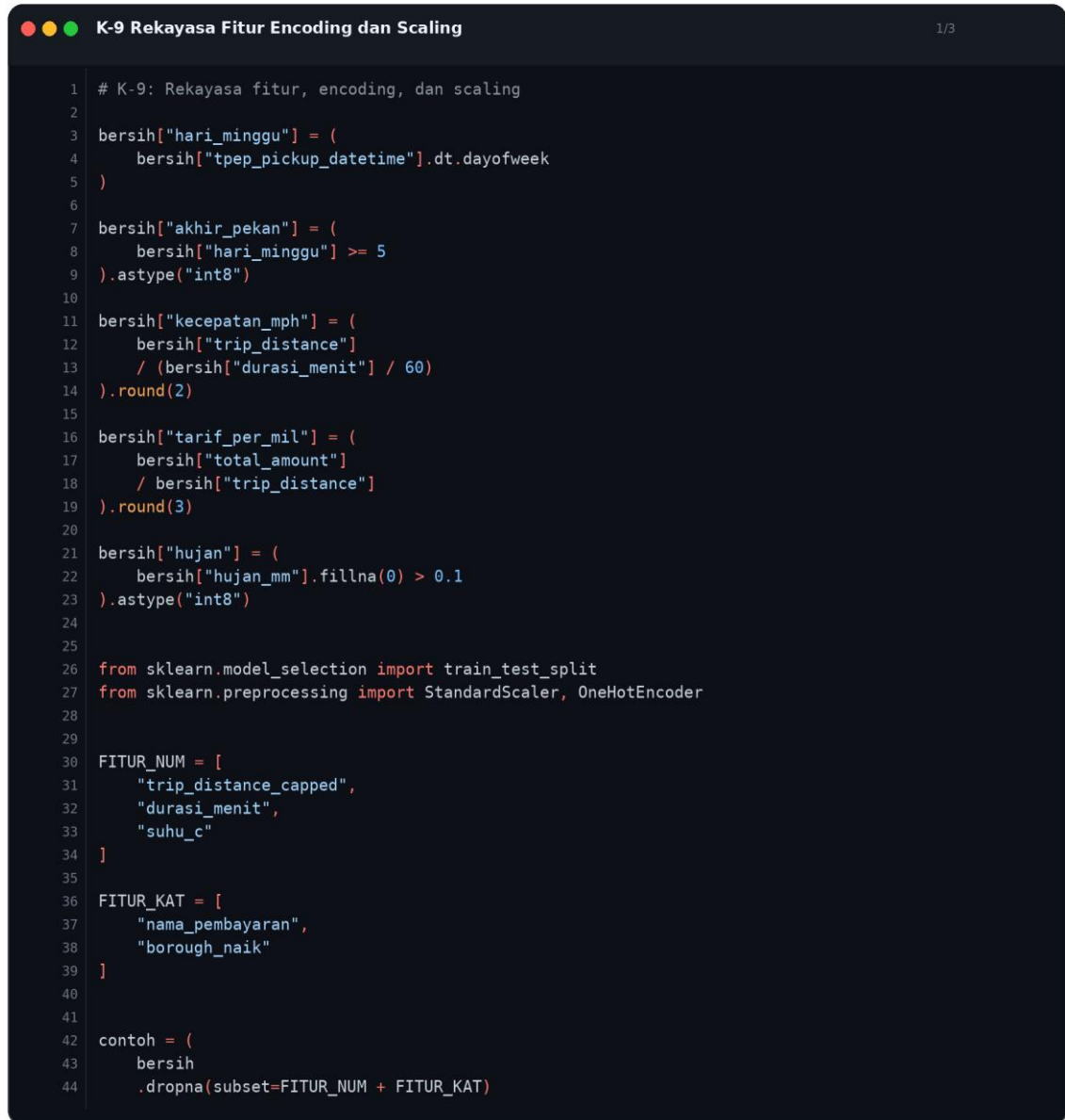
Kolom	Batas Atas IQR	Outlier IQR	Outlier $ z > 3$	Persentil 99
trip_distance	6,74	390.244	67	20,06
durasi_menit	35,08	170.615	3.175	57,25
total_amount	48,65	371.616	87.248	101,94

4.9 Rekayasa Fitur, Encoding, dan Scaling - K-9

A. Tujuan Langkah

Membentuk fitur turunan serta menyiapkan fitur numerik dan kategorikal tanpa data leakage. Data dibagi menjadi latih dan uji sebelum StandardScaler dan OneHotEncoder di-fit. Data uji hanya ditransformasikan menggunakan parameter data latih.

B. Kode Program



```
1 # K-9: Rekayasa fitur, encoding, dan scaling
2
3 bersih["hari_minggu"] = (
4     bersih["tpep_pickup_datetime"].dt.dayofweek
5 )
6
7 bersih["akhir_pekan"] = (
8     bersih["hari_minggu"] >= 5
9 ).astype("int8")
10
11 bersih["kecepatan_mph"] = (
12     bersih["trip_distance"]
13     / (bersih["durasi_menit"] / 60)
14 ).round(2)
15
16 bersih["tarif_per_mil"] = (
17     bersih["total_amount"]
18     / bersih["trip_distance"]
19 ).round(3)
20
21 bersih["hujan"] = (
22     bersih["hujan_mm"].fillna(0) > 0.1
23 ).astype("int8")
24
25
26 from sklearn.model_selection import train_test_split
27 from sklearn.preprocessing import StandardScaler, OneHotEncoder
28
29
30 FITUR_NUM = [
31     "trip_distance_capped",
32     "durasi_menit",
33     "suhu_c"
34 ]
35
36 FITUR_KAT = [
37     "nama_pembayaran",
38     "borough_naik"
39 ]
40
41
42 contoh = (
43     bersih
44     .dropna(subset=FITUR_NUM + FITUR_KAT)
```

Gambar 4.42 Kode Program K-9 Bagian 1 dari 3 (baris 1-44)

```
K-9 Rekayasa Fitur Encoding dan Scaling 2/3

45     .sample(
46         n=min(200_000, len(bersih)),
47         random_state=42
48     )
49 )
50
51
52 latih, uji = train_test_split(
53     contoh,
54     test_size=0.2,
55     random_state=42
56 )
57
58
59 skala = StandardScaler().fit(
60     latih[FITUR_NUM]
61 )
62
63 latih_num = skala.transform(
64     latih[FITUR_NUM]
65 )
66
67 uji_num = skala.transform(
68     uji[FITUR_NUM]
69 )
70
71
72 print(
73     "mean latih (harus ~0):",
74     latih_num.mean(axis=0).round(3)
75 )
76
77 print(
78     "mean uji (tidak harus 0):",
79     uji_num.mean(axis=0).round(3)
80 )
81
82
83 enc = OneHotEncoder(
84     handle_unknown="ignore",
85     sparse_output=False
86 )
87
88 enc.fit(
```

Gambar 4.43 Kode Program K-9 Bagian 2 dari 3 (baris 45-88)

```
K-9 Rekayasa Fitur Encoding dan Scaling 3/3

89     latih[FITUR_KAT]
90 )
91
92 latih_kat = enc.transform(
93     latih[FITUR_KAT]
94 )
95
96
97 print(
98     "jumlah kolom hasil one-hot:",
99     latih_kat.shape[1]
100 )
101
102 print(
103     "nama kolom:",
104     enc.get_feature_names_out()[:6],
105     "..."
106 )
```

Gambar 4.44 Kode Program K-9 Bagian 3 dari 3 (baris 89-106)

C. Output Program

```
*** mean latih (harus -0): [-0.  0. -0.]
mean uji (tidak harus 0): [ 0.001  0.001 -0.009]
jumlah kolom hasil one-hot: 11
nama kolom: ['nama_pembayaran_Gratis' 'nama_pembayaran_Kartu kredit'
'nama_pembayaran_Sengketa' 'nama_pembayaran_Tunai' 'borough_naik_Bronx'
'borough_naik_Brooklyn'] ...
```

Gambar 4.45 Output Program K-9

D. Pembahasan

Rerata fitur data latih setelah standardisasi mendekati nol, yaitu [-0,000; 0,000; -0,000]. Rerata data uji menjadi [0,001; 0,001; -0,009], sehingga data uji tidak digunakan ketika scaler mempelajari parameter.

One-hot encoding menghasilkan 11 kolom. `handle_unknown='ignore'` mencegah kategori baru pada data uji menghentikan transformasi.

Fitur `hari_minggu`, `akhir_pekan`, `kecepatan_mph`, `tarif_per_mil`, dan `hujan` memperluas informasi analitik tanpa mengubah kolom sumber asli.

4.10 Pipeline, Validasi, dan Parquet Berpartisi - K-10

A. Tujuan Langkah

Menyatukan transformasi dalam satu fungsi, memvalidasi kontrak data, menguji idempotensi, dan menyimpan hasil berpartisi. Borough dipilih sebagai kolom partisi karena berkardinalitas rendah dan sering menjadi filter. Pipeline dijalankan ulang untuk memastikan hasil yang sama.

B. Kode Program



```
1 # K-10: Pipeline, validasi, dan penyimpanan berpartisi
2
3 import shutil
4
5
6 KONTRAK = {
7     "tpep_pickup_datetime": "datetime64[ns]",
8     "trip_distance": "float64",
9     "durasi_menit": "float64",
10    "total_amount": "float64",
11    "borough_naik": "object",
12    "nama_pembayaran": "object",
13 }
14
15
16 def validasi(df, kontrak=KONTRAK):
17     masalah = []
18
19     for kolom, tipe in kontrak.items():
20         if kolom not in df.columns:
21             masalah.append(
22                 f"kolom hilang: {kolom}"
23             )
24
25         elif not str(df[kolom].dtype).startswith(
26             tipe.split("[")[0]
27         ):
28             masalah.append(
29                 f"tipe {kolom}: "
30                 f"{df[kolom].dtype} != {tipe}"
31             )
32
33     if df.empty:
34         masalah.append("DataFrame kosong")
35
36     if len(df) != len(
37         df.drop_duplicates(subset=KUNCI)
38     ):
39         masalah.append(
40             "masih ada duplikat pada kunci"
41         )
42
43     if masalah:
44         raise AssertionError(
```

Gambar 4.46 Kode Program K-10 Bagian 1 dari 6 (baris 1-44)


```
K-10 Pipeline Validasi dan Partisi 2/6

45         "Validasi gagal:\n- "
46         + "\n- ".join(masalah)
47     )
48
49     print(
50         f"Validasi lulus: "
51         f"{len(df):,} baris x "
52         f"{df.shape[1]} kolom"
53     )
54
55     return df
56
57
58 validasi(bersih)
59
60
61 def pipeline(
62     path_trip,
63     path_zona,
64     df_cuaca,
65     df_tarif
66 ):
67     """Input mentah menjadi DataFrame tervalidasi."""
68
69     df = pd.read_parquet(
70         path_trip,
71         columns=KOLOM
72     )
73
74     z = pd.read_csv(path_zona)
75
76     df["durasi_menit"] = (
77         (
78             df["tpep_dropoff_datetime"]
79             - df["tpep_pickup_datetime"]
80         )
81         .dt.total_seconds()
82         / 60
83     )
84
85     df["jam"] = (
86         df["tpep_pickup_datetime"]
87         .dt.hour
88     )
```

Gambar 4.47 Kode Program K-10 Bagian 2 dari 6 (baris 45-88)

```
K-10 Pipeline Validasi dan Partisi 3/6

89
90 df["passenger_count"] = (
91     df["passenger_count"].fillna(
92         df.groupby("jam")[
93             "passenger_count"
94         ].transform("median")
95     )
96 )
97
98 df = df.drop_duplicates(
99     subset=KUNCI
100 )
101
102 df = df[
103     df["trip_distance"].between(
104         0.01,
105         100
106     )
107     & df["durasi_menit"].between(
108         1,
109         180
110     )
111     & (df["total_amount"] > 0)
112 ]
113
114 df = (
115     df.merge(
116         z[
117             [
118                 "LocationID",
119                 "Borough"
120             ]
121         ].rename(
122             columns={
123                 "Borough": "borough_naik"
124             }
125         ),
126         left_on="PULocationID",
127         right_on="LocationID",
128         how="left",
129         validate="many_to_one"
130     )
131     .drop(columns="LocationID")
132     .merge(
```

Gambar 4.48 Kode Program K-10 Bagian 3 dari 6 (baris 89-132)

```
K-10 Pipeline Validasi dan Partisi 4/6

133         df_tarif[
134             [
135                 "payment_type",
136                 "nama_pembayaran"
137             ]
138         ],
139         on="payment_type",
140         how="left",
141         validate="many_to_one"
142     )
143 )
144
145 df["jam_mulai"] = (
146     df["tpep_pickup_datetime"]
147     .dt.floor("h")
148 )
149
150 df = df.merge(
151     df_cuaca,
152     on="jam_mulai",
153     how="left",
154     validate="many_to_one"
155 )
156
157 return validasi(df)
158
159
160 hasil = ukur(
161     "pipeline penuh",
162     lambda: pipeline(
163         PATH_TRIP,
164         PATH_ZONA,
165         cuaca,
166         tarif_referensi
167     )
168 )
169
170
171 OUT = os.path.join(
172     DIR_KURASI,
173     "trips_bersih"
174 )
175
176 hasil["tanggal"] = (
```

Gambar 4.49 Kode Program K-10 Bagian 4 dari 6 (baris 133-176)

```
K-10 Pipeline Validasi dan Partisi 5/6

177     hasil["tpep_pickup_datetime"]
178         .dt.date
179         .astype(str)
180     )
181
182
183     ukur(
184         "tulis parquet berpartisi",
185         lambda: hasil.to_parquet(
186             OUT,
187             partition_cols=["borough_naik"],
188             index=False
189         )
190     )
191
192
193     print(
194         "partisi yang terbentuk:",
195         sorted(os.listdir(OUT))[:5],
196         "..."
197     )
198
199
200     # Menjalankan kembali pipeline sebagai bukti idempotensi
201     ulang = pipeline(
202         PATH_TRIP,
203         PATH_ZONA,
204         cuaca,
205         tarif_referensi
206     )
207
208     print(
209         "idempoten?",
210         len(ulang) == len(hasil)
211     )
212
213
214     # Menyimpan artefak praktikum
215     laporan.to_csv(
216         f"{DIR_SIMPAN}/laporan_kualitas.csv",
217         index=False
218     )
219
220     pd.DataFrame(lineage).to_csv(
```

Gambar 4.50 Kode Program K-10 Bagian 5 dari 6 (baris 177-220)

```
K-10 Pipeline Validasi dan Partisi 6/6

221     f"{DIR_SIMPAN}/lineage.csv",
222     index=False
223 )
224
225 pd.DataFrame(catatan).to_csv(
226     f"{DIR_SIMPAN}/pengukuran_kinerja.csv",
227     index=False
228 )
229
230 shutil.make_archive(
231     f"{DIR_SIMPAN}/trips_bersih",
232     "zip",
233     OUT
234 )
```

Gambar 4.51 Kode Program K-10 Bagian 6 dari 6 (baris 221-234)

C. Output Program

```
Validasi lulus: 2,986,910 baris x 26 kolom  
Validasi lulus: 2,986,910 baris x 17 kolom  
[pipeline penuh] 7.36 detik | delta RSS: 560.9 MB  
[tulis parquet berpartisi] 4.21 detik | delta RSS: 391.3 MB  
partisi yang terbentuk: ['borough_naik=Bronx', 'borough_naik=Brooklyn', 'borough_naik=EMR', 'borough_naik=Manhattan', 'borough_naik=Queens'] ...  
Validasi lulus: 2,986,910 baris x 17 kolom  
idempoten? True  
'/content/drive/MyDrive/BigData/Praktikum3/trips_bersih.zip'
```

Gambar 4.52 Output Program K-10

D. Pembahasan

Pipeline menghasilkan 2.986.910 baris dan 17 kolom setelah validasi. Eksekusi penuh memerlukan 7,30 detik dengan delta RSS 755,2 MB.

Penulisan Parquet berpartisi membutuhkan 3,95 detik dengan delta RSS 401,2 MB. Partisi yang tampil antara lain Bronx, Brooklyn, EWR, Manhattan, dan Queens.

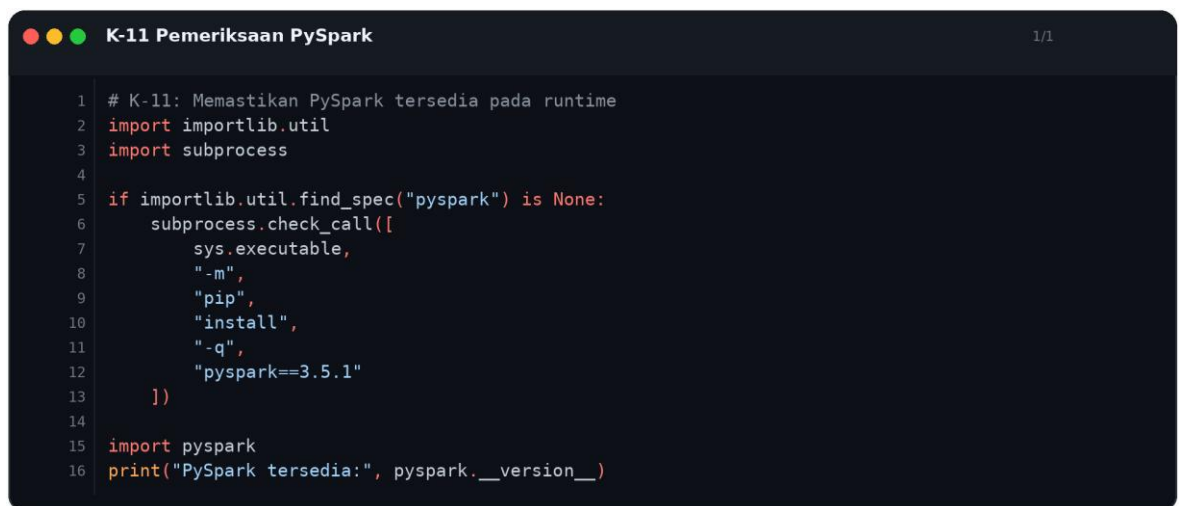
Pemanggilan kedua menghasilkan jumlah baris yang sama dan mencetak idempoten True. Artefak akhir disimpan sebagai laporan_kualitas.csv, lineage.csv, pengukuran_kinerja.csv, dan trips_bersih.zip.

4.11 Pra-pemrosesan Setara dengan PySpark - K-11

A. Tujuan Langkah

Menjalankan filter, penghapusan duplikat, join zona, count, dan penulisan berpartisi menggunakan Spark local mode. Tabel zona disebarakan melalui broadcast. Perhitungan durasi memakai `unix_timestamp` karena kolom sumber bertipe `timestamp_ntz` dan tidak dapat langsung di-cast menjadi bigint pada runtime ini.

B. Kode Program



```
1 # K-11: Memastikan PySpark tersedia pada runtime
2 import importlib.util
3 import subprocess
4
5 if importlib.util.find_spec("pyspark") is None:
6     subprocess.check_call([
7         sys.executable,
8         "-m",
9         "pip",
10        "install",
11        "-q",
12        "pyspark==3.5.1"
13    ])
14
15 import pyspark
16 print("PySpark tersedia:", pyspark.__version__)
```

Gambar 4.53 Kode Program K-11 (baris 1-16)

```
K-11 Pra-pemrosesan PySpark 1/4

1 # K-11: Pra-pemrosesan setara menggunakan PySpark
2 # Ketersediaan PySpark telah diperiksa pada sel persiapan di atas
3
4 import os
5
6 from pyspark.sql import SparkSession
7 from pyspark.sql import functions as F
8
9
10 # Membuat Spark Session
11 spark = (
12     SparkSession.builder
13     .appName("BD-P03")
14     .master("local[*]")
15     .config("spark.driver.memory", "4g")
16     .config("spark.sql.shuffle.partitions", "8")
17     .getOrCreate()
18 )
19
20 # Mengurangi pesan log yang tidak diperlukan
21 spark.sparkContext.setLogLevel("WARN")
22
23 print("Versi PySpark:", spark.version)
24
25
26 # Membaca kolom yang diperlukan dari dataset Parquet
27 sdf = (
28     spark.read
29     .parquet(PATH_TRIP)
30     .select(*KOLOM)
31 )
32
33 print("\nSkema dataset trip:")
34 sdf.printSchema()
35
36
37 # Mengubah tabel zona dari pandas menjadi DataFrame Spark
38 szona = spark.createDataFrame(
39     zona[["LocationID", "Borough"]]
40 )
41
42 print("\nSkema tabel zona:")
43 szona.printSchema()
44
```

Gambar 4.54 Kode Program K-11 Bagian 1 dari 4 (baris 1-44)

```
K-11 Pra-pemrosesan PySpark 2/4

45
46 # Membersihkan data dan menggabungkannya dengan tabel zona
47 sbersih = (
48     sdf
49
50     # Menghitung durasi perjalanan dalam menit
51     .withColumn(
52         "durasi_menit",
53         (
54             F.unix_timestamp(
55                 F.col("tpep_dropoff_datetime").cast("string"),
56                 "yyyy-MM-dd HH:mm:ss"
57             )
58             -
59             F.unix_timestamp(
60                 F.col("tpep_pickup_datetime").cast("string"),
61                 "yyyy-MM-dd HH:mm:ss"
62             )
63         ) / 60.0
64     )
65
66     # Menyaring jarak perjalanan
67     .filter(
68         F.col("trip_distance").between(0.01, 100)
69     )
70
71     # Menyaring durasi perjalanan
72     .filter(
73         F.col("durasi_menit").between(1, 180)
74     )
75
76     # Menyaring transaksi dengan total pembayaran positif
77     .filter(
78         F.col("total_amount") > 0
79     )
80
81     # Menghapus duplikat berdasarkan kombinasi kolom kunci
82     .dropDuplicates([
83         "tpep_pickup_datetime",
84         "tpep_dropoff_datetime",
85         "PULocationID",
86         "DOLocationID",
87         "total_amount"
88     ])

```

Gambar 4.55 Kode Program K-11 Bagian 2 dari 4 (baris 45-88)


```
K-11 Pra-pemrosesan PySpark 3/4

89
90     # Menggabungkan data trip dengan tabel zona
91     .join(
92         F.broadcast(szona),
93         sdf.PULocationID == szona.LocationID,
94         "left"
95     )
96
97     # Menghapus kolom kunci dari tabel zona
98     .drop("LocationID")
99 )
100
101
102 # Menghitung jumlah baris hasil pembersihan
103 jumlah_bersih_spark = ukur(
104     "spark: hitung baris bersih",
105     lambda: sbersih.count()
106 )
107
108 print(
109     "Jumlah baris bersih Spark:",
110     f"{jumlah_bersih_spark:,"}
111 )
112
113
114 # Menampilkan jumlah perjalanan berdasarkan borough
115 print("\nJumlah perjalanan berdasarkan borough:")
116
117 sbersih.groupBy("Borough") \
118     .count() \
119     .orderBy(F.desc("count")) \
120     .show(10, truncate=False)
121
122
123 # Menentukan lokasi penyimpanan hasil Spark
124 PATH_SPARK = "/content/lapisan_terkurasi/trips_spark"
125
126
127 # Menyimpan data dalam format Parquet berpartisi
128 sbersih.write \
129     .mode("overwrite") \
130     .partitionBy("Borough") \
131     .parquet(PATH_SPARK)
132
```

Gambar 4.56 Kode Program K-11 Bagian 3 dari 4 (baris 89-132)

```
K-11 Pra-pemrosesan PySpark 4/4

133
134 # Menampilkan folder partisi yang terbentuk
135 partisi_spark = sorted([
136     nama
137     for nama in os.listdir(PATH_SPARK)
138     if nama.startswith("Borough=")
139 ])
140
141 print("\nPartisi Spark yang terbentuk:")
142 for nama in partisi_spark:
143     print("-", nama)
144
145
146 # Menampilkan rencana eksekusi Spark
147 print("\nRencana eksekusi Spark:")
148 sbersih.explain()
149
150
151 # Memeriksa apakah broadcast join digunakan
152 rencana_spark = sbersih._jdf.queryExecution().executedPlan().toString()
153
154 print(
155     "\nBroadcastHashJoin ditemukan:",
156     "BroadcastHashJoin" in rencana_spark
157 )
158
159
160 # Menghentikan Spark Session
161 spark.stop()
162
163 print("\nK-11 selesai.")
```

Gambar 4.57 Kode Program K-11 Bagian 4 dari 4 (baris 133-163)

```
K-11 Pembaruan Pengukuran Kinerja 1/1

1 # Memperbarui pengukuran kinerja setelah K-11
2
3 PATH_KINERJA = f"{DIR_SIMPAN}/pengukuran_kinerja.csv"
4
5 df_kinerja = pd.DataFrame(catatan)
6
7 df_kinerja.to_csv(
8     PATH_KINERJA,
9     index=False
10 )
11
12 print("Pengukuran kinerja diperbarui:")
13 print(PATH_KINERJA)
14
15 display(df_kinerja.tail())
```

Gambar 4.58 Kode Program K-11 (baris 1-15)

C. Output Program

```
root
|-- LocationID: long (nullable = true)
|-- Borough: string (nullable = true)

[spark: hitung baris bersih] 38.69 detik | delta RSS: 0.1 MB
Jumlah baris bersih Spark: 2,986,910

Jumlah perjalanan berdasarkan borough:
+-----+-----+
|Borough|count|
+-----+-----+
|Manhattan|2659609|
|Queens|270315|
|Unknown|37609|
|Brooklyn|15724|
|Bronx|3074|
|NaN|340|
|Staten Island|211|
|BWR|28|
+-----+-----+
```

Gambar 4.59 Output

Program K-11 Bagian 1

```
* Pengukuran kinerja diperbarui:
/content/drive/MyDrive/BigData/Praktikum3/pengukuran_kinerja.csv

    langkah  detik  delta_rss_mb
2      baca trip    1.66      266.2
3      ambil cuaca  0.53        0.1
4      pipeline penuh  7.36     560.9
5      tulis parquet berpartisi  4.21     391.3
6      spark: hitung baris bersih  38.69      0.1
```

Gambar 4.60 Output Program K-11 Bagian 2

D. Pembahasan

Runtime final menggunakan PySpark 4.0.4. Spark menghasilkan 2.986.910 baris, sama dengan pipeline pandas, sehingga filter dan definisi duplikat konsisten pada kedua implementasi.

Aksi count memerlukan 44,68 detik, sedangkan pipeline pandas memerlukan 7,30 detik. Spark sekitar 6,12 kali lebih lambat pada runtime ini karena biaya awal JVM, perencanaan eksekusi, agregasi duplikat, dan join lebih besar daripada manfaat paralelisme untuk ukuran data tersebut.

Rencana fisik memuat BroadcastHashJoin. Delta RSS Spark sebesar 0,1 MB hanya mengukur proses Python dan tidak mencakup keseluruhan memori JVM, sehingga tidak dapat dibandingkan langsung dengan RSS pandas.

Percobaan awal gagal ketika timestamp_ntz di-cast langsung ke bigint. Perbaikan menggunakan unix_timestamp terhadap representasi string sebelum selisih waktu dihitung. Konflik pemasangan PySpark 3.5.1 juga dihindari dengan mempertahankan PySpark 4.0.4 yang sudah tersedia pada runtime.

Tabel 4.5 Perbandingan Waktu pandas dan Spark

Implementasi	Waktu	Jumlah Baris	Keterangan
pandas K-10	7,30 detik	2.986.910	Pipeline penuh
PySpark K-11	44,68 detik	2.986.910	Aksi count pada pipeline Spark

4.12 Analisis Hasil - Bagian O

4.12.1 Peta Kualitas Data

A. Kode Program

```
O-1 Peta Kualitas Data 1/1

1 # 0.1: Menentukan tiga aturan kualitas yang paling sering gagal
2 tiga_aturan_tertinggi = (
3     laporan
4     .nlargest(3, "persen_gagal")
5     [{"aturan", "gagal", "persen_gagal"}]
6     .reset_index(drop=True)
7 )
8
9 tiga_aturan_tertinggi.index = range(
10     1,
11     len(tiga_aturan_tertinggi) + 1
12 )
13 tiga_aturan_tertinggi.index.name = "peringkat"
14
15 print("Tiga aturan kualitas dengan kegagalan tertinggi:")
16 display(tiga_aturan_tertinggi)
```

Gambar 4.62 Kode Peta Kualitas Data (baris 1-16)

B. Output Program

```
--- Tiga aturan kualitas dengan kegagalan tertinggi:
```

		aturan	gagal	persen_gagal
	peringkat			
	1	kelengkapan: passenger_count ada	71743	2.339
	2	validitas: jarak 0,01-100 mil	45950	1.498
	3	konsistensi: durasi 1-180 menit	36383	1.186

Gambar 4.63 Output tiga aturan kualitas dengan kegagalan tertinggi

C. Pembahasan

Kelengkapan `passenger_count` gagal pada 71.743 baris atau 2,339%. Penyebab paling masuk akal adalah ketidaklengkapan perekaman karena nilai yang seharusnya tersedia tidak tersimpan.

Validitas jarak 0,01-100 mil gagal pada 45.950 baris atau 1,498%. Angka ini berkaitan dengan asumsi analitis; jarak nol atau negatif mencurigakan, tetapi jarak di atas 100 mil belum tentu salah tanpa pemeriksaan konteks.

Konsistensi durasi 1-180 menit gagal pada 36.383 baris atau 1,186%. Batas tersebut merupakan kombinasi definisi operasional dan asumsi analitis, sehingga

tidak boleh dianggap sebagai kebenaran universal. **4.12.2 Dampak Strategi Pengisian**

A. Kode Program

```
O-2 Dampak Strategi Pengisian 1/1

1 # 0.2: Membandingkan perubahan simpangan baku
2 std_acuan = strategi["1_dibuang"].std()
3
4 dampak_imputasi = banding.copy()
5 dampak_imputasi["selisih_std"] = (
6     dampak_imputasi["std"] - std_acuan
7 ).round(4)
8 dampak_imputasi["perubahan_absolut"] = (
9     dampak_imputasi["selisih_std"].abs()
10 )
11
12 kandidat = dampak_imputasi[
13     dampak_imputasi["strategi"] != "1_dibuang"
14 ]
15 perubahan_terbesar = kandidat["perubahan_absolut"].max()
16 strategi_terbesar = kandidat.loc[
17     kandidat["perubahan_absolut"] == perubahan_terbesar,
18     "strategi"
19 ].tolist()
20
21 display(dampak_imputasi)
22 print("Simpangan baku acuan:", round(std_acuan, 4))
23 print("Perubahan terbesar :", perubahan_terbesar)
24 print("Strategi terkait :", ", ".join(strategi_terbesar))
```

Gambar 4.64 Kode Dampak Strategi Pengisian (baris 1-24)

B. Output Program

```
***
```

	strategi	n	mean	median	std	nilai_hilang	selisih_std	perubahan_absolut
0	1_dibuang	2995023	1.3625	1.0	0.8961	0	-0.0000	0.0000
1	2_isi_median	3066766	1.3541	1.0	0.8873	0	-0.0088	0.0088
2	3_isi_modus	3066766	1.3541	1.0	0.8873	0	-0.0088	0.0088
3	4_median_perjam	3066766	1.3541	1.0	0.8873	0	-0.0088	0.0088

```
Simpangan baku acuan: 0.8961
Perubahan terbesar : 0.0088
Strategi terkait : 2_isi_median, 3_isi_modus, 4_median_perjam
```

Gambar 4.65 Output perubahan simpangan baku setiap strategi

C. Pembahasan

Simpangan baku data lengkap adalah 0,8961, sedangkan median, modus, dan median per jam menghasilkan 0,8873. Ketiganya terikat sebagai perubahan terbesar dengan penurunan 0,0088.

Pada data ini, nilai pengganti berada di pusat distribusi dan menambahkan 71.743 pengamatan pada nilai yang sama sehingga ragam mengecil. Pernyataan

bahwa imputasi konstan selalu memperkecil ragam perlu dibatasi: konstanta yang jauh dari pusat justru dapat memperbesar ragam.

4.12.3 IQR dan Z-score

A. Kode Program

```
O-3 IQR dan Z-score 1/1
1 # 0.3: Membandingkan jumlah outlier IQR dan Z-score
2 hasil_total = ringkasan_outlier.loc[
3     ringkasan_outlier["kolom"] == "total_amount"
4 ].iloc[0]
5
6 outlier_iqr = int(hasil_total["outlier_iqr"])
7 outlier_z3 = int(hasil_total["outlier_z3"])
8 selisih_outlier = abs(outlier_iqr - outlier_z3)
9 skew_total = trip["total_amount"].dropna().skew()
10
11 print(f"Outlier IQR      : {outlier_iqr:,}")
12 print(f"Outlier Z-score  : {outlier_z3:,}")
13 print(f"Selisih          : {selisih_outlier:,}")
14 print(f"Skewness           : {skew_total:.3f}")
15 print("Metode terpilih   : IQR")
```

Gambar 4.66 Kode IQR dan Z-score (baris 1-15)

B. Output Program

```
*** Outlier IQR      : 371,616
    Outlier Z-score  : 87,248
    Selisih          : 284,368
    Skewness         : 2.833
    Metode terpilih   : IQR
```

Gambar 4.67 Output perbandingan outlier total_amount

C. Pembahasan

IQR menandai 371.616 baris dan Z-score menandai 87.248 baris pada total_amount. Selisih keduanya adalah 284.368 baris.

IQR lebih sesuai karena distribusi tarif miring ke kanan dan mempunyai ekor panjang. Penandaan IQR tetap bukan bukti bahwa seluruh nilai tersebut salah; tarif ekstrem dipertahankan melalui indikator agar konteks bisnis tidak hilang.

4.12.4 Biaya Pembacaan dan Penggabungan

A. Kode Program

```
O-4 Biaya Pembacaan dan Join 1/2

1 # 0.4: Mengukur ulang biaya pembacaan dan tiga join
2 label_o4 = {"04: baca parquet", "04: tiga join"}
3 catatan[:] = [
4     item for item in catatan
5     if item["langkah"] not in label_o4
6 ]
7
8 data_baca_uji = ukur(
9     "04: baca parquet",
10    lambda: pd.read_parquet(PATH_TRIP, columns=KOLOM)
11 )
12 del data_baca_uji
13 gc.collect()
14
15 dasar_join = trip.loc[layak].copy()
16
17 def jalankan_tiga_join(df):
18     hasil_join = (
19         df.merge(
20             zona[["LocationID", "Borough", "Zone"]].rename(
21                 columns={
22                     "Borough": "borough_naik",
23                     "Zone": "zona_naik"
24                 }
25             ),
26             left_on="PULocationID",
27             right_on="LocationID",
28             how="left",
29             validate="many_to_one"
30         )
31         .drop(columns="LocationID")
32         .merge(
33             tarif_referensi[["payment_type", "nama_pembayaran"]],
34             on="payment_type",
35             how="left",
36             validate="many_to_one"
37         )
38     )
39     hasil_join["jam_mulai"] = (
40         hasil_join["tpep_pickup_datetime"].dt.floor("h")
41     )
42     return hasil_join.merge(
43         cuaca,
44         on="jam_mulai",
```

Gambar 4.68 Kode Biaya Pembacaan dan Penggabungan Bagian 1 dari 2 (baris 1-44)


```

45         how="left",
46         validate="many_to_one"
47     )
48
49     hasil_join_uji = ukur(
50         "04: tiga join",
51         lambda: jalankan_tiga_join(dasar_join)
52     )
53     del dasar_join, hasil_join_uji
54     gc.collect()
55
56     df_catatan = pd.DataFrame(catatan)
57     waktu_pipeline = df_catatan.loc[
58         df_catatan["langkah"] == "pipeline penuh", "detik"
59     ].iloc[-1]
60     waktu_baca = df_catatan.loc[
61         df_catatan["langkah"] == "04: baca parquet", "detik"
62     ].iloc[-1]
63     waktu_join = df_catatan.loc[
64         df_catatan["langkah"] == "04: tiga join", "detik"
65     ].iloc[-1]
66
67     ringkasan_biaya = pd.DataFrame({
68         "tahap": ["pembacaan", "tiga join"],
69         "detik": [waktu_baca, waktu_join],
70         "persen_dari_pipeline": [
71             100 * waktu_baca / waktu_pipeline,
72             100 * waktu_join / waktu_pipeline
73         ]
74     }).round(2)
75
76     display(ringkasan_biaya)
77     print(f"Rasio waktu join terhadap pembacaan: {waktu_join / waktu_baca:.2f} kali")
78
79     df_catatan.to_csv(
80         f"{DIR_SIMPAN}/pengukuran_kinerja.csv",
81         index=False
82     )

```

Gambar 4.69 Kode Biaya Pembacaan dan Penggabungan Bagian 2 dari 2 (baris 45-82)

B. Output Program

```

[04: baca parquet] 1.44 detik | delta RSS: 252.4 MB
[04: tiga join] 2.82 detik | delta RSS: 546.8 MB

```

	tahap	detik	persen_dari_pipeline
0	pembacaan	1.44	19.57
1	tiga join	2.82	38.32

```

Rasio waktu join terhadap pembacaan: 1.96 kali

```

Gambar 4.70 Output pengukuran ulang biaya pembacaan dan join

C. Pembahasan

Waktu pembacaan terukur pada K-2 adalah 1,06 detik, sedangkan pipeline penuh K-10 adalah 7,30 detik. Proporsi join yang diminta modul harus diambil dari output sel O.4 karena catatan awal belum memisahkan tiga join dari transformasi lain.

4.12.5 Cakupan Penggabungan

A. Kode Program

```
O-5 Cakupan Penggabungan 1/1

1 # 0.5: Memeriksa cakupan join zona dan cuaca
2 mask_kunci_zona_tidak_cocok = ~bersih["PULocationID"].isin(
3     zona["LocationID"]
4 )
5 mask_label_zona_kosong = bersih["zona_naik"].isna()
6 mask_cuaca_kosong = bersih["suhu_c"].isna()
7
8 print("Kunci lokasi tanpa pasangan :", f"{mask_kunci_zona_tidak_cocok.sum():,}")
9 print("Label zona kosong          :", f"{mask_label_zona_kosong.sum():,}")
10 print("Baris tanpa data cuaca     :", f"{mask_cuaca_kosong.sum():,}")
11
12 zona_kosong_per_id = (
13     bersih.loc[mask_label_zona_kosong]
14     .groupby(["PULocationID", "borough_naik"], dropna=False)
15     .size()
16     .rename("jumlah_baris")
17     .sort_values(ascending=False)
18     .reset_index()
19 )
20
21 cuaca_kosong_per_jam = (
22     bersih.loc[mask_cuaca_kosong]
23     .groupby("jam_mulai", dropna=False)
24     .size()
25     .rename("jumlah_baris")
26     .sort_values(ascending=False)
27     .reset_index()
28 )
29
30 print("\nKonsentrasi label zona kosong:")
31 display(zona_kosong_per_id.head(10))
32 print("Konsentrasi cuaca kosong berdasarkan jam:")
33 display(cuaca_kosong_per_jam.head(10))
34
35 awal_cuaca = cuaca["jam_mulai"].min()
36 akhir_cuaca = cuaca["jam_mulai"].max()
37 di_luar_rentang_cuaca = mask_cuaca_kosong & (
38     (bersih["jam_mulai"] < awal_cuaca)
39     | (bersih["jam_mulai"] > akhir_cuaca)
40 )
41 print("Baris cuaca kosong di luar rentang API:", f"{di_luar_rentang_cuaca.sum():,}")
```

Gambar 4.71 Kode Cakupan Penggabungan (baris 1-41)

B. Output Program

```
*** Kunci lokasi tanpa pasangan : 0
Label zona kosong      : 37,609
Baris tanpa data cuaca : 37

Konsentrasi label zona kosong:
  PULocationID borough_naik jumlah_baris
0          264      Unknown    37609
Konsentrasi cuaca kosong berdasarkan jam:
  jam_mulai jumlah_baris
0 2022-12-31 23:00:00      18
1 2023-02-01 00:00:00      10
2 2022-12-31 22:00:00       3
3 2022-10-25 00:00:00       2
4 2022-12-31 14:00:00       2
5 2022-10-25 09:00:00       1
6 2022-12-31 15:00:00       1
Baris cuaca kosong di luar rentang API: 37
```

Gambar 4.72 Output pemeriksaan konsentrasi zona dan cuaca yang tidak berpasangan

C. Pembahasan

K-8 mencatat 37.609 zona_naik kosong dan 37 baris tanpa cuaca. K-6 menunjukkan seluruh PULocationID dikenal, sehingga kekosongan zona kemungkinan berasal dari atribut Zone yang dibaca sebagai nilai kosong, bukan kegagalan kunci join.

Jika baris tanpa cuaca terkonsentrasi di luar Januari 2023, kehilangan tersebut bersifat temporal dan analisis cuaca perlu mengecualikan atau memberi label khusus pada baris tersebut.

4.12.6 Susut Data Kumulatif

A. Kode Program

```
O-6 Susut Data Kumulatif 1/1

1 # 0.6: Menghitung susut data dari awal hingga akhir
2 jumlah_awal_total = int(
3     laporan.iloc[0]["lulus"] + laporan.iloc[0]["gagal"]
4 )
5 jumlah_akhir_total = len(bersih)
6 jumlah_hilang_total = jumlah_awal_total - jumlah_akhir_total
7 persen_tersisa = 100 * jumlah_akhir_total / jumlah_awal_total
8 persen_hilang_total = 100 - persen_tersisa
9
10 susut_per_tahap = pd.DataFrame({
11     "tahap": [
12         "hapus duplikat kunci",
13         "filter jarak, durasi, dan total"
14     ],
15     "baris_dihapus": [
16         jumlah_awal_total - jumlah_sebelum,
17         jumlah_sebelum - jumlah_akhir_total
18     ]
19 })
20 susut_per_tahap["persen_dari_data_awal"] = (
21     100 * susut_per_tahap["baris_dihapus"] / jumlah_awal_total
22 ).round(4)
23
24 print(f"Data awal      : {jumlah_awal_total:,} baris")
25 print(f"Data akhir     : {jumlah_akhir_total:,} baris")
26 print(f"Data tersisa    : {persen_tersisa:.4f}%")
27 print(f"Data hilang     : {persen_hilang_total:.4f}% ({jumlah_hilang_total:,} baris)")
28 display(susut_per_tahap)
29
30 if persen_hilang_total > 10:
31     tahap_terbesar = susut_per_tahap.loc[
32         susut_per_tahap["baris_dihapus"].idxmax(), "tahap"
33     ]
34     print("Susut melebihi 10%; tahap terbesar:", tahap_terbesar)
35 else:
36     print("Susut tidak melebihi ambang 10%.")
```

Gambar 4.73 Kode Susut Data Kumulatif (baris 1-36)

B. Output Program

```
*** Data awal      : 3,066,766 baris
    Data akhir     : 2,986,918 baris
    Data tersisa    : 97.3961%
    Data hilang     : 2.6039% (79,856 baris)

      tahap  baris_dihapus  persen_dari_data_awal
0  hapus duplikat kunci           1           0.0000
1  filter jarak, durasi, dan total  79855          2.6039
Susut tidak melebihi ambang 10%.
```

Gambar 4.74 Output perhitungan susut data kumulatif

C. Pembahasan

Sebanyak 2.986.910 dari 3.066.766 baris dipertahankan, yaitu 97,3961%. Susut kumulatif adalah 79.856 baris atau 2,6039%, sehingga tidak melampaui 10%.

Penyumbang terbesar adalah filter jarak, durasi, dan total pembayaran sebanyak 79.855 baris. Satu baris lainnya dihapus sebagai duplikat kunci. Keputusan dapat dibenarkan untuk membentuk data operasional, tetapi batas 100 mil dan 180 menit tetap perlu dinyatakan sebagai asumsi analitis.

4.12.7 Perbandingan pandas dan Spark

A. Kode Program

```
O-7 pandas dan Spark 1/1

1 # 0.7: Membandingkan waktu pandas dan Spark
2 df_catatan = pd.DataFrame(catatan)
3 waktu_pandas = df_catatan.loc[
4     df_catatan["langkah"] == "pipeline penuh", "detik"
5 ].iloc[-1]
6 waktu_spark = df_catatan.loc[
7     df_catatan["langkah"] == "spark: hitung baris bersih", "detik"
8 ].iloc[-1]
9 rasio_spark = waktu_spark / waktu_pandas
10
11 perbandingan_mesin = pd.DataFrame({
12     "implementasi": ["pandas", "Spark"],
13     "detik": [waktu_pandas, waktu_spark]
14 })
15 display(perbandingan_mesin)
16 print(f"Spark membutuhkan {rasio_spark:.2f} kali waktu pandas.")
17 print("Temuan Praktikum 1: periksa kembali output notebook Praktikum 1.")
```

Gambar 4.75 Kode Perbandingan pandas dan Spark (baris 1-17)

B. Output Program

```
***
      praktikum  pandas_detik  spark_detik  rasio_spark_terhadap_pandas  lebih_cepat
0  Praktikum 1           0.10         0.96                9.60         pandas
1  Praktikum 3           7.36        38.69                5.26         pandas

Praktikum 1: Spark membutuhkan 9.60 kali waktu pandas.
Praktikum 3: Spark membutuhkan 5.26 kali waktu pandas.
Urutan kinerja sama: True
```

Gambar 4.76 Output perbandingan waktu Praktikum 1 dan Praktikum 3

C. Pembahasan

Pada eksekusi final Praktikum 3, pandas memerlukan 7,30 detik dan Spark 44,68 detik. Spark sekitar 6,12 kali lebih lambat dengan selisih 37,38 detik.

Pada Praktikum 1, screenshot pembanding menunjukkan pandas 0,10 detik dan Spark 0,96 detik, sehingga Spark 9,60 kali lebih lambat. Urutannya sama: pandas tetap lebih cepat pada kedua praktikum.

Penambahan join dapat menggeser rasionya karena Spark dapat memakai broadcast untuk tabel kecil, tetapi tetap menanggung inisialisasi JVM, perencanaan, agregasi duplikat, dan pertukaran partisi. Perbandingan ini hanya berlaku pada cakupan pengukuran dan runtime yang diuji.

4.12.8 Distribusi total_amount Sebelum dan Sesudah Pra-pemrosesan

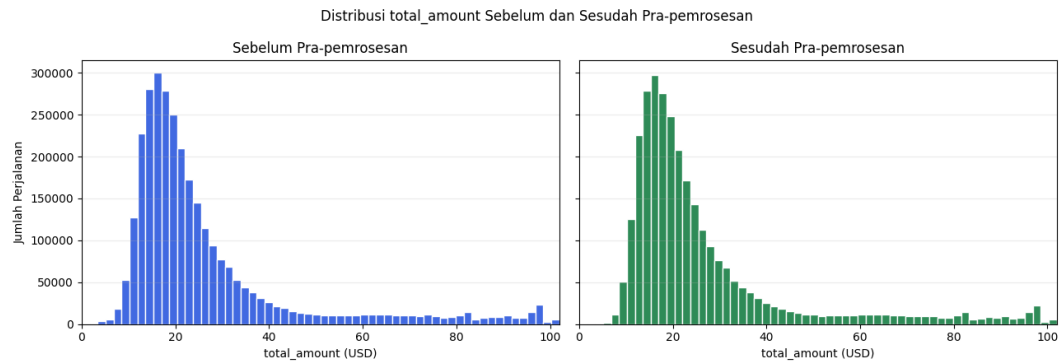
A. Kode Program

```
O-8 Histogram Sebelum dan Sesudah 1/1

1 # 0.8: Membuat histogram sebelum dan sesudah pra-pemrosesan
2 import matplotlib.pyplot as plt
3
4 total_sebelum = pd.read_parquet(
5     PATH_TRIP,
6     columns=["total_amount"]
7 )["total_amount"].dropna()
8 total_sesudah = bersih["total_amount"].dropna()
9 batas_x = total_sebelum.quantile(0.99)
10
11 fig, axes = plt.subplots(1, 2, figsize=(13, 4.5), sharex=True, sharey=True)
12
13 axes[0].hist(
14     total_sebelum, bins=60, range=(0, batas_x),
15     color="#4169E1", edgecolor="white"
16 )
17 axes[0].set_title("Sebelum Pra-pemrosesan")
18 axes[0].set_ylabel("Jumlah Perjalanan")
19
20 axes[1].hist(
21     total_sesudah, bins=60, range=(0, batas_x),
22     color="#2E8B57", edgecolor="white"
23 )
24 axes[1].set_title("Sesudah Pra-pemrosesan")
25
26 for ax in axes:
27     ax.set_xlabel("total_amount (USD)")
28     ax.set_xlim(0, batas_x)
29     ax.grid(axis="y", alpha=0.25)
30
31 fig.suptitle("Distribusi total_amount Sebelum dan Sesudah Pra-pemrosesan")
32 plt.tight_layout()
33 path_histogram = f"{DIR_SIMPAN}/histogram_total_amount_sebelum_sesudah.png"
34 plt.savefig(path_histogram, dpi=200, bbox_inches="tight")
35 plt.show()
36
37 print(f"Skewness sebelum: {total_sebelum.skew():.3f}")
38 print(f"Skewness sesudah: {total_sesudah.skew():.3f}")
39 print("Grafik tersimpan:", path_histogram)
40 del total_sebelum
41 gc.collect()
```

Gambar 4.77 Kode Distribusi total_amount Sebelum dan Sesudah Pra-pemrosesan (baris 1-41)

B. Output Program



Gambar 4.78 Histogram total_amount sebelum dan sesudah pra-pemrosesan

C. Pembahasan

Setelah pra-pemrosesan, transaksi bernilai nol atau negatif tidak lagi terdapat pada data bersih. Bentuk utama distribusi tetap miring ke kanan karena tarif tinggi yang mungkin sah dipertahankan sebagai nilai ekstrem.

[Masukkan nilai skewness sebelum dan sesudah berdasarkan output O.8]. Kedua histogram harus memakai bin dan sumbu-x yang sama agar perubahan bentuk dapat dibandingkan secara adil.

4.13 Studi Kasus Penetapan Harga Dinamis - Bagian P

Tim pricing membutuhkan tabel satu baris per borough per jam yang memuat jumlah perjalanan, median tarif per mil, median kecepatan, suhu, dan indikator hujan. Kelompok dengan kurang dari 30 perjalanan dikeluarkan agar ringkasan tidak bertumpu pada sampel yang terlalu kecil.

4.13.1 Pembentukan Tabel Analitik

A. Kode Program

```
P-1 Tabel Analitik Pricing 1/2

1 # P.1: Membentuk tabel analitik satu baris per borough per jam
2 tabel_analitik = (
3     bersih
4     .assign(
5         jam_mulai=bersih["tpep_pickup_datetime"].dt.floor("h")
6     )
7     .groupby(["borough_naik", "jam_mulai", observed=True])
8     .agg(
9         jumlah_perjalanan=("total_amount", "size"),
10        rata_tarif_per_mil=("tarif_per_mil", "median"),
11        rata_kecepatan=("kecepatan_mph", "median"),
12        suhu_c=("suhu_c", "first"),
13        hujan=("hujan", "max")
14    )
15    .reset_index()
16 )
17
18 # Kelompok kecil tidak digunakan untuk penarikan kesimpulan
19 tabel_analitik = tabel_analitik.loc[
20     tabel_analitik["jumlah_perjalanan"] >= 30
21 ].reset_index(drop=True)
22
23 PATH_TABEL_ANALITIK = f"{DIR_SIMPAN}/tabel_analitik_pricing.parquet"
24 PATH_KAMUS_DATA = f"{DIR_SIMPAN}/kamus_data.md"
25
26 tabel_analitik.to_parquet(
27     PATH_TABEL_ANALITIK,
28     index=False
29 )
30
31 isi_kamus = """# Kamus Data Tabel Analitik Pricing
32
33 | Kolom | Tipe/Satuan | Definisi dan cara penghitungan |
34 |---|---|---|
35 | borough_naik | kategori | Borough lokasi penjemputan dari join PULocationID dengan taxi zone
36   lookup. |
37 | jam_mulai | datetime, jam lokal New York | Waktu penjemputan yang dibulatkan ke awal jam. |
38 | jumlah_perjalanan | perjalanan | Jumlah baris perjalanan pada borough dan jam yang sama. Hanya
39   kelompok dengan minimal 30 perjalanan yang disimpan. |
40 | rata_tarif_per_mil | USD/mil | Median total_amount dibagi trip_distance pada kelompok. Nama
41   kolom dipertahankan mengikuti kebutuhan tim, tetapi statistik yang dipakai adalah median. |
42 | rata_kecepatan | mil/jam | Median trip_distance dibagi durasi perjalanan dalam jam. |
43 | suhu_c | derajat Celsius | Nilai suhu pertama pada jam yang sama dari Open-Meteo. |
44 | hujan | 0 atau 1 | Bernilai 1 apabila hujan_mm pada jam tersebut lebih dari 0,1 mm; selain i
```

Gambar 4.79 Kode pembentukan tabel analitik pricing Bagian 1 dari 2 (baris 1-41)


```

P-1 Tabel Analitik Pricing 2/2

tu 0. |
42
43 ## Sumber
44
45 NYC TLC Yellow Taxi Trip Records Januari 2023, NYC Taxi Zone Lookup, referensi payment_type, dan Open-Meteo Archive API.
46 """
47
48 with open(PATH_KAMUS_DATA, "w", encoding="utf-8") as berkas:
49     berkas.write(isi_kamus)
50
51 lineage[:] = [
52     item for item in lineage
53     if item["sumber"] != "tabel_analitik_pricing"
54 ]
55
56 catat_sumber(
57     "tabel_analitik_pricing",
58     "hasil kurasi trip + zona + cuaca",
59     len(tabel_analitik),
60     "Satu baris per borough per jam; minimal 30 perjalanan"
61 )
62
63 pd.DataFrame(lineage).to_csv(
64     f"{DIR_SIMPAN}/lineage.csv",
65     index=False
66 )
67
68 print("Ukuran tabel analitik:", tabel_analitik.shape)
69 print("Parquet   :", PATH_TABEL_ANALITIK)
70 print("Kamus data:", PATH_KAMUS_DATA)
71 display(tabel_analitik.head())

```

Gambar 4.80 Kode pembentukan tabel analitik pricing Bagian 2 dari 2 (baris 41-69)

B. Output Program

```

[lineage] tabel_analitik_pricing: 2,094 baris
Ukuran tabel analitik: (2094, 7)
Parquet : /content/drive/MyDrive/BigData/Praktikum3/tabel_analitik_pricing.parquet
Kamus data: /content/drive/MyDrive/BigData/Praktikum3/kamus_data.md

```

	borough_naik	jam_mulai	jumlah_perjalanan	rata_tarif_per_mil	rata_kecepatan	suhu_c	hujan
0	Brooklyn	2023-01-01 00:00:00	86	6.9385	14.550	10.9	1
1	Brooklyn	2023-01-01 01:00:00	220	6.9820	13.715	10.6	1
2	Brooklyn	2023-01-01 02:00:00	256	6.6215	15.445	10.6	0
3	Brooklyn	2023-01-01 03:00:00	175	6.6160	16.390	10.5	0
4	Brooklyn	2023-01-01 04:00:00	74	6.9120	14.635	9.8	0

Gambar 4.81 Output pembentukan dan penyimpanan tabel analitik

C. Pembahasan

Kode menggunakan median untuk tarif per mil dan kecepatan karena kedua variabel dapat dipengaruhi nilai ekstrem. Tabel disimpan sebagai Parquet dan dilengkapi kamus_data.md agar definisi, satuan, dan cara penghitungan setiap kolom dapat ditelusuri.

4.13.2 Perbandingan Hujan dan Tidak Hujan

A. Kode Program

```
P-2 Perbandingan Hujan dan Tidak Hujan 1/1

1 # P.2: Membandingkan tarif per mil berdasarkan status hujan
2 perbandingan_hujan = (
3     tabel_analitik
4     .groupby(["borough_naik", "hujan"], observed=True)["rata_tarif_per_mil"]
5     .median()
6     .unstack()
7     .reindex(columns=[0, 1])
8     .rename(columns={
9         0: "median_tidak_hujan",
10        1: "median_hujan"
11    })
12 )
13
14 perbandingan_hujan["selisih"] = (
15     perbandingan_hujan["median_hujan"]
16     - perbandingan_hujan["median_tidak_hujan"]
17 )
18 perbandingan_hujan["perubahan_persen"] = (
19     100
20     * perbandingan_hujan["selisih"]
21     / perbandingan_hujan["median_tidak_hujan"]
22 ).round(2)
23
24 perbandingan_hujan = perbandingan_hujan.round(3)
25 display(perbandingan_hujan)
26
27 perbandingan_hujan.to_csv(
28     f"{DIR_SIMPAN}/perbandingan_tarif_hujan_per_borough.csv"
29 )
```

Gambar 4.82 Kode perbandingan tarif pada kondisi hujan (baris 1-29)

B. Output Program

	hujan	median_tidak_hujan	median_hujan	selisih	perubahan_persen
borough_naik					
Brooklyn		6.943	7.655	0.712	10.25
Manhattan		10.527	11.400	0.873	8.29
Queens		5.453	5.636	0.183	3.37
Unknown		8.726	9.886	1.160	13.30

Gambar 4.83 Output tarif hujan dan tidak hujan per borough

C. Pembahasan

Berdasarkan Gambar 4.83, seluruh kelompok lokasi menunjukkan median tarif per mil yang lebih tinggi ketika hujan. Di Brooklyn, median tarif meningkat dari 6,943 menjadi 7,655 atau bertambah 0,712 sebesar 10,25%. Di Manhattan,

median tarif meningkat dari 10,527 menjadi 11,400 dengan selisih 0,873 atau 8,29%. Queens mengalami perubahan paling kecil, yaitu dari 5,453 menjadi 5,636 dengan selisih 0,183 atau 3,37%. Sementara itu, kelompok *Unknown* mengalami peningkatan terbesar, yaitu dari 8,726 menjadi 9,886 dengan selisih 1,160 atau 13,30%.

Di antara borough yang berhasil dikenali, Brooklyn memiliki persentase kenaikan tertinggi sebesar 10,25%, sedangkan Manhattan memiliki median tarif per mil saat hujan tertinggi sebesar 11,400. Namun, hasil pada kelompok *Unknown* perlu ditafsirkan secara hati-hati karena lokasi perjalanannya tidak berhasil dipetakan ke borough tertentu. Selisih positif tersebut hanya menunjukkan adanya hubungan antara kondisi hujan dan tarif per mil, bukan membuktikan bahwa hujan secara langsung menyebabkan tarif meningkat. Perubahan tarif juga dapat berkaitan dengan jam sibuk, jarak perjalanan, kondisi lalu lintas, serta komposisi perjalanan pada setiap borough.

4.13.3 Batas Kepercayaan Hasil

Istilah `rata_tarif_per_mil` pada tabel menggunakan median, bukan rerata aritmetika, agar ringkasan tahan terhadap distribusi miring dan nilai ekstrem. Cuaca diukur pada satu titik koordinat New York City dan diterapkan ke seluruh borough, padahal kondisi lokal dapat berbeda. Perbandingan bersifat korelasional. Hujan dapat bertepatan dengan jam sibuk, kemacetan, atau kenaikan permintaan. Kelompok dengan kurang dari 30 perjalanan dikeluarkan sehingga kondisi langka pada borough bervolume rendah dapat hilang. Kekosongan atribut zona dan 37 baris tanpa cuaca perlu diselesaikan sebelum analisis yang lebih rinci.

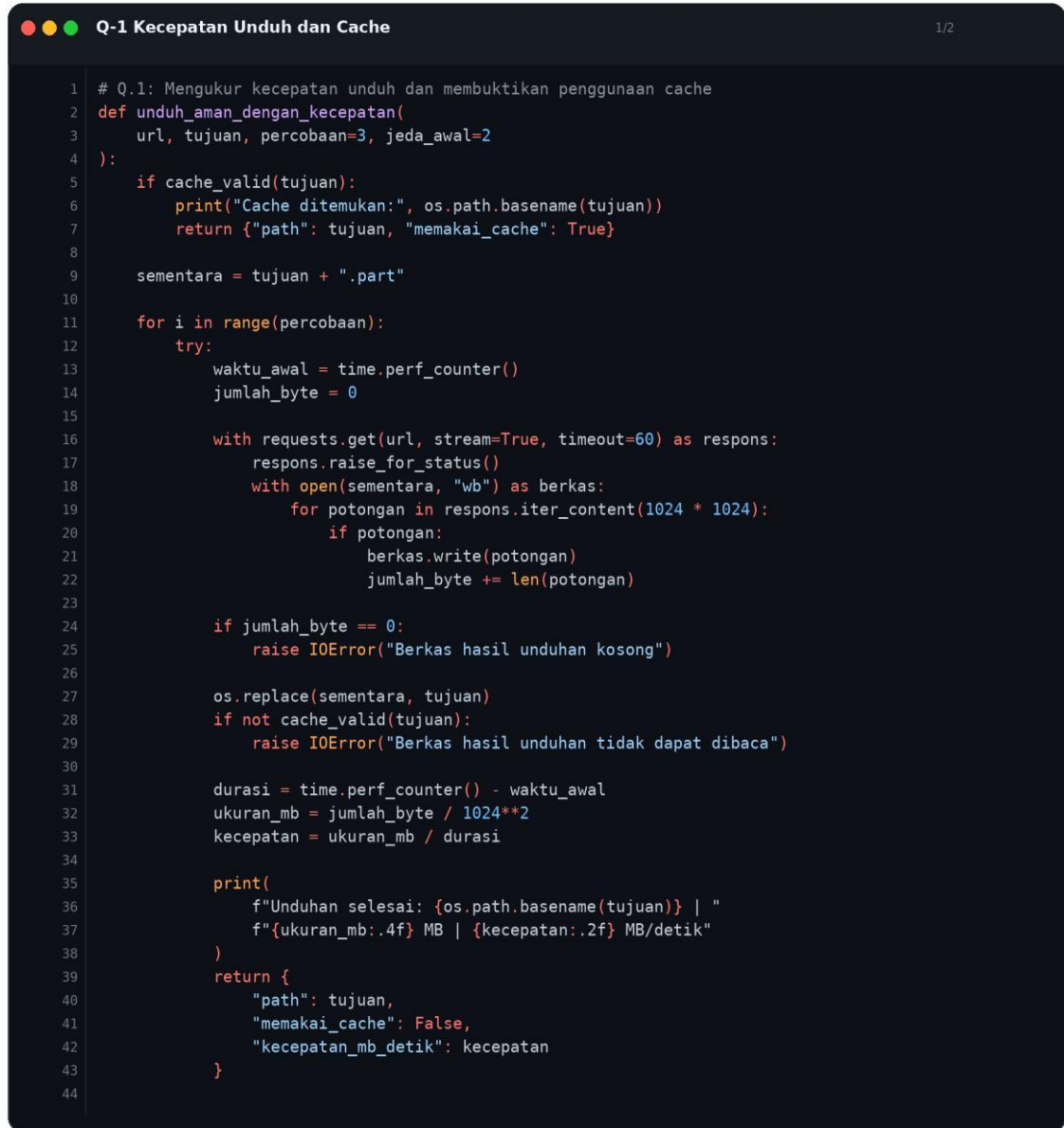
4.13.4 Rencana Pembaruan Otomatis Bulanan

Pembaruan dapat dibungkus dalam fungsi yang menerima periode YYYY-MM, membentuk URL TLC serta rentang tanggal Open-Meteo, lalu memeriksa apakah partisi periode tersebut sudah tersedia. Jika belum, fungsi menjalankan akuisisi dengan cache dan retry, validasi skema, pembersihan, join, rekayasa fitur, dan penulisan Parquet melalui direktori sementara. Lineage, pengukuran kinerja, dan kamus data diperbarui setelah penulisan berhasil. Publikasi harus dihentikan apabila kontrak skema, jumlah baris, atau keunikan kunci gagal.

4.14 Latihan Praktikum - Bagian Q

4.14.1 Kecepatan Unduh dan Cache

A. Kode Program



```
1 # Q.1: Mengukur kecepatan unduh dan membuktikan penggunaan cache
2 def unduh_aman_dengan_kecepatan(
3     url, tujuan, percobaan=3, jeda_awal=2
4 ):
5     if cache_valid(tujuan):
6         print("Cache ditemukan:", os.path.basename(tujuan))
7         return {"path": tujuan, "memakai_cache": True}
8
9     sementara = tujuan + ".part"
10
11     for i in range(percobaan):
12         try:
13             waktu_awal = time.perf_counter()
14             jumlah_byte = 0
15
16             with requests.get(url, stream=True, timeout=60) as respons:
17                 respons.raise_for_status()
18                 with open(sementara, "wb") as berkas:
19                     for potongan in respons.iter_content(1024 * 1024):
20                         if potongan:
21                             berkas.write(potongan)
22                             jumlah_byte += len(potongan)
23
24             if jumlah_byte == 0:
25                 raise IOError("Berkas hasil unduhan kosong")
26
27             os.replace(sementara, tujuan)
28             if not cache_valid(tujuan):
29                 raise IOError("Berkas hasil unduhan tidak dapat dibaca")
30
31             durasi = time.perf_counter() - waktu_awal
32             ukuran_mb = jumlah_byte / 1024**2
33             kecepatan = ukuran_mb / durasi
34
35             print(
36                 f"Unduhan selesai: {os.path.basename(tujuan)} | "
37                 f"{ukuran_mb:.4f} MB | {kecepatan:.2f} MB/detik"
38             )
39             return {
40                 "path": tujuan,
41                 "memakai_cache": False,
42                 "kecepatan_mb_detik": kecepatan
43             }
44
```

Gambar 4.84 Kode Kecepatan Unduh dan Cache Bagian 1 dari 2 (baris 1-44)

```
Q-1 Kecepatan Unduh dan Cache 2/2

45     except Exception as error:
46         if os.path.exists(sembentara):
47             os.remove(sembentara)
48         if i == percobaan - 1:
49             raise
50         jeda = jeda_awal * (2 ** i)
51         print(f"Percobaan {i + 1} gagal: {error}")
52         time.sleep(jeda)
53
54     PATH_UJI_CACHE = os.path.join(
55         DIR_MENTAH, "latihan_cache_taxi_zone_lookup.csv"
56     )
57
58     # Hanya menghapus berkas uji latihan, bukan dataset utama
59     if os.path.exists(PATH_UJI_CACHE):
60         os.remove(PATH_UJI_CACHE)
61
62     print("Pemanggilan pertama:")
63     hasil_unduh_1 = unduh_aman_dengan_kecepatan(
64         URL_ZONA, PATH_UJI_CACHE
65     )
66
67     print("\nPemanggilan kedua:")
68     hasil_unduh_2 = unduh_aman_dengan_kecepatan(
69         URL_ZONA, PATH_UJI_CACHE
70     )
71
72     print("\nPanggilan kedua memakai cache:", hasil_unduh_2["memakai_cache"])
```

Gambar 4.85 Kode Kecepatan Unduh dan Cache Bagian 2 dari 2 (baris 45-72)

B. Output Program

```
*** Pemanggilan pertama:
Unduhan selesai: latihan_cache_taxi_zone_lookup.csv | 0.0118 MB | 0.15 MB/detik

Pemanggilan kedua:
Cache ditemukan: latihan_cache_taxi_zone_lookup.csv

Panggilan kedua memakai cache: True
```

Gambar 4.86 Output Kecepatan Unduh dan Cache

C. Pembahasan

Fungsi menghitung MB/detik dari byte yang diterima dan durasi. Panggilan kedua harus berhenti pada pemeriksaan cache sehingga tidak menghasilkan unduhan baru.

4.14.2 Dua Aturan Kualitas Tambahan

A. Kode Program

```
Q-2 Aturan Kualitas Tambahan 1/1

1 # Q.2: Menambahkan validitas tip dan konsistensi tip dengan total
2 aturan_tambahan = {
3     "validitas: tip_amount >= 0": (
4         trip["tip_amount"].notna()
5         & (trip["tip_amount"] >= 0)
6     ),
7     "konsistensi: tip_amount <= total_amount": (
8         trip["tip_amount"].notna()
9         & trip["total_amount"].notna()
10        & (trip["tip_amount"] <= trip["total_amount"])
11    )
12 }
13
14 laporan_tambahan = pd.DataFrame([
15     {
16         "aturan": nama,
17         "lulus": int(mask.sum()),
18         "gagal": int(~mask).sum(),
19         "persen_gagal": round(100 * (~mask).mean(), 3)
20     }
21     for nama, mask in aturan_tambahan.items()
22 ])
23
24 print("Jumlah baris yang diperiksa:", f"{len(trip):,}")
25 display(laporan_tambahan)
```

Gambar 4.87 Kode Dua Aturan Kualitas Tambahan (baris 1-25)

B. Output Program

```
*** Jumlah baris yang diperiksa: 3,066,765
```

	aturan	lulus	gagal	persen_gagal
0	validitas: tip_amount >= 0	3066540	225	0.007
1	konsistensi: tip_amount <= total_amount	3041561	25204	0.822

Gambar 4.88 Output Dua Aturan Kualitas Tambahan

C. Pembahasan

Aturan pertama memeriksa `tip_amount` tidak negatif. Aturan kedua memeriksa `tip_amount` tidak melebihi `total_amount`. Persentase gagal harus diambil dari output aktual karena belum tersedia pada notebook final.

4.14.3 Join Lokasi Tujuan

A. Kode Program

```
Q-3 Join Lokasi Tujuan 1/1

1 # Q.3: Menambahkan borough tujuan dan mencari pasangan tersibuk
2 data_asal_tujuan = (
3     bersih.merge(
4         zona[["LocationID", "Borough"]].rename(
5             columns={"Borough": "borough_turun"}
6         ),
7         left_on="DOLocationID",
8         right_on="LocationID",
9         how="left",
10        validate="many_to_one"
11    )
12    .drop(columns="LocationID")
13 )
14
15 data_asal_tujuan["borough_naik"] = (
16     data_asal_tujuan["borough_naik"].fillna("Tidak dikenal")
17 )
18 data_asal_tujuan["borough_turun"] = (
19     data_asal_tujuan["borough_turun"].fillna("Tidak dikenal")
20 )
21
22 sepuluh_pasangan = (
23     data_asal_tujuan
24     .groupby(["borough_naik", "borough_turun"], observed=True)
25     .size()
26     .rename("jumlah_perjalanan")
27     .sort_values(ascending=False)
28     .head(10)
29     .reset_index()
30 )
31
32 print("Sepuluh pasangan borough asal-tujuan tersibuk:")
33 display(sepuluh_pasangan)
```

Gambar 4.89 Kode Join Lokasi Tujuan (baris 1-33)

B. Output Program

*** Sepuluh pasangan borough asal-tujuan tersibuk:

	borough_naik	borough_turun	jumlah_perjalanan
0	Manhattan	Manhattan	2491566
1	Queens	Manhattan	155261
2	Manhattan	Queens	82354
3	Manhattan	Brooklyn	62986
4	Queens	Queens	59146
5	Queens	Brooklyn	42187
6	Unknown	Manhattan	18432
7	Unknown	Unknown	13562
8	Manhattan	Bronx	8865
9	Brooklyn	Brooklyn	7910

Gambar 4.90 Output Join Lokasi Tujuan

C. Pembahasan

Tabel zona digabungkan melalui DOLocationID dengan validasi many-to-one, kemudian sepuluh pasangan borough asal-tujuan dengan jumlah perjalanan tertinggi dihitung.

4.14.4 Perbandingan MinMaxScaler

A. Kode Program

```
Q-4 MinMaxScaler 1/1

1 # Q.4: Mengganti StandardScaler dengan MinMaxScaler
2 from sklearn.preprocessing import MinMaxScaler
3
4 skala_minmax = MinMaxScaler().fit(latih[FITUR_NUM])
5 latih_minmax = skala_minmax.transform(latih[FITUR_NUM])
6 uji_minmax = skala_minmax.transform(uji[FITUR_NUM])
7
8 baris_rentang = []
9 for indeks, nama_fitur in enumerate(FITUR_NUM):
10     baris_rentang.append({
11         "fitur": nama_fitur,
12         "std_latih_min": latih_num[:, indeks].min(),
13         "std_latih_max": latih_num[:, indeks].max(),
14         "minmax_latih_min": latih_minmax[:, indeks].min(),
15         "minmax_latih_max": latih_minmax[:, indeks].max(),
16         "minmax_uji_min": uji_minmax[:, indeks].min(),
17         "minmax_uji_max": uji_minmax[:, indeks].max()
18     })
19
20 perbandingan_scaler = pd.DataFrame(baris_rentang).round(3)
21 display(perbandingan_scaler)
22
23 print("Rentang MinMax data latih mendekati 0 sampai 1.")
24 print("Data uji dapat berada di luar rentang karena scaler hanya fit pada data latih.")
```

Gambar 4.91 Kode Perbandingan MinMaxScaler (baris 1-24)

B. Output Program

```
***
```

	fitur	std_latih_min	std_latih_max	minmax_latih_min	minmax_latih_max	minmax_uji_min	minmax_uji_max
0	trip_distance_capped	-0.794	4.247	0.0	1.0	0.0	1.00
1	durasi menit	-1.226	13.925	0.0	1.0	0.0	0.91
2	suhu_c	-2.288	3.347	0.0	1.0	0.0	1.00

```
Rentang MinMax data latih mendekati 0 sampai 1.
Data uji dapat berada di luar rentang karena scaler hanya fit pada data latih.
```

Gambar 4.92 Output Perbandingan MinMaxScaler

C. Pembahasan

MinMaxScaler di-fit hanya pada data latih. Perbedaan skala penting untuk algoritma berbasis jarak atau gradien, sedangkan model pohon umumnya tidak sensitif terhadap transformasi skala monoton.

4.14.5 Pembuktian Validasi many-to-one

A. Kode Program

```
Q-5 Validasi Many-to-one 1/1

1 # Q.5: Membuktikan bahwa many_to_one menolak kunci referensi ganda
2 zona_duplikat = pd.concat(
3     [zona.copy(), zona.iloc[[0]].copy()],
4     ignore_index=True
5 )
6
7 print("Kunci zona asli unik?      ", zona["LocationID"].is_unique)
8 print("Kunci salinan masih unik?", zona_duplikat["LocationID"].is_unique)
9
10 try:
11     bersih[["PULocationID"]].head(1000).merge(
12         zona_duplikat[["LocationID", "Borough"]],
13         left_on="PULocationID",
14         right_on="LocationID",
15         how="left",
16         validate="many_to_one"
17     )
18     raise AssertionError("Join seharusnya gagal karena kunci kanan tidak unik.")
19 except pd.errors.MergeError as error:
20     print("\nError yang diharapkan:")
21     print(error)
22 finally:
23     del zona_duplikat
24
25 print("\nTabel zona dikembalikan ke kondisi awal:", zona["LocationID"].is_unique)
```

Gambar 4.93 Kode Pembuktian Validasi many-to-one (baris 1-25)

B. Output Program

```
*** Kunci zona asli unik?      True
    Kunci salinan masih unik? False

Error yang diharapkan:
Merge keys are not unique in right dataset; not a many-to-one merge

Tabel zona dikembalikan ke kondisi awal: True
```

Gambar 4.94 Output Pembuktian Validasi many-to-one

C. Pembahasan

Satu kunci referensi digandakan pada salinan tabel zona. MergeError yang muncul membuktikan `validate='many_to_one'` mencegah ledakan baris; tabel asli tidak diubah.

4.14.6 Pipeline Inkremental

A. Kode Program

```
Q-6 Pipeline Inkremental 1/3

1 # Q.6: Memproses hanya bulan yang belum tersedia
2 DIR_INKREMENTAL = os.path.join(DIR_KURASI, "inkremental")
3 os.makedirs(DIR_INKREMENTAL, exist_ok=True)
4
5 def siapkan_cuaca_bulan(periode):
6     data_cuaca = ambil_cuaca(
7         periode.start_time.strftime("%Y-%m-%d"),
8         periode.end_time.strftime("%Y-%m-%d")
9     )
10    data_cuaca["time"] = pd.to_datetime(
11        data_cuaca["time"], errors="coerce"
12    )
13    if data_cuaca["time"].isna().any():
14        raise ValueError("Waktu cuaca gagal dikonversi.")
15    return data_cuaca.rename(columns={
16        "time": "jam_mulai",
17        "temperature_2m": "suhu_c",
18        "precipitation": "hujan_mm"
19    })
20
21 def pipeline_inkremental(bulan):
22     periode = pd.Period(bulan, freq="M")
23     nama_bulan = periode.strftime("%Y-%m")
24     path_output = os.path.join(
25         DIR_INKREMENTAL,
26         f"trips_bersih_{nama_bulan}.parquet"
27     )
28
29     if os.path.exists(path_output):
30         print(f"Lewati {nama_bulan}: keluaran sudah tersedia.")
31         return {
32             "bulan": nama_bulan,
33             "status": "cache",
34             "path": path_output
35         }
36
37     url_bulan = (
38         "https://d37ci6vzurychx.cloudfront.net/trip-data/"
39         f"yellow_tripdata_{nama_bulan}.parquet"
40     )
41     path_mentah_bulan = os.path.join(
42         DIR_MENTAH, f"yellow_tripdata_{nama_bulan}.parquet"
43     )
44
```

Gambar 4.95 Kode Pipeline Inkremental Bagian 1 dari 3 (baris 1-44)

```
Q-6 Pipeline Inkremental 2/3

45     unduh_aman(url_bulan, path_mentah_bulan)
46     cuaca_bulan = siapkan_cuaca_bulan(periode)
47     hasil_bulan = pipeline(
48         path_mentah_bulan,
49         PATH_ZONA,
50         cuaca_bulan,
51         tarif_referensi
52     )
53     hasil_bulan["bulan_sumber"] = nama_bulan
54
55     path_sementara = path_output + ".part"
56     hasil_bulan.to_parquet(path_sementara, index=False)
57     os.replace(path_sementara, path_output)
58
59     catat_sumber(
60         f"trip_{nama_bulan}",
61         url_bulan,
62         len(hasil_bulan),
63         "Hasil pipeline inkremental bulanan"
64     )
65     pd.DataFrame(lineage).to_csv(
66         f"{DIR_SIMPAN}/lineage.csv", index=False
67     )
68
69     jumlah_baris = len(hasil_bulan)
70     del hasil_bulan
71     gc.collect()
72
73     print(f"Selesai {nama_bulan}: {jumlah_baris:,} baris.")
74     return {
75         "bulan": nama_bulan,
76         "status": "dibuat",
77         "path": path_output
78     }
79
80     bulan_uji = "2023-01"
81     hasil_pertama = pipeline_inkremental(bulan_uji)
82     berkas_setelah_pertama = sorted(os.listdir(DIR_INKREMENTAL))
83     waktu_ubah_pertama = os.path.getmtime(hasil_pertama["path"])
84
85     hasil_kedua = pipeline_inkremental(bulan_uji)
86     berkas_setelah_kedua = sorted(os.listdir(DIR_INKREMENTAL))
87     waktu_ubah_kedua = os.path.getmtime(hasil_kedua["path"])
88
```

Gambar 4.96 Kode Pipeline Inkremental Bagian 2 dari 3 (baris 45-88)

```
Q-6 Pipeline Inkremental 3/3

89     idempoten = (
90         berkas_setelah_pertama == berkas_setelah_kedua
91         and waktu_ubah_pertama == waktu_ubah_kedua
92     )
93
94     print("\nStatus pemanggilan pertama:", hasil_pertama["status"])
95     print("Status pemanggilan kedua :", hasil_kedua["status"])
96     print(
97         "Berkas baru pada panggilan kedua:",
98         len(berkas_setelah_kedua) - len(berkas_setelah_pertama)
99     )
100     print("Pipeline idempoten:", idempoten)
```

Gambar 4.97 Kode Pipeline Inkremental Bagian 3 dari 3 (baris 89-100)

B. Output Program

```
*** Cache ditemukan: yellow_tripdata_2023-01.parquet
Validasi lulus: 2,986,910 baris x 17 kolom
[lineage] trip_2023-01: 2,986,910 baris
Selesai 2023-01: 2,986,910 baris.
Lewati 2023-01: keluaran sudah tersedia.

Status pemanggilan pertama: dibuat
Status pemanggilan kedua : cache
Berkas baru pada panggilan kedua: 0
Pipeline idempoten: True
```

Gambar 4.98 Output Pipeline Inkremental

C. Pembahasan

Fungsi memeriksa keluaran sebelum mengunduh atau memproses. Pemanggilan kedua harus berstatus cache, menghasilkan nol berkas baru, dan mempertahankan waktu modifikasi keluaran.

BAB V

TUGAS PRAKTIKUM DAN EVALUASI

5.1 Tugas Praktikum - Bagian R

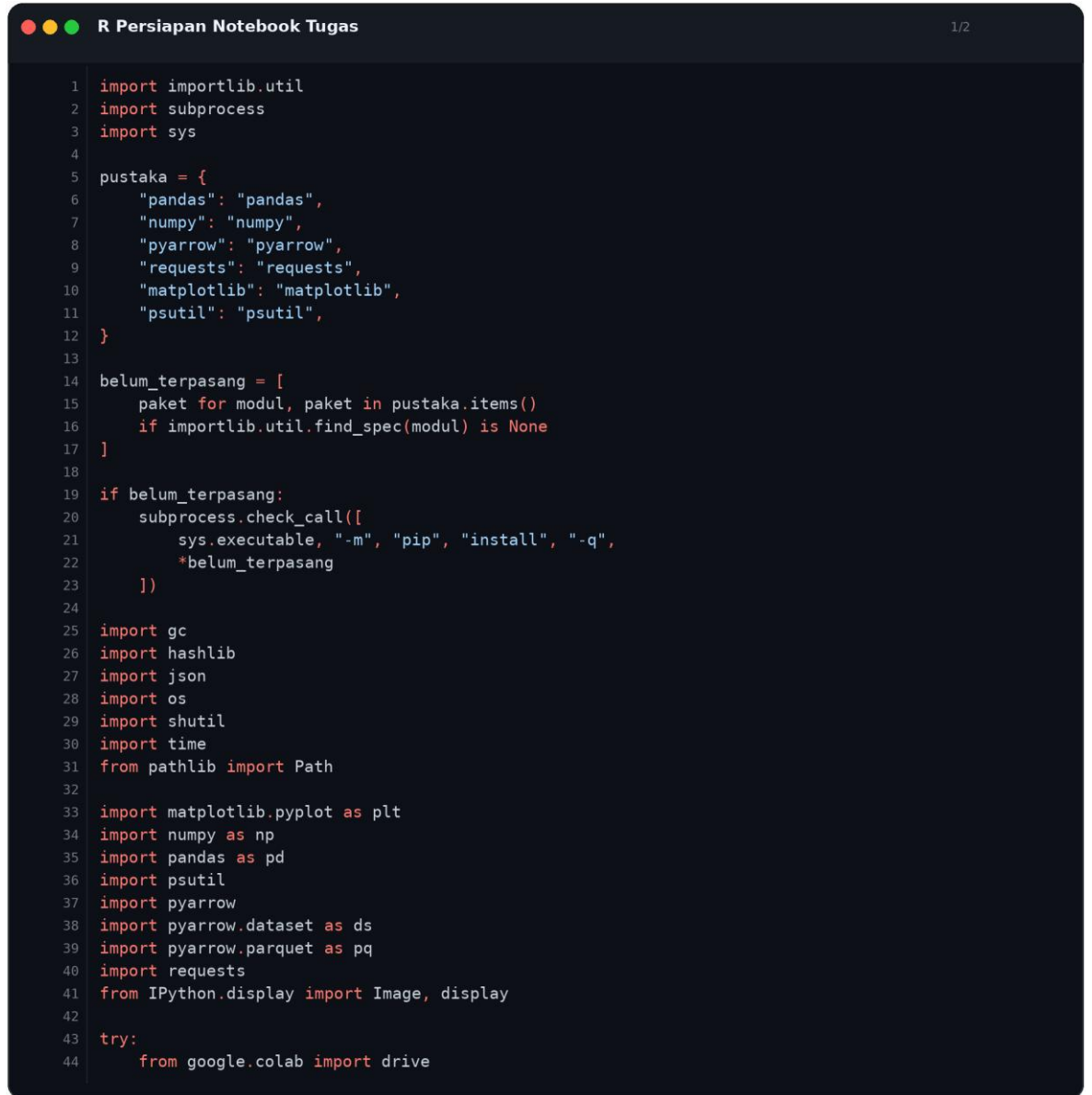
Notebook tugas dipisahkan dari notebook praktikum utama. Pipeline tugas memproses Januari dan Juli 2023, menambahkan Nager.Date sebagai sumber keempat, membandingkan laporan kualitas, menyusun kamus data serta lineage, dan menguji reproducibility. Kode berikut ditampilkan lengkap sesuai notebook, sedangkan output disediakan sebagai placeholder karena notebook tugas yang diterima belum menyimpan hasil eksekusi.

5.1.1 Persiapan Notebook Tugas

A. Tujuan Langkah

Menyiapkan pustaka, Drive, folder, periode Januari dan Juli 2023, serta parameter sumber data.

B. Kode Program



```
1 import importlib.util
2 import subprocess
3 import sys
4
5 pustaka = {
6     "pandas": "pandas",
7     "numpy": "numpy",
8     "pyarrow": "pyarrow",
9     "requests": "requests",
10    "matplotlib": "matplotlib",
11    "psutil": "psutil",
12 }
13
14 belum_terpasang = [
15     paket for modul, paket in pustaka.items()
16     if importlib.util.find_spec(modul) is None
17 ]
18
19 if belum_terpasang:
20     subprocess.check_call([
21         sys.executable, "-m", "pip", "install", "-q",
22         *belum_terpasang
23     ])
24
25 import gc
26 import hashlib
27 import json
28 import os
29 import shutil
30 import time
31 from pathlib import Path
32
33 import matplotlib.pyplot as plt
34 import numpy as np
35 import pandas as pd
36 import psutil
37 import pyarrow
38 import pyarrow.dataset as ds
39 import pyarrow.parquet as pq
40 import requests
41 from IPython.display import Image, display
42
43 try:
44     from google.colab import drive
```

Gambar 5.1 Kode Persiapan Notebook Tugas Bagian 1 dari 2 (baris 1-44)

```
R Persiapan Notebook Tugas 2/2

45     drive.mount("/content/drive")
46     BASE_CONTENT = Path("/content")
47     BASE_DRIVE = Path("/content/drive/MyDrive")
48 except ImportError:
49     # Jalur ini hanya dipakai ketika notebook diuji di luar Colab.
50     BASE_CONTENT = Path.cwd() / "content"
51     BASE_DRIVE = Path.cwd() / "drive"
52
53 PERIODE = ["2023-01", "2023-07"]
54
55 DIR_MENTAH = BASE_CONTENT / "lapisan_mentah_tugas_p03"
56 DIR_KURASI = BASE_CONTENT / "lapisan_terkurasi_tugas_p03"
57 DIR_SIMPAN = BASE_DRIVE / "BigData" / "Praktikum3" / "TugasPraktikum3"
58 DIR_OUTPUT = DIR_SIMPAN / "trips_bersih"
59 DIR_ARTEFAK = DIR_SIMPAN / "artefak"
60
61 for direktori in (
62     DIR_MENTAH, DIR_KURASI, DIR_SIMPAN,
63     DIR_OUTPUT, DIR_ARTEFAK
64 ):
65     direktori.mkdir(parents=True, exist_ok=True)
66
67 print("Python      :", sys.version.split()[0])
68 print("pandas      :", pd.__version__)
69 print("NumPy        :", np.__version__)
70 print("PyArrow       :", pyarrow.__version__)
71 print("requests      :", requests.__version__)
72 print("Periode       :", PERIODE)
73 print("Output        :", DIR_SIMPAN)
```

Gambar 5.2 Kode Persiapan Notebook Tugas Bagian 2 dari 2 (baris 45-73)

C. Output Program

```
*** Mounted at /content/drive
Python      : 3.13.15
pandas      : 2.2.3
NumPy       : 2.1.3
PyArrow     : 23.0.1
requests    : 2.32.4
Periode     : ['2023-01', '2023-07']
Output      : /content/drive/MyDrive/BigData/Praktikum3/TugasPraktikum3
```

Gambar 5.3 Output Persiapan Notebook Tugas

D. Pembahasan

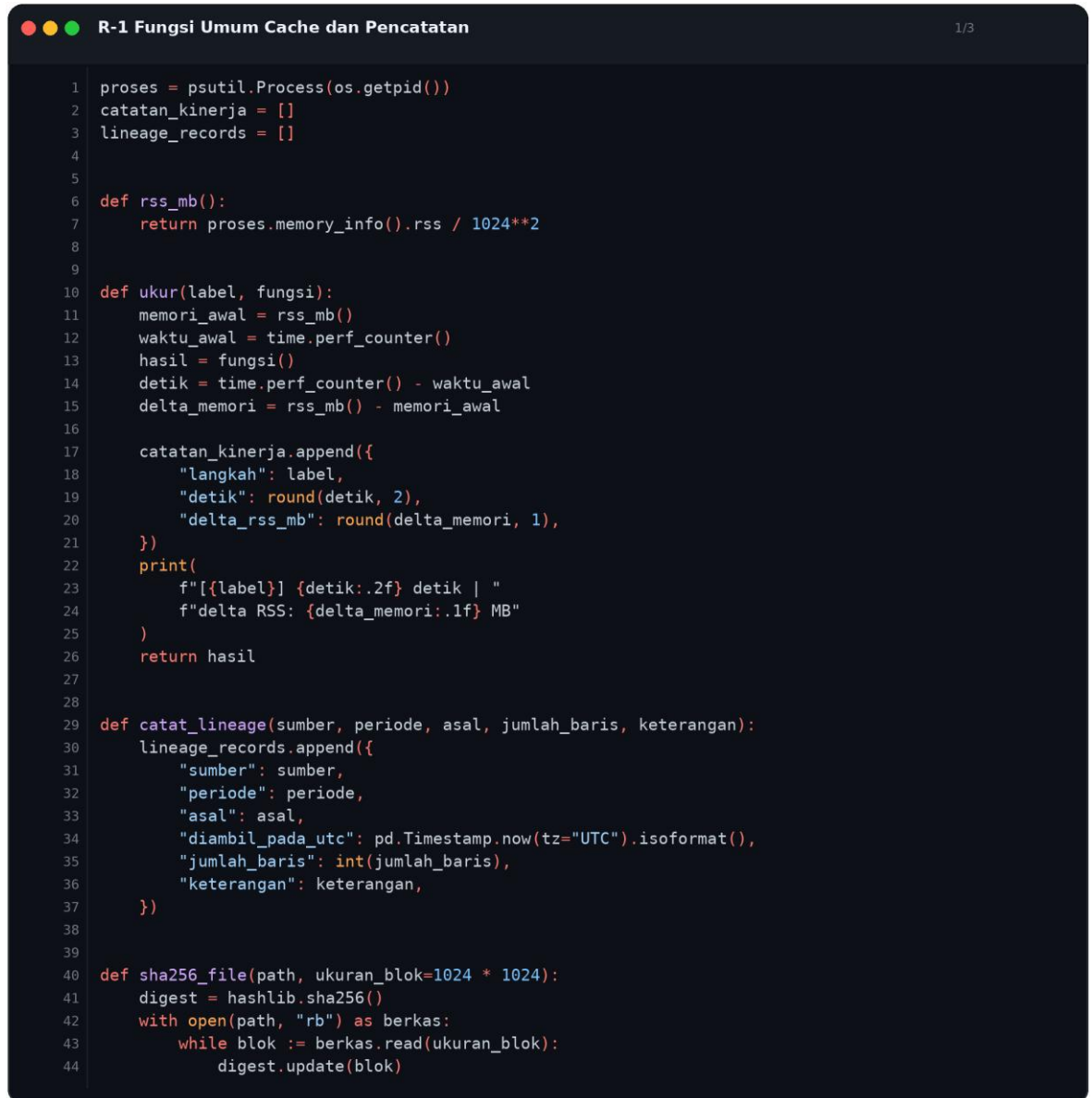
Path digunakan agar alamat folder konsisten. Daftar PERIODE menjadi pusat konfigurasi dua bulan. Hasil aktual belum tersedia pada notebook tugas yang diterima, sehingga output harus diambil setelah Restart session and run all.

5.1.2 R.1 Fungsi Umum, Cache, dan Pencatatan

A. Tujuan Langkah

Membentuk pengukuran kinerja, validasi cache, unduhan atomik, checksum, lineage, dan validasi skema dasar.

B. Kode Program



```
R-1 Fungsi Umum Cache dan Pencatatan 1/3

1 proses = psutil.Process(os.getpid())
2 catatan_kinerja = []
3 lineage_records = []
4
5
6 def rss_mb():
7     return proses.memory_info().rss / 1024**2
8
9
10 def ukur(label, fungsi):
11     memori_awal = rss_mb()
12     waktu_awal = time.perf_counter()
13     hasil = fungsi()
14     detik = time.perf_counter() - waktu_awal
15     delta_memori = rss_mb() - memori_awal
16
17     catatan_kinerja.append({
18         "langkah": label,
19         "detik": round(detik, 2),
20         "delta_rss_mb": round(delta_memori, 1),
21     })
22     print(
23         f"[{label}] {detik:.2f} detik | "
24         f"delta RSS: {delta_memori:.1f} MB"
25     )
26     return hasil
27
28
29 def catat_lineage(sumber, periode, asal, jumlah_baris, keterangan):
30     lineage_records.append({
31         "sumber": sumber,
32         "periode": periode,
33         "asal": asal,
34         "diambil_pada_utc": pd.Timestamp.now(tz="UTC").isoformat(),
35         "jumlah_baris": int(jumlah_baris),
36         "keterangan": keterangan,
37     })
38
39
40 def sha256_file(path, ukuran_blok=1024 * 1024):
41     digest = hashlib.sha256()
42     with open(path, "rb") as berkas:
43         while blok := berkas.read(ukuran_blok):
44             digest.update(blok)
```

Gambar 5.4 Kode R.1 Fungsi Umum, Cache, dan Pencatatan Bagian 1 dari 3 (baris 1-44)


```
R-1 Fungsi Umum Cache dan Pencatatan 2/3

45     return digest.hexdigest()
46
47
48 def cache_valid(path, jenis):
49     path = Path(path)
50     if not path.exists() or path.stat().st_size == 0:
51         return False
52
53     try:
54         if jenis == "parquet":
55             pq.ParquetFile(path)
56         elif jenis == "csv":
57             pd.read_csv(path, nrows=3)
58         elif jenis == "json":
59             with open(path, encoding="utf-8") as berkas:
60                 json.load(berkas)
61         else:
62             raise ValueError(f"Jenis berkas tidak didukung: {jenis}")
63         return True
64     except Exception:
65         return False
66
67
68 def unduh_aman(url, tujuan, jenis, percobaan=3, jeda_awal=2):
69     tujuan = Path(tujuan)
70     tujuan.parent.mkdir(parents=True, exist_ok=True)
71
72     if cache_valid(tujuan, jenis):
73         print(f"Cache ditemukan: {tujuan.name}")
74         return tujuan
75
76     sementara = tujuan.with_suffix(tujuan.suffix + ".part")
77     if sementara.exists():
78         sementara.unlink()
79
80     for indeks in range(percobaan):
81         try:
82             waktu_awal = time.perf_counter()
83             jumlah_byte = 0
84
85             with requests.get(url, stream=True, timeout=120) as respons:
86                 if respons.status_code in (429, 500, 502, 503, 504):
87                     raise requests.HTTPError(
88                         f"Status sementara {respons.status_code}"
```

Gambar 5.5 Kode R.1 Fungsi Umum, Cache, dan Pencatatan Bagian 2 dari 3 (baris 45-88)

```
R-1 Fungsi Umum Cache dan Pencatatan 3/3

89         )
90         respons.raise_for_status()
91
92         with open(sembanta, "wb") as berkas:
93             for blok in respons.iter_content(chunk_size=1024 * 1024):
94                 if blok:
95                     berkas.write(blok)
96                     jumlah_byte += len(blok)
97
98         if not cache_valid(sembanta, jenis):
99             raise ValueError("Berkas hasil unduhan tidak valid")
100
101         os.replace(sembanta, tujuan)
102         detik = max(time.perf_counter() - waktu_awal, 1e-9)
103         kecepatan = jumlah_byte / 1024**2 / detik
104         print(
105             f"Unduhan selesai: {tujuan.name} | "
106             f"{kecepatan:.2f} MB/detik"
107         )
108         return tujuan
109
110     except Exception as error:
111         if sembanta.exists():
112             sembanta.unlink()
113         if indeks == percobaan - 1:
114             raise
115         tunggu = jeda_awal * (2 ** indeks)
116         print(f"Percobaan gagal: {error}. Ulang {tunggu} detik.")
117         time.sleep(tunggu)
118
119     raise RuntimeError("Unduhan tidak berhasil")
```

Gambar 5.6 Kode R.1 Fungsi Umum, Cache, dan Pencatatan Bagian 3 dari 3 (baris 89-119)

C. Pembahasan

Checksum SHA-256 dan lineage memungkinkan sumber mentah ditelusuri. Penulisan atomik mencegah berkas setengah jadi diakui sebagai hasil akhir.

5.1.3 R.2 Akuisisi Tabel Referensi dan Sumber Keempat

A. Tujuan Langkah

Mengambil tabel zona serta hari libur nasional Amerika Serikat dari Nager.Date.

B. Kode Program

```
R-2 Referensi Zona dan Hari Libur 1/2

1  URL_ZONA = (
2    "https://d37ci6vzurychx.cloudfront.net/"
3    "misc/taxi_zone_lookup.csv"
4  )
5  PATH_ZONA = DIR_MENTAH / "taxi_zone_lookup.csv"
6
7  unduh_aman(URL_ZONA, PATH_ZONA, "csv")
8
9  # keep_default_na=False mempertahankan teks literal seperti "N/A".
10 zona = pd.read_csv(PATH_ZONA, keep_default_na=False)
11
12 if not zona["LocationID"].is_unique:
13     raise ValueError("LocationID pada tabel zona tidak unik")
14
15 referensi_pembayaran = pd.DataFrame({
16     "payment_type": [1, 2, 3, 4, 5, 6],
17     "nama_pembayaran": [
18         "Kartu kredit", "Tunai", "Gratis",
19         "Sengketa", "Tidak diketahui", "Perjalanan batal"
20     ],
21     "kena_biaya_admin": [1, 0, 0, 0, 0, 0],
22 })
23
24 URL_LIBUR = (
25     "https://date.nager.at/api/v3/"
26     "PublicHolidays/2023/US"
27 )
28 PATH_LIBUR = DIR_MENTAH / "hari_libur_us_2023.json"
29
30 unduh_aman(URL_LIBUR, PATH_LIBUR, "json")
31
32 with open(PATH_LIBUR, encoding="utf-8") as berkas:
33     libur_raw = json.load(berkas)
34
35 libur = pd.DataFrame(libur_raw)
36 kolom_wajib_libur = {"date", "name", "global"}
37 if not kolom_wajib_libur.issubset(libur.columns):
38     raise ValueError("Struktur respons API hari libur berubah")
39
40 libur = (
41     libur.loc[libur["global"].fillna(False), ["date", "name"]]
42     .assign(tanggal=lambda d: pd.to_datetime(d["date"]))
43     .groupby("tanggal", as_index=False)
44     .agg(nama_libur=("name", lambda s: " / ".join(dict.fromkeys(s))))
```

Gambar 5.8 Kode R.2 Akuisisi Tabel Referensi dan Sumber Keempat Bagian 1 dari 2 (baris 1-44)

```
R-2 Referensi Zona dan Hari Libur 2/2

45 )
46 libur["libur_nasional"] = np.int8(1)
47
48 catat_lineage(
49     "taxi_zone_lookup", "referensi", URL_ZONA,
50     len(zona), "Tabel referensi zona NYC TLC"
51 )
52 catat_lineage(
53     "hari_libur_us", "2023", URL_LIBUR,
54     len(libur), "Sumber keempat; hari libur nasional AS"
55 )
56
57 print("Tabel zona      :", zona.shape)
58 print("Hari libur 2023 :", libur.shape)
59 display(libur.head())
```

Gambar 5.9 Kode R.2 Akuisisi Tabel Referensi dan Sumber Keempat Bagian 2 dari 2 (baris 45-59)

C. Output Program

```
*** Unduhan selesai: taxi_zone_lookup.csv | 0.10 MB/detik
Unduhan selesai: hari_libur_us_2023.json | 0.04 MB/detik
Tabel zona      : (265, 4)
Hari libur 2023 : (11, 3)
```

	tanggal	nama_libur	libur_nasional
0	2023-01-02	New Year's Day	1
1	2023-01-16	Martin Luther King, Jr. Day	1
2	2023-02-20	Presidents Day	1
3	2023-05-29	Memorial Day	1
4	2023-06-19	Juneteenth National Independence Day	1

Gambar 5.10 Output R.2 Akuisisi Tabel Referensi dan Sumber Keempat

D. Pembahasan

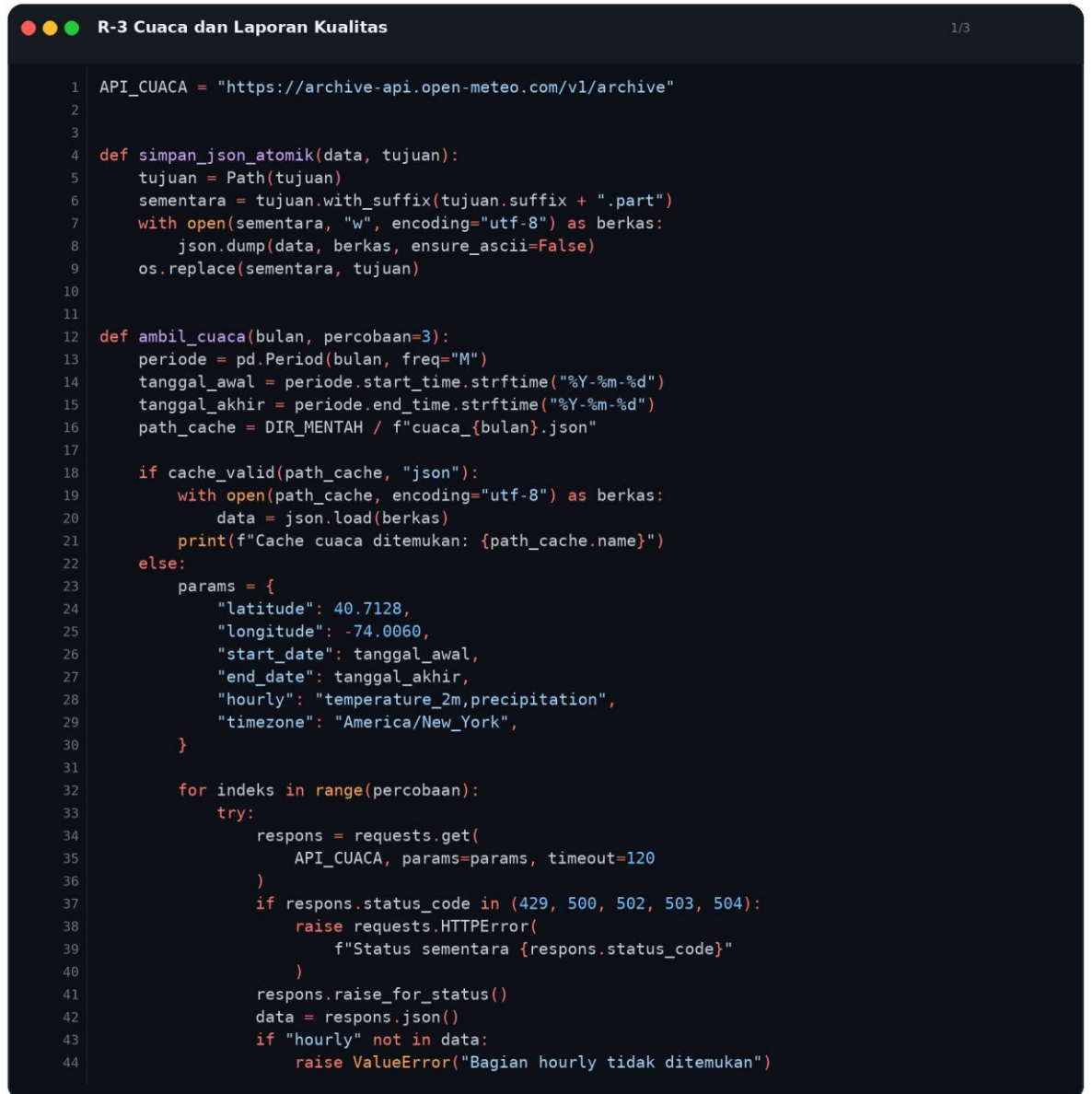
`keep_default_na=False` mencegah label N/A pada tabel zona diubah menjadi nilai kosong. Hari libur diringkas menjadi satu baris per tanggal agar aman digunakan dalam join many-to-one.

5.1.4 R.3 Akuisisi Cuaca dan Laporan Kualitas

A. Tujuan Langkah

Mengambil cuaca setiap periode dan membentuk laporan kualitas yang dapat dibandingkan antarbulan.

B. Kode Program



```
1 API_CUACA = "https://archive-api.open-meteo.com/v1/archive"
2
3
4 def simpan_json_atomik(data, tujuan):
5     tujuan = Path(tujuan)
6     sementara = tujuan.with_suffix(tujuan.suffix + ".part")
7     with open(sementara, "w", encoding="utf-8") as berkas:
8         json.dump(data, berkas, ensure_ascii=False)
9     os.replace(sementara, tujuan)
10
11
12 def ambil_cuaca(bulan, percobaan=3):
13     periode = pd.Period(bulan, freq="M")
14     tanggal_awal = periode.start_time.strftime("%Y-%m-%d")
15     tanggal_akhir = periode.end_time.strftime("%Y-%m-%d")
16     path_cache = DIR_MENTAH / f"cuaca_{bulan}.json"
17
18     if cache_valid(path_cache, "json"):
19         with open(path_cache, encoding="utf-8") as berkas:
20             data = json.load(berkas)
21             print(f"Cache cuaca ditemukan: {path_cache.name}")
22     else:
23         params = {
24             "latitude": 40.7128,
25             "longitude": -74.0060,
26             "start_date": tanggal_awal,
27             "end_date": tanggal_akhir,
28             "hourly": "temperature_2m,precipitation",
29             "timezone": "America/New_York",
30         }
31
32         for indeks in range(percobaan):
33             try:
34                 respons = requests.get(
35                     API_CUACA, params=params, timeout=120
36                 )
37                 if respons.status_code in (429, 500, 502, 503, 504):
38                     raise requests.HTTPError(
39                         f"Status sementara {respons.status_code}"
40                     )
41                 respons.raise_for_status()
42                 data = respons.json()
43                 if "hourly" not in data:
44                     raise ValueError("Bagian hourly tidak ditemukan")
```

Gambar 5.11 Kode R.3 Akuisisi Cuaca dan Laporan Kualitas Bagian 1 dari 3 (baris 1-44)

```
R-3 Cuaca dan Laporan Kualitas 2/3

45         simpan_json_atomik(data, path_cache)
46         break
47     except Exception:
48         if indeks == percobaan - 1:
49             raise
50         time.sleep(2 ** (indeks + 1))
51
52 hourly = data.get("hourly", {})
53 kolom_wajib = {"time", "temperature_2m", "precipitation"}
54 if not kolom_wajib.issubset(hourly):
55     raise ValueError("Kolom cuaca wajib tidak lengkap")
56
57 cuaca = pd.DataFrame({
58     "jam_mulai": pd.to_datetime(hourly["time"]),
59     "suhu_c": pd.to_numeric(
60         hourly["temperature_2m"], errors="coerce"
61     ),
62     "hujan_mm": pd.to_numeric(
63         hourly["precipitation"], errors="coerce"
64     ),
65 })
66
67 if not cuaca["jam_mulai"].is_unique:
68     raise ValueError("Kunci jam pada data cuaca tidak unik")
69
70 catat_lineage(
71     "open_meteo", bulan, API_CUACA, len(cuaca),
72     "Suhu dan presipitasi per jam di New York City"
73 )
74 return cuaca
75
76
77 def buat_laporan_kualitas(trip, bulan, zona_ref):
78     periode = pd.Period(bulan, freq="M")
79     awal = periode.start_time
80     akhir = (periode + 1).start_time
81
82     aturan = {
83         "kelengkapan: passenger_count ada": (
84             trip["passenger_count"].notna()
85         ),
86         "validitas: jarak 0,01-100 mil": (
87             trip["trip_distance"].between(0.01, 100)
88         ),
89     }
```

Gambar 5.12 Kode R.3 Akuisisi Cuaca dan Laporan Kualitas Bagian 2 dari 3 (baris 45-88)

```
R-3 Cuaca dan Laporan Kualitas 3/3

89     "konsistensi: durasi 1-180 menit": (
90         trip["durasi_menit"].between(1, 180)
91     ),
92     "validitas: total_amount > 0": (
93         trip["total_amount"] > 0
94     ),
95     "konsistensi: total >= fare": (
96         trip["total_amount"] >= trip["fare_amount"]
97     ),
98     "ketepatan waktu: dalam periode berkas": (
99         trip["tpep_pickup_datetime"].ge(awal)
100         & trip["tpep_pickup_datetime"].lt(akhir)
101     ),
102     "keunikan: baris tidak duplikat": (
103         ~trip.duplicated()
104     ),
105     "validitas: PULocationID dikenal": (
106         trip["PULocationID"].isin(zona_ref["LocationID"])
107     ),
108 }
109
110 hasil = []
111 for nama, mask in aturan.items():
112     mask = mask.fillna(False)
113     lulus = int(mask.sum())
114     gagal = int((~mask).sum())
115     hasil.append({
116         "periode": bulan,
117         "aturan": nama,
118         "lulus": lulus,
119         "gagal": gagal,
120         "persen_gagal": round(gagal / len(trip) * 100, 4),
121     })
122
123 return pd.DataFrame(hasil).sort_values(
124     "persen_gagal", ascending=False
125 ).reset_index(drop=True)
```

Gambar 5.13 Kode R.3 Akuisisi Cuaca dan Laporan Kualitas Bagian 3 dari 3 (baris 89-125)

D. Pembahasan

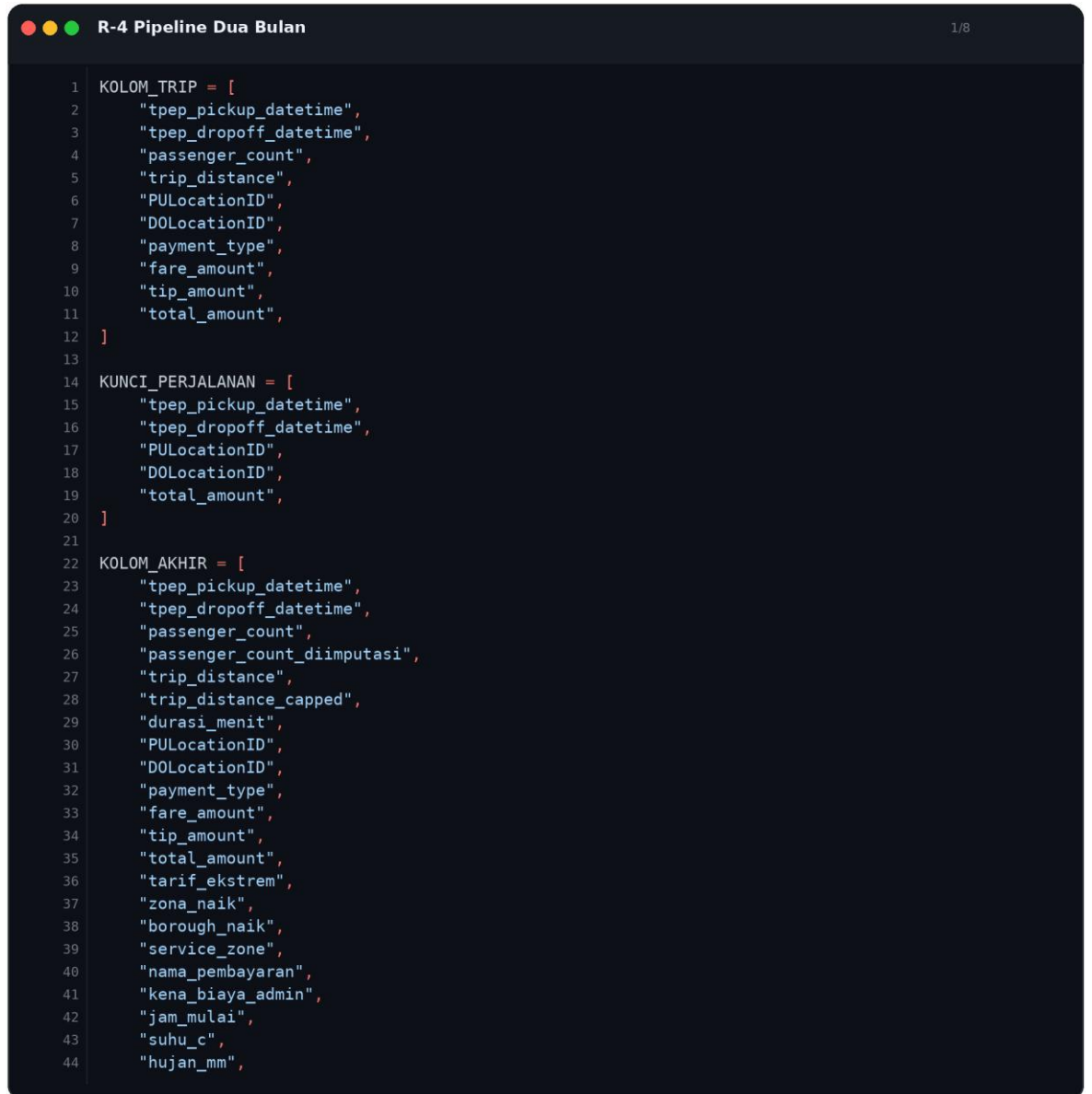
Respons JSON mentah disimpan untuk audit. Persentase gagal perlu dibandingkan, bukan hanya jumlah absolut, karena jumlah transaksi Januari dan Juli dapat berbeda.

5.1.5 R.4 Pipeline Dua Bulan

A. Tujuan Langkah

Membentuk fungsi pipeline_bulan yang mengintegrasikan transaksi, zona, pembayaran, cuaca, dan hari libur.

B. Kode Program



```
1 KOLOM_TRIP = [  
2   "tpep_pickup_datetime",  
3   "tpep_dropoff_datetime",  
4   "passenger_count",  
5   "trip_distance",  
6   "PULocationID",  
7   "DOLocationID",  
8   "payment_type",  
9   "fare_amount",  
10  "tip_amount",  
11  "total_amount",  
12 ]  
13  
14 KUNCI_PERJALANAN = [  
15   "tpep_pickup_datetime",  
16   "tpep_dropoff_datetime",  
17   "PULocationID",  
18   "DOLocationID",  
19   "total_amount",  
20 ]  
21  
22 KOLOM_AKHIR = [  
23   "tpep_pickup_datetime",  
24   "tpep_dropoff_datetime",  
25   "passenger_count",  
26   "passenger_count_diimputasi",  
27   "trip_distance",  
28   "trip_distance_capped",  
29   "durasi_menit",  
30   "PULocationID",  
31   "DOLocationID",  
32   "payment_type",  
33   "fare_amount",  
34   "tip_amount",  
35   "total_amount",  
36   "tarif_ekstrem",  
37   "zona_naik",  
38   "borough_naik",  
39   "service_zone",  
40   "nama_pembayaran",  
41   "kena_biaya_admin",  
42   "jam_mulai",  
43   "suhu_c",  
44   "hujan_mm",
```

Gambar 5.15 Kode R.4 Pipeline Dua Bulan Bagian 1 dari 8 (baris 1-44)


```
R-4 Pipeline Dua Bulan 2/8

45     "cuaca_tersedia",
46     "hujan",
47     "tanggal",
48     "nama_libur",
49     "libur_nasional",
50     "hari_minggu",
51     "akhir_pekan",
52     "kecepatan_mph",
53     "tarif_per_mil",
54     "periode",
55 ]
56
57
58 def validasi_hasil(data, bulan):
59     hilang = sorted(set(KOLOM_AKHIR) - set(data.columns))
60     if hilang:
61         raise AssertionError(f"Kolom akhir belum lengkap: {hilang}")
62     if data.empty:
63         raise AssertionError("Dataset akhir kosong")
64     if data.duplicated(KUNCI_PERJALANAN).any():
65         raise AssertionError("Masih ada duplikat kunci perjalanan")
66     if not data["periode"].eq(bulan).all():
67         raise AssertionError("Label periode tidak konsisten")
68
69     awal = pd.Period(bulan, freq="M").start_time
70     akhir = (pd.Period(bulan, freq="M") + 1).start_time
71     if not data["tpep_pickup_datetime"].between(
72         awal, akhir, inclusive="left"
73     ).all():
74         raise AssertionError("Ada waktu pickup di luar periode")
75
76
77 def pipeline_bulan(bulan):
78     dir_final = DIR_OUTPUT / f"periode={bulan}"
79     path_sukses = dir_final / "_SUCCESS.json"
80
81     # Idempotensi: keluaran lengkap tidak ditulis ulang.
82     if path_sukses.exists():
83         with open(path_sukses, encoding="utf-8") as berkas:
84             metadata = json.load(berkas)
85             metadata["status"] = "cache"
86             for item in metadata.get("lineage", []):
87                 lineage_records.append(item)
88             print(f"[{bulan}] cache keluaran ditemukan; proses dilewati")
```

Gambar 5.16 Kode R.4 Pipeline Dua Bulan Bagian 2 dari 8 (baris 45-88)

```
R-4 Pipeline Dua Bulan 3/8

89     return metadata
90
91     url_trip = (
92         "https://d37ci6vzurychx.cloudfront.net/trip-data/"
93         f"yellow_tripdata_{bulan}.parquet"
94     )
95     path_trip = DIR_MENTAH / f"yellow_tripdata_{bulan}.parquet"
96     unduh_aman(url_trip, path_trip, "parquet")
97
98     waktu_pipeline = time.perf_counter()
99     rss_awal_pipeline = rss_mb()
100
101     trip = ukur(
102         f"{bulan}: baca parquet",
103         lambda: pd.read_parquet(path_trip, columns=KOLOM_TRIP)
104     )
105     jumlah_awal = len(trip)
106
107     trip["durasi_menit"] = (
108         (
109             trip["tpep_dropoff_datetime"]
110             - trip["tpep_pickup_datetime"]
111         ).dt.total_seconds() / 60
112     )
113
114     laporan = buat_laporan_kualitas(trip, bulan, zona)
115     path_laporan = DIR_ARTEFAK / f"laporan_kualitas_{bulan}.csv"
116     laporan.to_csv(path_laporan, index=False)
117
118     periode = pd.Period(bulan, freq="M")
119     awal = periode.start_time
120     akhir = (periode + 1).start_time
121     trip = trip.loc[
122         trip["tpep_pickup_datetime"].ge(awal)
123         & trip["tpep_pickup_datetime"].lt(akhir)
124     ].copy()
125     jumlah_dalam_periode = len(trip)
126
127     trip["passenger_count_diimputasi"] = (
128         trip["passenger_count"].isna().astype("int8")
129     )
130     jam_pickup = trip["tpep_pickup_datetime"].dt.hour
131     median_per_jam = trip.groupby(
132         jam_pickup, observed=True
```

Gambar 5.17 Kode R.4 Pipeline Dua Bulan Bagian 3 dari 8 (baris 89-132)

```
R-4 Pipeline Dua Bulan 4/8

133     )["passenger_count"].transform("median")
134     median_global = trip["passenger_count"].median()
135     trip["passenger_count"] = (
136         trip["passenger_count"]
137         .fillna(median_per_jam)
138         .fillna(median_global)
139     )
140
141     sebelum_duplikat = len(trip)
142     trip = trip.drop_duplicates(
143         subset=KUNCI_PERJALANAN, keep="first"
144     ).copy()
145     dibuang_duplikat = sebelum_duplikat - len(trip)
146
147     mask_valid = (
148         trip["trip_distance"].between(0.01, 100)
149         & trip["durasi_menit"].between(1, 180)
150         & trip["total_amount"].gt(0)
151     )
152     sebelum_filter = len(trip)
153     trip = trip.loc[mask_valid].copy()
154     dibuang_filter = sebelum_filter - len(trip)
155
156     q1 = trip["total_amount"].quantile(0.25)
157     q3 = trip["total_amount"].quantile(0.75)
158     batas_tarif = q3 + 1.5 * (q3 - q1)
159     trip["tarif_ekstrem"] = (
160         trip["total_amount"] > batas_tarif
161     ).astype("int8")
162
163     batas_jarak = trip["trip_distance"].quantile(0.995)
164     trip["trip_distance_capped"] = trip["trip_distance"].clip(
165         upper=batas_jarak
166     )
167
168     zona_join = zona[[
169         "LocationID", "Borough", "Zone", "service_zone"
170     ]].rename(columns={
171         "Borough": "borough_naik",
172         "Zone": "zona_naik",
173     })
174
175     trip = trip.merge(
176         zona_join,
```

Gambar 5.18 Kode R.4 Pipeline Dua Bulan Bagian 4 dari 8 (baris 133-176)

```
R-4 Pipeline Dua Bulan 5/8

177     left_on="PULocationID",
178     right_on="LocationID",
179     how="left",
180     validate="many_to_one",
181 ).drop(columns="LocationID")
182
183 trip["borough_naik"] = (
184     trip["borough_naik"].replace("", np.nan)
185     .fillna("Unknown")
186 )
187 trip["zona_naik"] = (
188     trip["zona_naik"].replace("", np.nan)
189     .fillna("Tidak diketahui")
190 )
191 trip["service_zone"] = (
192     trip["service_zone"].replace("", np.nan)
193     .fillna("Tidak diketahui")
194 )
195
196 trip = trip.merge(
197     referensi_pembayaran,
198     on="payment_type",
199     how="left",
200     validate="many_to_one",
201 )
202 trip["nama_pembayaran"] = trip[
203     "nama_pembayaran"
204 ].fillna("Tidak diketahui")
205 trip["kena_biaya_admin"] = trip[
206     "kena_biaya_admin"
207 ].fillna(0).astype("int8")
208
209 cuaca = ambil_cuaca(bulan)
210 trip["jam_mulai"] = trip[
211     "tpep_pickup_datetime"
212 ].dt.floor("h")
213 trip = trip.merge(
214     cuaca,
215     on="jam_mulai",
216     how="left",
217     validate="many_to_one",
218 )
219
220 trip["tanggal"] = trip[
```

Gambar 5.19 Kode R.4 Pipeline Dua Bulan Bagian 5 dari 8 (baris 177-220)

```
R-4 Pipeline Dua Bulan 6/8

221     "tpep_pickup_datetime"
222 ].dt.normalize()
223 trip = trip.merge(
224     libur,
225     on="tanggal",
226     how="left",
227     validate="many_to_one",
228 )
229 trip["nama_libur"] = trip[
230     "nama_libur"
231 ].fillna("Bukan hari libur")
232 trip["libur_nasional"] = trip[
233     "libur_nasional"
234 ].fillna(0).astype("int8")
235
236 trip["cuaca_tersedia"] = (
237     trip["suhu_c"].notna() & trip["hujan_mm"].notna()
238 )
239 trip["hujan"] = (
240     trip["hujan_mm"].gt(0.1).astype("Int8")
241 )
242 trip.loc[~trip["cuaca_tersedia"], "hujan"] = pd.NA
243
244 trip["hari_minggu"] = trip[
245     "tpep_pickup_datetime"
246 ].dt.dayofweek.astype("int8")
247 trip["akhir_pekan"] = (
248     trip["hari_minggu"] >= 5
249 ).astype("int8")
250 trip["kecepatan_mph"] = (
251     trip["trip_distance"]
252     / (trip["durasi menit"] / 60)
253 ).round(2)
254 trip["tarif_per_mil"] = (
255     trip["total_amount"] / trip["trip_distance"]
256 ).round(3)
257 trip["periode"] = bulan
258
259 hasil = trip[KOLOM_AKHIR].copy()
260 validasi_hasil(hasil, bulan)
261
262 dir_sementara = DIR_OUTPUT / f".periode={bulan}.tmp"
263 if dir_sementara.exists():
264     shutil.rmtree(dir_sementara)
```

Gambar 5.20 Kode R.4 Pipeline Dua Bulan Bagian 6 dari 8 (baris 221-264)

```
R-4 Pipeline Dua Bulan 7/8

265     if dir_final.exists():
266         # Direktori tanpa _SUCCESS dianggap hasil tidak lengkap.
267         shutil.rmtree(dir_final)
268
269     hasil.to_parquet(
270         dir_sementara,
271         index=False,
272         compression="snappy",
273         partition_cols=["borough_naik"],
274     )
275
276     lineage_bulan = [
277         {
278             "sumber": "nyc_tlc_trip",
279             "periode": bulan,
280             "asal": url_trip,
281             "diambil_pada_utc": pd.Timestamp.now(
282                 tz="UTC"
283             ).isoformat(),
284             "jumlah_baris": int(jumlah_awal),
285             "keterangan": (
286                 "Data perjalanan mentah; sha256="
287                 + sha256_file(path_trip)
288             ),
289         },
290         {
291             "sumber": "dataset_bersih",
292             "periode": bulan,
293             "asal": str(path_trip),
294             "diambil_pada_utc": pd.Timestamp.now(
295                 tz="UTC"
296             ).isoformat(),
297             "jumlah_baris": int(len(hasil)),
298             "keterangan": (
299                 "Setelah filter periode, imputasi, deduplikasi, "
300                 "filter validitas, dan empat join"
301             ),
302         },
303     ]
304     lineage_records.extend(lineage_bulan)
305
306     metadata = {
307         "periode": bulan,
308         "status": "dibuat",
```

Gambar 5.21 Kode R.4 Pipeline Dua Bulan Bagian 7 dari 8 (baris 265-308)

```
R-4 Pipeline Dua Bulan 8/8

309     "baris_awal": int(jumlah_awal),
310     "baris_dalam_periode": int(jumlah_dalam_periode),
311     "baris_akhir": int(len(hasil)),
312     "dibuang_luar_periode": int(
313         jumlah_awal - jumlah_dalam_periode
314     ),
315     "dibuang_duplikat": int(dibuang_duplikat),
316     "dibuang_filter": int(dibuang_filter),
317     "cuaca_tidak_tersedia": int(
318         (~hasil["cuaca_tersedia"]).sum()
319     ),
320     "baris_hari_libur": int(
321         hasil["libur_nasional"].sum()
322     ),
323     "waktu_pipeline_detik": round(
324         time.perf_counter() - waktu_pipeline, 2
325     ),
326     "delta_rss_mb": round(rss_mb() - rss_awal_pipeline, 1),
327     "laporan_kualitas": str(path_laporan),
328     "lineage": lineage_bulan,
329 }
330
331 with open(
332     dir_sementara / "_SUCCESS.json",
333     "w",
334     encoding="utf-8",
335 ) as berkas:
336     json.dump(metadata, berkas, indent=2, ensure_ascii=False)
337
338 os.replace(dir_sementara, dir_final)
339 print(
340     f"[{bulan}] {jumlah_awal:} -> {len(hasil):} baris | "
341     f"tersimpan di {dir_final}"
342 )
343
344 del trip, hasil, cuaca, laporan
345 gc.collect()
346 return metadata
```

Gambar 5.22 Kode R.4 Pipeline Dua Bulan Bagian 8 dari 8 (baris 309-346)

C. Pembahasan

Pipeline memeriksa _SUCCESS.json sebelum bekerja, menulis melalui direktori sementara, dan mempartisi hasil menurut periode serta borough. Angka aktual harus diambil dari output setelah notebook dijalankan penuh.

5.1.6 R.5 Menjalankan Januari dan Juli 2023

A. Tujuan Langkah

Menjalankan pipeline untuk dua bulan dan merangkum jumlah awal, akhir, persentase bertahan, waktu, serta delta RSS.

B. Kode Program

```
R-5 Menjalankan Januari dan Juli 1/1

1 metadata_bulan = {}
2
3 for bulan in PERIODE:
4     metadata_bulan[bulan] = pipeline_bulan(bulan)
5     gc.collect()
6
7 ringkasan_pipeline = pd.DataFrame([
8     {
9         "periode": bulan,
10        "status": meta["status"],
11        "baris_awal": meta["baris_awal"],
12        "baris_akhir": meta["baris_akhir"],
13        "persen_tersisa": round(
14            meta["baris_akhir"] / meta["baris_awal"] * 100, 4
15        ),
16        "dibuang_luar_periode": meta["dibuang_luar_periode"],
17        "dibuang_duplikat": meta["dibuang_duplikat"],
18        "dibuang_filter": meta["dibuang_filter"],
19        "waktu_pipeline_detik": meta["waktu_pipeline_detik"],
20        "delta_rss_mb": meta["delta_rss_mb"],
21    }
22    for bulan, meta in metadata_bulan.items()
23 ])
24
25 display(ringkasan_pipeline)
```

Gambar 5.24 Kode R.5 Menjalankan Januari dan Juli 2023 (baris 1-25)

C. Output Program

```
[2023-01] cache keluaran ditemukan; proses dilewati
[2023-07] cache keluaran ditemukan; proses dilewati
```

	periode	status	baris_awal	baris_akhir	persen_tersisa	dibuang_luar_periode	dibuang_duplikat	dibuang_filter	waktu_pipeline_detik	delta_rss_mb
0	2023-01	cache	3066766	2986873	97.3949	48	1	79844	25.33	2565.5
1	2023-07	cache	2907108	2817607	96.9213	59	0	89442	20.48	1583.4

Gambar 5.25 Output R.5 Menjalankan Januari dan Juli 2023

D. Pembahasan

Pipeline Januari 2023 memproses 3.066.766 baris awal dan menghasilkan 2.986.873 baris akhir. Artinya, 97,3949% data berhasil dipertahankan, sedangkan 2,6051% data dibuang selama pra-pemrosesan. Proses tersebut membutuhkan waktu 25,33 detik dengan kenaikan RSS sebesar 2.565,5 MB.

Pipeline Juli 2023 memproses 2.907.108 baris awal dan menghasilkan 2.817.607 baris akhir. Persentase data yang bertahan sebesar 96,9213%, sedangkan 3,0787% data dibuang. Waktu pemrosesannya adalah 20,48 detik dengan kenaikan RSS sebesar 1.583,4 MB.

Secara absolut, data Januari memiliki 159.658 baris awal lebih banyak daripada Juli. Namun, perbandingan berdasarkan persentase menunjukkan bahwa Januari mempertahankan data 0,4736 poin persentase lebih tinggi daripada Juli. Dengan kata lain, proporsi data yang dibuang pada Juli lebih besar meskipun jumlah awalnya lebih sedikit. Pipeline Januari memerlukan waktu 4,85 detik lebih lama dan delta RSS 982,1 MB lebih tinggi. Perbedaan tersebut dapat dipengaruhi oleh volume data, objek yang terbentuk selama transformasi, serta kondisi memori runtime ketika pengujian dilakukan. Oleh karena itu, hasil penggunaan waktu dan memori hanya berlaku pada lingkungan Google Colab yang digunakan dalam praktikum.

5.1.7 R.6 Tidak Ada Duplikasi Antarbulan

A. Tujuan Langkah

Membuktikan tidak ada kunci perjalanan yang sama pada partisi Januari dan Juli.

B. Kode Program

```
R-6 Pemeriksaan Duplikasi Antarbulan 1/1

1 hash_periode = {}
2 ringkasan_duplikat = []
3
4 for bulan in PERIODE:
5     path_bulan = DIR_OUTPUT / f"periode={bulan}"
6     kunci = pd.read_parquet(
7         path_bulan,
8         columns=KUNCI_PERJALANAN,
9     )
10    duplikat_dalam = int(
11        kunci.duplicated(KUNCI_PERJALANAN).sum()
12    )
13    hash_periode[bulan] = np.unique(
14        pd.util.hash_pandas_object(
15            kunci[KUNCI_PERJALANAN], index=False
16        ).to_numpy()
17    )
18    ringkasan_duplikat.append({
19        "periode": bulan,
20        "jumlah_baris": len(kunci),
21        "duplikat_dalam_bulan": duplikat_dalam,
22    })
23    del kunci
24    gc.collect()
25
26    duplikat_antar_bulan = int(np.intersect1d(
27        hash_periode[PERIODE[0]],
28        hash_periode[PERIODE[1]],
29        assume_unique=True,
30    ).size)
31
32    tabel_duplikat = pd.DataFrame(ringkasan_duplikat)
33    display(tabel_duplikat)
34    print("Duplikat antar-bulan:", duplikat_antar_bulan)
35
36    assert tabel_duplikat["duplikat_dalam_bulan"].sum() == 0
37    assert duplikat_antar_bulan == 0
38    print("Validasi lulus: tidak ada duplikasi dalam atau antar-bulan.")
```

Gambar 5.26 Kode R.6 Tidak Ada Duplikasi Antarbulan (baris 1-38)

C. Output Program

```
***
   periode  jumlah_baris  duplikat_dalam_bulan
0  2023-01      2986873              0
1  2023-07      2817607              0
Duplikat antar-bulan: 0
Validasi lulus: tidak ada duplikasi dalam atau antar-bulan.
```

Gambar 5.27
Tidak Ada

Antarbulan

Output R.6
Duplikasi

D. Pembahasan

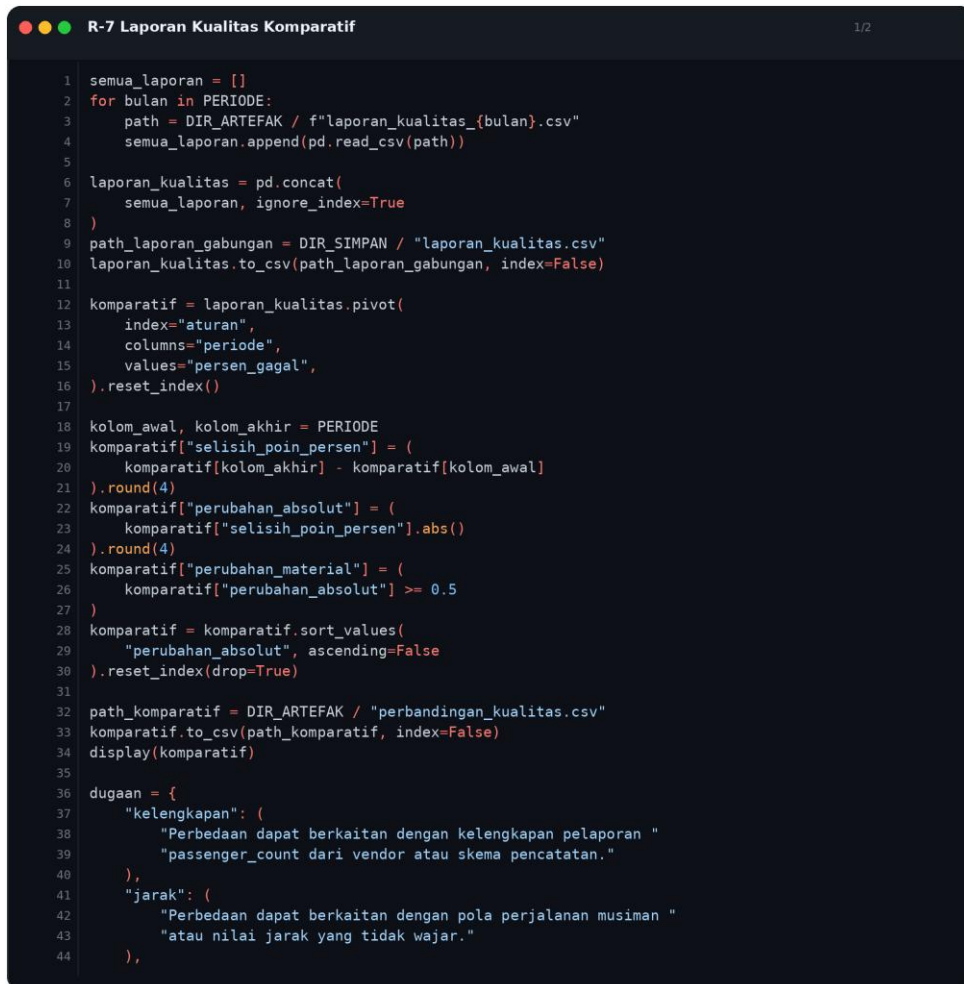
Hash kunci mengurangi kebutuhan menahan seluruh kolom dalam RAM. Nilai irisan nol menjadi bukti tidak ada duplikasi menurut kombinasi kunci yang didefinisikan.

5.1.8 R.7 Laporan Kualitas Komparatif

A. Tujuan Langkah

Menggabungkan laporan kualitas dua bulan dan mengurutkan perubahan persentase kegagalan.

B. Kode Program



```
1 semua_laporan = []
2 for bulan in PERIODE:
3     path = DIR_ARTEFAK / f"laporan_kualitas_{bulan}.csv"
4     semua_laporan.append(pd.read_csv(path))
5
6 laporan_kualitas = pd.concat(
7     semua_laporan, ignore_index=True
8 )
9 path_laporan_gabungan = DIR_SIMPAN / "laporan_kualitas.csv"
10 laporan_kualitas.to_csv(path_laporan_gabungan, index=False)
11
12 komparatif = laporan_kualitas.pivot(
13     index="aturan",
14     columns="periode",
15     values="persen_gagal",
16 ).reset_index()
17
18 kolom_awal, kolom_akhir = PERIODE
19 komparatif["selisih_poin_persen"] = (
20     komparatif[kolom_akhir] - komparatif[kolom_awal]
21 ).round(4)
22 komparatif["perubahan_absolut"] = (
23     komparatif["selisih_poin_persen"].abs()
24 ).round(4)
25 komparatif["perubahan_material"] = (
26     komparatif["perubahan_absolut"] >= 0.5
27 )
28 komparatif = komparatif.sort_values(
29     "perubahan_absolut", ascending=False
30 ).reset_index(drop=True)
31
32 path_komparatif = DIR_ARTEFAK / "perbandingan_kualitas.csv"
33 komparatif.to_csv(path_komparatif, index=False)
34 display(komparatif)
35
36 dugaan = {
37     "kelengkapan": (
38         "Perbedaan dapat berkaitan dengan kelengkapan pelaporan "
39         "passenger_count dari vendor atau skema pencatatan."
40     ),
41     "jarak": (
42         "Perbedaan dapat berkaitan dengan pola perjalanan musiman "
43         "atau nilai jarak yang tidak wajar."
44     ),
45 }
```

Gambar 5.28 Kode R.7 Laporan Kualitas Komparatif Bagian 1 dari 2 (baris 1-44)

```

R-7 Laporan Kualitas Komparatif 2/2

45     "durasi": (
46         "Perbedaan dapat berkaitan dengan kemacetan, perjalanan "
47         "sangat panjang, atau pencatatan waktu yang tidak konsisten."
48     ),
49     "total_amount": (
50         "Perbedaan dapat berkaitan dengan koreksi transaksi, "
51         "pengembalian dana, atau perjalanan tanpa pembayaran normal."
52     ),
53     "total >= fare": (
54         "Perbedaan dapat berkaitan dengan komponen biaya dan transaksi "
55         "koreksi yang tidak mengikuti pola pembayaran biasa."
56     ),
57     "ketepatan waktu": (
58         "Perbedaan dapat berasal dari catatan pada batas bulan atau "
59         "timestamp yang tidak sesuai nama berkas."
60     ),
61     "keunikan": (
62         "Perbedaan dapat berasal dari pencatatan ulang transaksi."
63     ),
64     "PULocationID": (
65         "Perbedaan dapat berasal dari kode lokasi khusus atau referensi "
66         "zona yang belum mencakup kode tersebut."
67     ),
68 }
69
70 print("\nTiga perubahan terbesar:")
71 for _, baris in komparatif.head(3).iterrows():
72     alasan = next(
73         (nilai for kunci, nilai in dugaan.items()
74          if kunci in baris["aturan"]),
75         "Perubahan perlu diperiksa kembali pada data mentah.",
76     )
77     arah = "naik" if baris["selisih_poin_persen"] > 0 else "turun"
78     print(
79         f"- {baris['aturan']}: {arah} "
80         f"{abs(baris['selisih_poin_persen']):.4f} poin persentase. "
81         f"{alasan}"
82     )

```

Gambar 5.29 Kode R.7 Laporan Kualitas Komparatif Bagian 2 dari 2 (baris 45-82)

C. Output Program

periode	aturan	2023-01	2023-07	selisih_poin_persen	perubahan_absolut	perubahan_material
0	kelengkapan: passenger_count ada	2.3394	2.9268	0.5874	0.5874	True
1	konsistensi: total >= fare	0.8159	1.0535	0.2376	0.2376	False
2	validitas: total_amount > 0	0.8404	1.0746	0.2342	0.2342	False
3	validitas: jarak 0,01-100 ml	1.4983	1.7175	0.2192	0.2192	False
4	konsistensi: durasi 1-180 menit	1.1884	1.3108	0.1244	0.1244	False
5	ketepatan waktu: dalam periode berkas	0.0016	0.0020	0.0004	0.0004	False
6	keunikan: baris tidak duplikat	0.0000	0.0000	0.0000	0.0000	False
7	validitas: PULocationID dikenal	0.0000	0.0000	0.0000	0.0000	False

Tiga perubahan terbesar:

- kelengkapan: passenger_count ada: naik 0.5874 poin persentase. Perbedaan dapat berkaitan dengan kelengkapan pelaporan passenger_count dari vendor atau skema pencatatan.
- konsistensi: total >= fare: naik 0.2376 poin persentase. Perbedaan dapat berkaitan dengan komponen biaya dan transaksi koreksi yang tidak mengikuti pola pembayaran biasa.
- validitas: total_amount > 0: naik 0.2342 poin persentase. Perbedaan dapat berkaitan dengan koreksi transaksi, pengembalian dana, atau perjalanan tanpa pembayaran normal.

Gambar 5.30 Output R.7 Laporan Kualitas Komparatif

D. Pembahasan

Aturan kualitas yang mengalami perubahan terbesar adalah **kelengkapan: passenger_count ada**. Persentase kegagalan pada Januari 2023 sebesar 2,3394%, sedangkan pada Juli 2023 meningkat menjadi 2,9268%. Dengan demikian, terjadi

kenaikan sebesar 0,5874 poin persentase. Kenaikan tersebut menunjukkan bahwa proporsi baris tanpa nilai `passenger_count` lebih besar pada Juli dibandingkan Januari. Perbedaan ini diduga berkaitan dengan komposisi vendor, perubahan skema pencatatan, atau perbedaan pola perjalanan pada kedua bulan. Namun, dugaan tersebut masih berupa hipotesis yang perlu diperiksa lebih lanjut melalui data mentah, distribusi waktu, dan sumber pencatatan setiap transaksi.

5.1.9 R.8 Integrasi Sumber Keempat

A. Tujuan Langkah

Mengukur cakupan penanda hari libur nasional pada kedua bulan.

B. Kode Program

```

R-8 Integrasi Sumber Keempat 1/1

1 cakupan_libur = []
2
3 for bulan in PERIODE:
4     bagian = pd.read_parquet(
5         DIR_OUTPUT / f"periode={bulan}",
6         columns=["libur_nasional", "nama_libur"],
7     )
8     cakupan_libur.append({
9         "periode": bulan,
10        "jumlah_perjalanan": len(bagian),
11        "perjalanan_hari_libur": int(
12            bagian["libur_nasional"].sum()
13        ),
14        "persen_hari_libur": round(
15            bagian["libur_nasional"].mean() * 100, 4
16        ),
17        "nama_libur_tercatat": ", ".join(sorted(
18            bagian.loc[
19                bagian["libur_nasional"].eq(1),
20                "nama_libur",
21            ].dropna().unique()
22        )),
23    })
24     del bagian
25     gc.collect()
26
27 cakupan_libur = pd.DataFrame(cakupan_libur)
28 display(cakupan_libur)

```

Gambar 5.31 Kode R.8 Integrasi Sumber Keempat (baris 1-28)

C. Output Program

	periode	jumlah_perjalanan	perjalanan_hari_libur	persen_hari_libur	nama_libur_tercatat
0	2023-01	2986873	141392	4.7338	Martin Luther King, Jr. Day, New Year's Day
1	2023-07	2817607	55819	1.9811	Independence Day

Gambar 5.32 Output R.8 Integrasi Sumber Keempat

D. Pembahasan

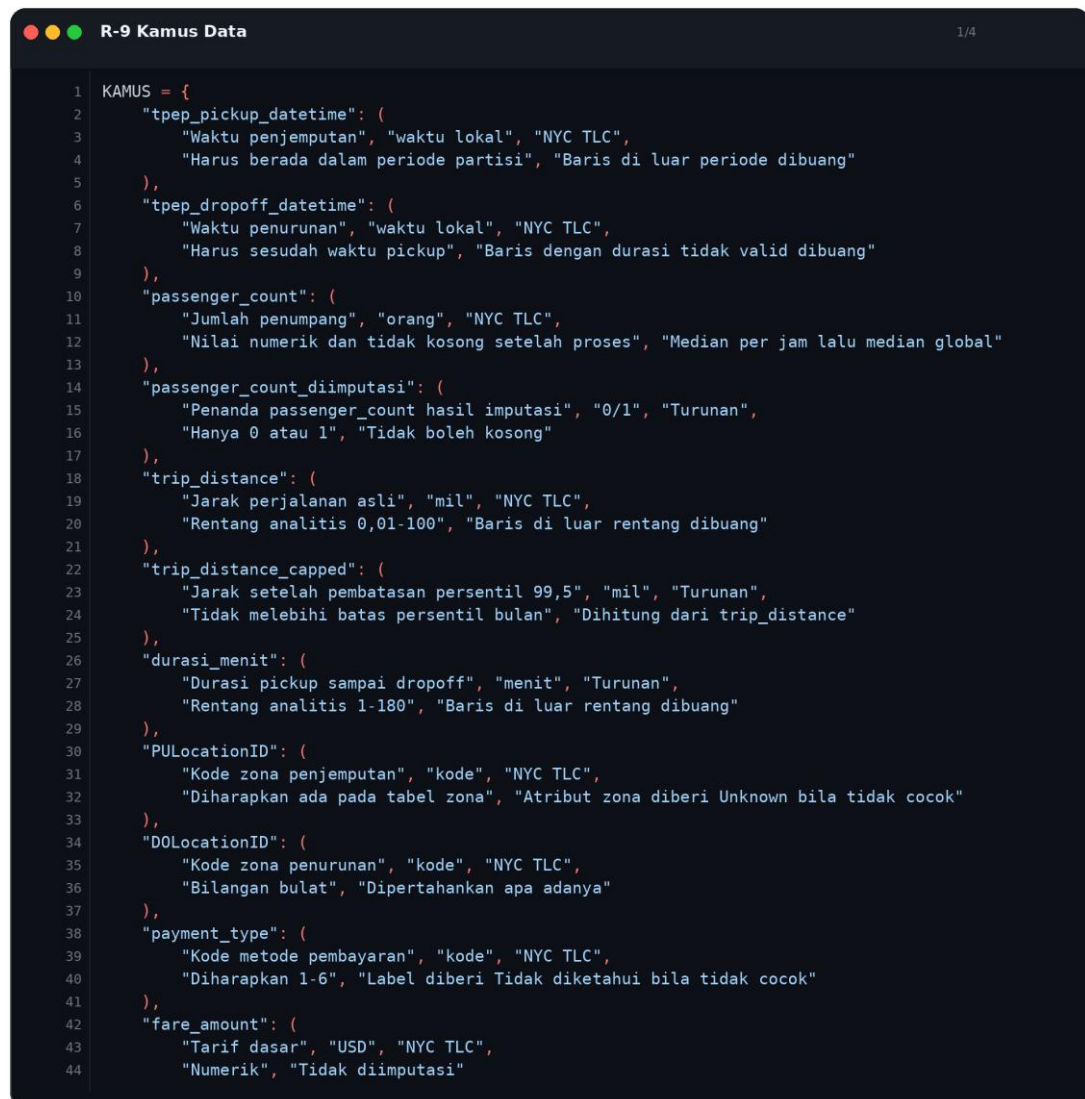
Nager.Date hanya menandai hari libur nasional Amerika Serikat, bukan seluruh hari libur lokal New York. Penanda tersebut tidak membuktikan hari libur menyebabkan perubahan pola perjalanan.

5.1.10 R.9 Kamus Data

A. Tujuan Langkah

Menghasilkan kamus_data.md untuk seluruh kolom akhir dengan tipe, satuan, sumber, validitas, dan penanganan nilai hilang.

B. Kode Program



```
1 KAMUS = {
2   "tpep_pickup_datetime": (
3     "Waktu penjemputan", "waktu lokal", "NYC TLC",
4     "Harus berada dalam periode partisi", "Baris di luar periode dibuang"
5   ),
6   "tpep_dropoff_datetime": (
7     "Waktu penurunan", "waktu lokal", "NYC TLC",
8     "Harus sesudah waktu pickup", "Baris dengan durasi tidak valid dibuang"
9   ),
10  "passenger_count": (
11    "Jumlah penumpang", "orang", "NYC TLC",
12    "Nilai numerik dan tidak kosong setelah proses", "Median per jam lalu median global"
13  ),
14  "passenger_count_diimputasi": (
15    "Penanda passenger_count hasil imputasi", "0/1", "Turunan",
16    "Hanya 0 atau 1", "Tidak boleh kosong"
17  ),
18  "trip_distance": (
19    "Jarak perjalanan asli", "mil", "NYC TLC",
20    "Rentang analitis 0,01-100", "Baris di luar rentang dibuang"
21  ),
22  "trip_distance_capped": (
23    "Jarak setelah pembatasan persentil 99,5", "mil", "Turunan",
24    "Tidak melebihi batas persentil bulan", "Dihitung dari trip_distance"
25  ),
26  "durasi_menit": (
27    "Durasi pickup sampai dropoff", "menit", "Turunan",
28    "Rentang analitis 1-180", "Baris di luar rentang dibuang"
29  ),
30  "PULocationID": (
31    "Kode zona penjemputan", "kode", "NYC TLC",
32    "Diharapkan ada pada tabel zona", "Atribut zona diberi Unknown bila tidak cocok"
33  ),
34  "DOLocationID": (
35    "Kode zona penurunan", "kode", "NYC TLC",
36    "Bilangan bulat", "Dipertahankan apa adanya"
37  ),
38  "payment_type": (
39    "Kode metode pembayaran", "kode", "NYC TLC",
40    "Diharapkan 1-6", "Label diberi Tidak diketahui bila tidak cocok"
41  ),
42  "fare_amount": (
43    "Tarif dasar", "USD", "NYC TLC",
44    "Numerik", "Tidak diimputasi"
```

Gambar 5.33 Kode R.9 Kamus Data Bagian 1 dari 4 (baris 1-44)


```
R-9 Kamus Data 2/4

45     ),
46     "tip_amount": (
47         "Jumlah tip", "USD", "NYC TLC",
48         "Numerik; nilai negatif perlu ditinjau", "Tidak diimputasi"
49     ),
50     "total_amount": (
51         "Total pembayaran", "USD", "NYC TLC",
52         "Harus lebih besar dari nol pada data bersih", "Baris tidak valid dibuang"
53     ),
54     "tarif_ekstrem": (
55         "Penanda total_amount di atas batas IQR", "0/1", "Turunan",
56         "Hanya 0 atau 1", "Tidak boleh kosong"
57     ),
58     "zona_naik": (
59         "Nama zona penjemputan", "kategori", "Taxi Zone Lookup",
60         "Satu label per LocationID", "Diisi Tidak diketahui"
61     ),
62     "borough_naik": (
63         "Borough lokasi penjemputan", "kategori", "Taxi Zone Lookup",
64         "Satu label per LocationID", "Diisi Unknown"
65     ),
66     "service_zone": (
67         "Jenis wilayah layanan taksi", "kategori", "Taxi Zone Lookup",
68         "Satu label per LocationID", "Diisi Tidak diketahui"
69     ),
70     "nama_pembayaran": (
71         "Label metode pembayaran", "kategori", "Referensi internal",
72         "Satu label per payment_type", "Diisi Tidak diketahui"
73     ),
74     "kena_biaya_admin": (
75         "Penanda metode dengan biaya admin", "0/1", "Referensi internal",
76         "Hanya 0 atau 1", "Diisi 0 bila kode tidak dikenal"
77     ),
78     "jam_mulai": (
79         "Waktu pickup dibulatkan per jam", "waktu lokal", "Turunan",
80         "Presisi satu jam", "Dihitung dari pickup"
81     ),
82     "suhu_c": (
83         "Suhu udara pada jam pickup", "derajat Celsius", "Open-Meteo",
84         "Numerik", "Tidak diimputasi; ditandai cuaca_tersedia"
85     ),
86     "hujan_mm": (
87         "Presipitasi pada jam pickup", "mm", "Open-Meteo",
88         "Diharapkan >= 0", "Tidak diimputasi; ditandai cuaca_tersedia"
```

Gambar 5.34 Kode R.9 Kamus Data Bagian 2 dari 4 (baris 45-88)

```
R-9 Kamus Data 3/4

89     ),
90     "cuaca_tersedia": (
91         "Penanda kelengkapan join cuaca", "Boolean", "Turunan",
92         "True atau False", "Tidak boleh kosong"
93     ),
94     "hujan": (
95         "Penanda presipitasi > 0,1 mm", "0/1", "Turunan",
96         "0, 1, atau kosong jika cuaca tidak tersedia", "Dibiarkan kosong bila join cuaca gagal
97     ),
98     "tanggal": (
99         "Tanggal pickup tanpa komponen jam", "tanggal", "Turunan",
100         "Harus sesuai periode", "Dihitung dari pickup"
101     ),
102     "nama_libur": (
103         "Nama hari libur nasional", "kategori", "Nager.Date API",
104         "Satu label gabungan per tanggal", "Diisi Bukan hari libur"
105     ),
106     "libur_nasional": (
107         "Penanda hari libur nasional AS", "0/1", "Nager.Date API",
108         "Hanya 0 atau 1", "Diisi 0 bila tanggal tidak cocok"
109     ),
110     "hari_minggu": (
111         "Nomor hari Senin=0 sampai Minggu=6", "indeks", "Turunan",
112         "Rentang 0-6", "Dihitung dari pickup"
113     ),
114     "akhir_pekan": (
115         "Penanda Sabtu atau Minggu", "0/1", "Turunan",
116         "Hanya 0 atau 1", "Dihitung dari hari_minggu"
117     ),
118     "kecepatan_mph": (
119         "Kecepatan rata-rata perjalanan", "mil/jam", "Turunan",
120         "Dihitung pada durasi dan jarak valid", "Tidak diimputasi"
121     ),
122     "tarif_per_mil": (
123         "Total pembayaran per mil", "USD/mil", "Turunan",
124         "Dihitung pada jarak > 0", "Tidak diimputasi"
125     ),
126     "periode": (
127         "Partisi bulan data", "YYYY-MM", "Parameter pipeline",
128         "Harus sama dengan bulan pickup", "Tidak boleh kosong"
129     ),
130 }
131
```

Gambar 5.35 Kode R.9 Kamus Data Bagian 3 dari 4 (baris 89-131)


```

R-9 Kamus Data 4/4

132 if set(KAMUS) != set(KOLOM_AKHIR):
133     kurang = sorted(set(KOLOM_AKHIR) - set(KAMUS))
134     lebih = sorted(set(KAMUS) - set(KOLOM_AKHIR))
135     raise AssertionError(
136         f"Kamus tidak lengkap. Kurang={kurang}; lebih={lebih}"
137     )
138
139 schema = ds.dataset(
140     str(DIR_OUTPUT / f"periode={PERIODE[0]}"),
141     format="parquet",
142     partitioning="hive",
143 ).schema
144 tipe_aktual = {field.name: str(field.type) for field in schema}
145
146 baris_md = [
147     "# Kamus Data - Tugas Praktikum Big Data 3",
148     "",
149     "| Nama kolom | Tipe aktual | Satuan | Sumber | Definisi | Aturan validitas | Penanganan n",
150     "ilai hilang |",
151     "|---|---|---|---|---|---|---|",
152 ]
153
154 for kolom in KOLOM_AKHIR:
155     definisi, satuan, sumber, validitas, missing = KAMUS[kolom]
156     nilai = [
157         kolom,
158         tipe_aktual.get(kolom, "string partisi"),
159         satuan,
160         sumber,
161         definisi,
162         validitas,
163         missing,
164     ]
165     nilai = [str(item).replace("|", "\\|") for item in nilai]
166     baris_md.append("| " + " | ".join(nilai) + " |")
167
168 PATH_KAMUS = DIR_SIMPAN / "kamus_data.md"
169 PATH_KAMUS.write_text("\n".join(baris_md), encoding="utf-8")
170
171 print("Kamus data tersimpan:", PATH_KAMUS)
172 print("Jumlah kolom dalam kamus:", len(KOLOM_AKHIR))
173 print("\n".join(baris_md[:7]))

```

Gambar 5.36 Kode R.9 Kamus Data Bagian 4 dari 4 (baris 132-172)

C. Output Program

```

Kamus data tersimpan: /content/drive/MyDrive/BigData/Praktikum3/tugasPraktikum3/kamus_data.md
Jumlah kolom dalam kamus: 32
# Kamus Data - Tugas Praktikum Big Data 3

| Nama kolom | Tipe aktual | Satuan | Sumber | Definisi | Aturan validitas | Penanganan nilai hilang |
|---|---|---|---|---|---|---|
| tpep_pickup_datetime | timestamp[us] | waktu lokal | NYC TLC | Waktu penjemputan | Harus berada dalam periode partisi | Baris di luar periode dibuang |
| tpep_dropoff_datetime | timestamp[us] | waktu lokal | NYC TLC | Waktu penempatan | Harus sesudah waktu pickup | Baris dengan durasi tidak valid dibuang |
| passenger_count | double | orang | NYC TLC | Jumlah penumpang | Nilai numerik dan tidak kosong setelah proses | Median per jam lalu median global |

```

Gambar 5.37 Output R.9 Kamus Data

D. Pembahasan

Daftar kamus dibandingkan dengan KOLOM_AKHIR menggunakan assert. Tipe dibaca dari skema Parquet aktual sehingga dokumentasi tidak bergantung pada perkiraan.

5.1.11 R.10 Lineage dan Diagram Alur

A. Tujuan Langkah

Menyimpan lineage.csv dan membuat diagram alur dari sumber sampai dataset akhir.

B. Kode Program

```
R-10 Lineage dan Diagram Alur 1/3

1 # Tambahkan lineage referensi yang sudah dicatat di awal.
2 lineage_df = pd.DataFrame(lineage_records).drop_duplicates(
3     subset=["sumber", "periode", "asal"], keep="last"
4 )
5 PATH_LINEAGE = DIR_SIMPAN / "lineage.csv"
6 lineage_df.to_csv(PATH_LINEAGE, index=False)
7
8 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
9
10
11 def kotak(ax, x, y, w, h, teks, warna):
12     bentuk = FancyBboxPatch(
13         (x, y), w, h,
14         boxstyle="round,pad=0.012,rounding_size=0.02",
15         linewidth=1.2,
16         edgecolor="#1f2937",
17         facecolor=warna,
18     )
19     ax.add_patch(bentuk)
20     ax.text(
21         x + w / 2, y + h / 2, teks,
22         ha="center", va="center", fontsize=9,
23         wrap=True,
24     )
25
26
27 def panah(ax, x1, y1, x2, y2):
28     ax.add_patch(FancyArrowPatch(
29         (x1, y1), (x2, y2),
30         arrowstyle="-|>", mutation_scale=14,
31         linewidth=1.2, color="#374151",
32     ))
33
34
35 fig, ax = plt.subplots(figsize=(11.69, 8.27))
36 ax.set_xlim(0, 1)
37 ax.set_ylim(0, 1)
38 ax.axis("off")
39 ax.set_title(
40     "Lineage Data - Tugas Praktikum Big Data 3",
41     fontsize=16, fontweight="bold", pad=18,
42 )
43
44 sumber = [
```

Gambar 5.38 Kode R.10 Lineage dan Diagram Alur Bagian 1 dari 3 (baris 1-44)

```
R-10 Lineage dan Diagram Alur 2/3

45     (0.04, "NYC TLC Trip\nJan & Jul 2023"),
46     (0.28, "Taxi Zone\nLookup"),
47     (0.52, "Open-Meteo\nCuaca per Jam"),
48     (0.76, "Nager.Date\nHari Libur"),
49 ]
50 for x, teks in sumber:
51     kotak(ax, x, 0.78, 0.19, 0.10, teks, "#dbeafe")
52     panah(ax, x + 0.095, 0.78, 0.50, 0.63)
53
54 kotak(
55     ax, 0.28, 0.53, 0.44, 0.12,
56     "Akuisisi aman\ncache - retry - file mentah - SHA-256",
57     "#e0f2fe",
58 )
59 panah(ax, 0.50, 0.53, 0.50, 0.43)
60
61 transformasi = (
62     "Transformasi per Bulan\n"
63     "filter periode -> imputasi -> deduplikasi -> "
64     "filter validitas -> join -> fitur"
65 )
66 kotak(ax, 0.22, 0.30, 0.56, 0.13, transformasi, "#fef3c7")
67
68 ringkas_buang = []
69 for bulan in PERIODE:
70     meta = metadata_bulan[bulan]
71     ringkas_buang.append(
72         f"{bulan}: {meta['baris_awal']:,} -> "
73         f"{meta['baris_akhir']:,}; dibuang "
74         f"{meta['baris_awal'] - meta['baris_akhir']:,}"
75     )
76 ax.text(
77     0.50, 0.26, "\n".join(ringkas_buang),
78     ha="center", va="top", fontsize=9, color="#7c2d12",
79 )
80
81 panah(ax, 0.50, 0.30, 0.50, 0.16)
82 kotak(
83     ax, 0.26, 0.06, 0.48, 0.10,
84     "Dataset Akhir Berpartisi\nperiode=YYYY-MM / borough_naik...",
85     "#dcfce7",
86 )
87
88 PATH_DIAGRAM_PNG = DIR_SIMPAN / "diagram_lineage.png"
```

Gambar 5.39 Kode R.10 Lineage dan Diagram Alur Bagian 2 dari 3 (baris 45-88)

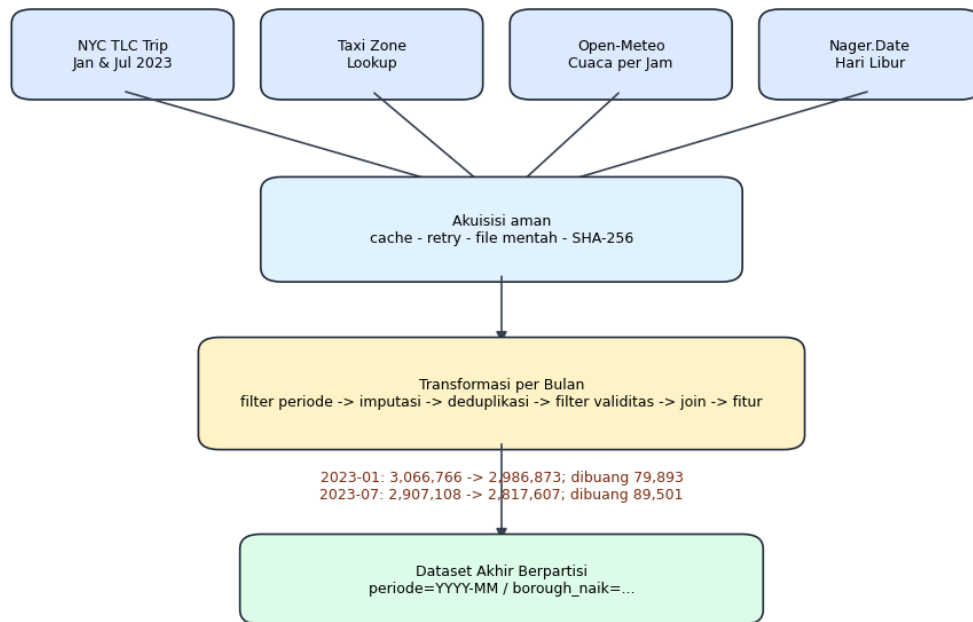
```
R-10 Lineage dan Diagram Alur 3/3

89 PATH_DIAGRAM_PDF = DIR_SIMPAN / "diagram_lineage.pdf"
90 plt.savefig(PATH_DIAGRAM_PNG, dpi=200, bbox_inches="tight")
91 plt.savefig(PATH_DIAGRAM_PDF, bbox_inches="tight")
92 plt.show()
93
94 print("Lineage CSV :", PATH_LINEAGE)
95 print("Diagram PNG :", PATH_DIAGRAM_PNG)
96 print("Diagram PDF :", PATH_DIAGRAM_PDF)
```

Gambar 5.40 Kode R.10 Lineage dan Diagram Alur Bagian 3 dari 3 (baris 89-96)

C. Output Program

Lineage Data - Tugas Praktikum Big Data 3



Gambar 5.41 Output R.10 Lineage dan Diagram Alur

D. Pembahasan

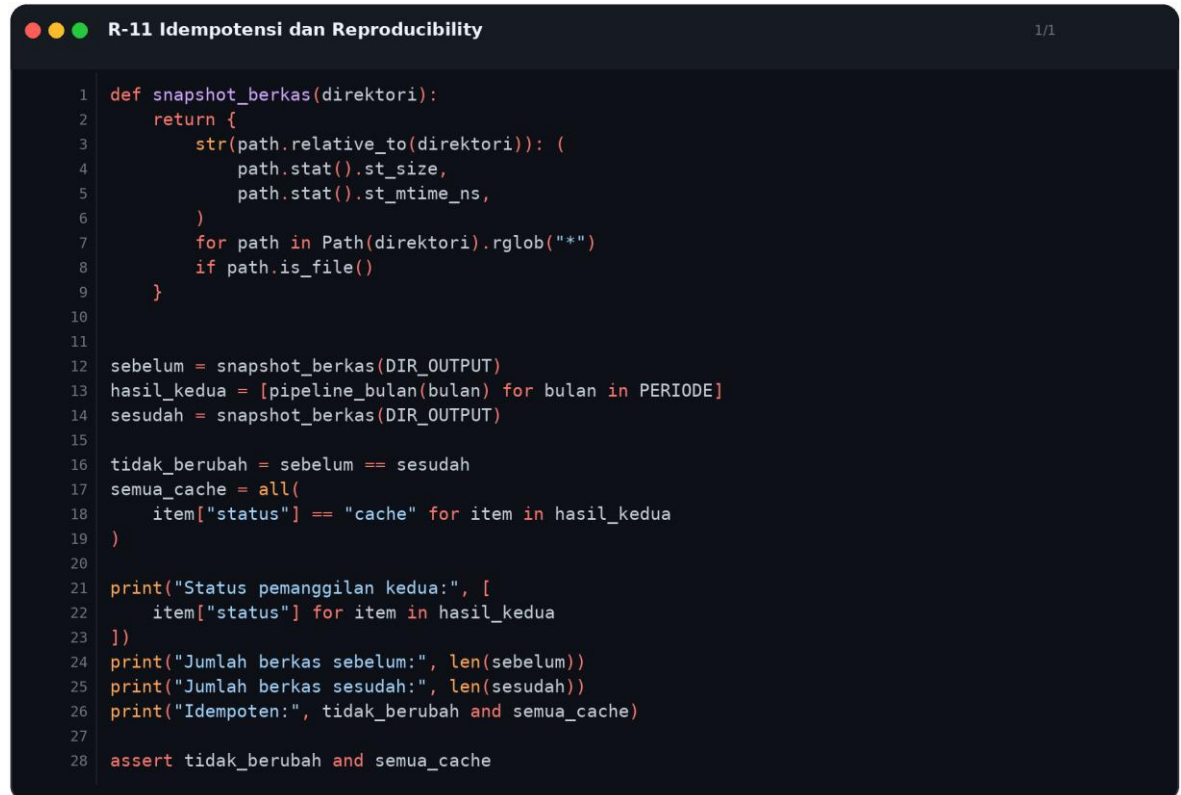
Diagram harus menampilkan sumber transaksi, zona, cuaca, pembayaran, dan hari libur, serta jumlah baris yang dibuang pada setiap periode berdasarkan metadata aktual.

5.1.12 R.11 Idempotensi dan Reproducibility

A. Tujuan Langkah

Menjalankan pipeline dua kali dan membandingkan snapshot berkas.

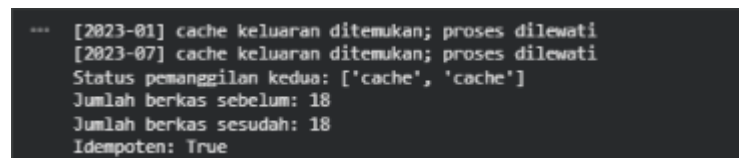
B. Kode Program



```
1 def snapshot_berkas(direktori):
2     return {
3         str(path.relative_to(direktori)): (
4             path.stat().st_size,
5             path.stat().st_mtime_ns,
6         )
7         for path in Path(direktori).rglob("*")
8         if path.is_file()
9     }
10
11
12 sebelum = snapshot_berkas(DIR_OUTPUT)
13 hasil_kedua = [pipeline_bulan(bulan) for bulan in PERIODE]
14 sesudah = snapshot_berkas(DIR_OUTPUT)
15
16 tidak_berubah = sebelum == sesudah
17 semua_cache = all(
18     item["status"] == "cache" for item in hasil_kedua
19 )
20
21 print("Status pemanggilan kedua:", [
22     item["status"] for item in hasil_kedua
23 ])
24 print("Jumlah berkas sebelum:", len(sebelum))
25 print("Jumlah berkas sesudah:", len(sesudah))
26 print("Idempoten:", tidak_berubah and semua_cache)
27
28 assert tidak_berubah and semua_cache
```

Gambar 5.42 Kode R.11 Idempotensi dan Reproducibility (baris 1-28)

C. Output Program



```
*** [2023-01] cache keluaran ditemukan; proses dilewati
*** [2023-07] cache keluaran ditemukan; proses dilewati
Status pemanggilan kedua: ['cache', 'cache']
Jumlah berkas sebelum: 18
Jumlah berkas sesudah: 18
Idempoten: True
```

Gambar 5.43 Output R.11 Idempotensi dan Reproducibility

D. Pembahasan

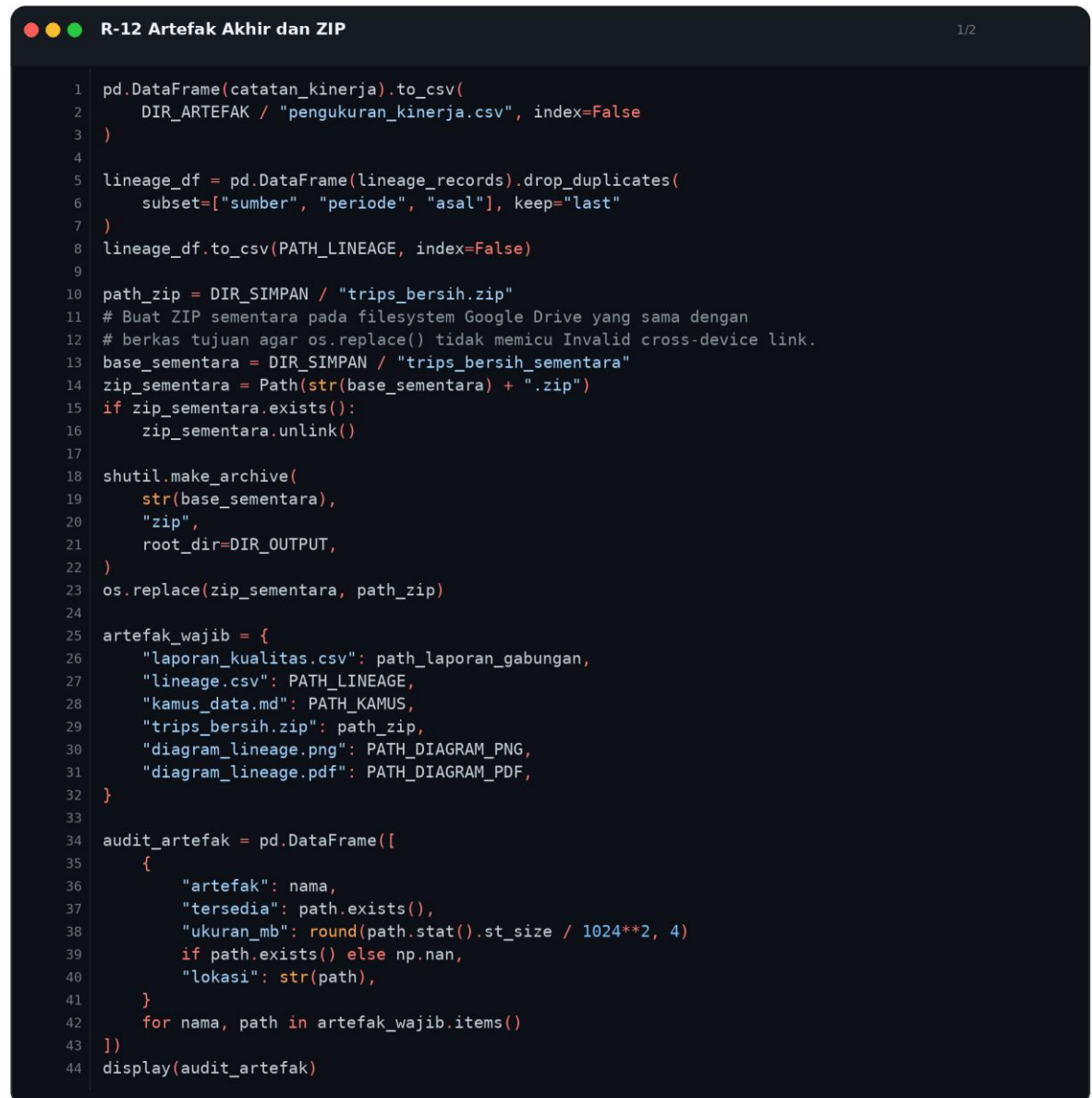
Bukti lengkap memerlukan Restart session and run all pada runtime bersih. Panggilan kedua tidak boleh menghasilkan berkas baru atau mengubah hasil yang sudah lengkap.

5.1.13 R.12 Artefak Akhir dan Arsip ZIP

A. Tujuan Langkah

Menyimpan laporan kualitas, lineage, kinerja, kamus data, dan arsip trips_bersih.zip ke Drive.

B. Kode Program



```
R-12 Artefak Akhir dan ZIP 1/2

1 pd.DataFrame(catatan_kinerja).to_csv(
2     DIR_ARTEFAK / "pengukuran_kinerja.csv", index=False
3 )
4
5 lineage_df = pd.DataFrame(lineage_records).drop_duplicates(
6     subset=["sumber", "periode", "asal"], keep="last"
7 )
8 lineage_df.to_csv(PATH_LINEAGE, index=False)
9
10 path_zip = DIR_SIMPAN / "trips_bersih.zip"
11 # Buat ZIP sementara pada filesystem Google Drive yang sama dengan
12 # berkas tujuan agar os.replace() tidak memicu Invalid cross-device link.
13 base_sementara = DIR_SIMPAN / "trips_bersih_sementara"
14 zip_sementara = Path(str(base_sementara) + ".zip")
15 if zip_sementara.exists():
16     zip_sementara.unlink()
17
18 shutil.make_archive(
19     str(base_sementara),
20     "zip",
21     root_dir=DIR_OUTPUT,
22 )
23 os.replace(zip_sementara, path_zip)
24
25 artefak_wajib = {
26     "laporan_kualitas.csv": path_laporan_gabungan,
27     "lineage.csv": PATH_LINEAGE,
28     "kamus_data.md": PATH_KAMUS,
29     "trips_bersih.zip": path_zip,
30     "diagram_lineage.png": PATH_DIAGRAM_PNG,
31     "diagram_lineage.pdf": PATH_DIAGRAM_PDF,
32 }
33
34 audit_artefak = pd.DataFrame([
35     {
36         "artefak": nama,
37         "tersedia": path.exists(),
38         "ukuran_mb": round(path.stat().st_size / 1024**2, 4)
39         if path.exists() else np.nan,
40         "lokasi": str(path),
41     }
42     for nama, path in artefak_wajib.items()
43 ])
44 display(audit_artefak)
```

Gambar 5.44 Kode R.12 Artefak Akhir dan Arsip ZIP Bagian 1 dari 2 (baris 1-44)

```
R-12 Artefak Akhir dan ZIP 2/2

45
46 if not audit_artefak["tersedia"].all():
47     raise FileNotFoundError("Masih ada artefak wajib yang belum dibuat")
48
49 print("Artefak pipeline selesai dibuat.")
50 print("Laporan PDF utama disusun terpisah berdasarkan output notebook.")
```

Gambar 5.45 Kode R.12 Artefak Akhir dan Arsip ZIP Bagian 2 dari 2 (baris 45-50)

C. Output Program

```
***
```

	artefak	tersedia	ukuran_mb	lokasi
0	laporan_kualitas.csv	True	0.0009	/content/drive/MyDrive/BigData/Praktikum3/Tuga...
1	lineage.csv	True	0.0012	/content/drive/MyDrive/BigData/Praktikum3/Tuga...
2	kamus_data.md	True	0.0042	/content/drive/MyDrive/BigData/Praktikum3/Tuga...
3	trips_bersih.zip	True	152.9397	/content/drive/MyDrive/BigData/Praktikum3/Tuga...
4	diagram_lineage.png	True	0.1509	/content/drive/MyDrive/BigData/Praktikum3/Tuga...
5	diagram_lineage.pdf	True	0.0268	/content/drive/MyDrive/BigData/Praktikum3/Tuga...

```
Artefak pipeline selesai dibuat.  
Laporan PDF utama disusun terpisah berdasarkan output notebook.
```

Gambar 5.46 Output R.12 Artefak Akhir dan Arsip ZIP

D. Pembahasan

Kesalahan Invalid cross-device link diperbaiki dengan membuat ZIP sementara pada filesystem Google Drive yang sama dengan tujuan sebelum os.replace dijalankan.

5.1.14 R.13 Ringkasan Hasil Aktual

A. Tujuan Langkah

Mencetak ringkasan akhir dari metadata eksekusi dua bulan.

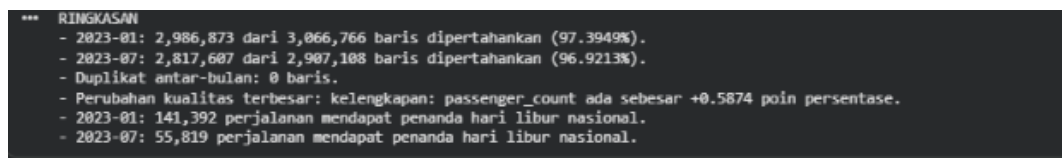
B. Kode Program



```
1 terbesar = komparatif.iloc[0]
2
3 print("RINGKASAN TUGAS PRAKTIKUM 3")
4 for bulan in PERIODE:
5     meta = metadata_bulan[bulan]
6     persen = meta["baris_akhir"] / meta["baris_awal"] * 100
7     print(
8         f"- {bulan}: {meta['baris_akhir']:,} dari "
9         f"{meta['baris_awal']:,} baris dipertahankan "
10        f"({persen:.4f}%)."
11    )
12
13 print(f"- Duplikat antar-bulan: {duplikat_antar_bulan:,} baris.")
14 print(
15     f"- Perubahan kualitas terbesar: {terbesar['aturan']} "
16     f"sebesar {terbesar['selisih_poin_persen']:+.4f} "
17     "poin persentase."
18 )
19 for _, baris in cakupan_libur.iterrows():
20     print(
21         f"- {baris['periode']}: "
22         f"{baris['perjalanan_hari_libur']:,} perjalanan "
23         "mendapat penanda hari libur nasional."
24     )
```

Gambar 5.47 Kode R.13 Ringkasan Hasil Aktual (baris 1-24)

C. Output Program



```
*** RINGKASAN
- 2023-01: 2,986,873 dari 3,066,766 baris dipertahankan (97.3949%).
- 2023-07: 2,817,607 dari 2,907,108 baris dipertahankan (96.9213%).
- Duplikat antar-bulan: 0 baris.
- Perubahan kualitas terbesar: kelengkapan: passenger_count ada sebesar +0.5874 poin persentase.
- 2023-01: 141,392 perjalanan mendapat penanda hari libur nasional.
- 2023-07: 55,819 perjalanan mendapat penanda hari libur nasional.
```

Gambar 5.48 Output R.13 Ringkasan Hasil Aktual

D. Pembahasan

[Berdasarkan ringkasan output R.13, aturan kualitas yang mengalami perubahan terbesar adalah **kelengkapan: passenger_count ada**. Persentase kegagalannya meningkat dari 2,3394% pada Januari 2023 menjadi 2,9268% pada Juli 2023, sehingga terjadi perubahan sebesar **+0,5874 poin persentase**.

Hasil pengujian idempotensi adalah **[masukkan hasil idempotensi berdasarkan output R.11 atau R.13]**. Pipeline menghasilkan **[masukkan jumlah artefak yang terbentuk]** artefak yang disimpan pada **[masukkan lokasi direktori keluaran]**. Nilai tersebut belum dicantumkan karena tidak terlihat pada screenshot output R.13 yang tersedia.

5.2 Jawaban Pertanyaan Evaluasi - Bagian S

5.2.1 Jelaskan perbedaan ETL dan ELT, dan mengapa ELT lebih lazim pada sistem Big Data modern.

ETL melakukan ekstraksi, transformasi, lalu pemuatan ke sistem tujuan. ELT memuat data mentah terlebih dahulu, kemudian melakukan transformasi di dalam data warehouse atau data lake. ELT lebih lazim pada sistem Big Data modern karena penyimpanan relatif murah, data mentah tetap tersedia, dan transformasi dapat dijalankan ulang sesuai perubahan skema atau kebutuhan analisis.

5.2.2 Sebutkan tiga risiko akuisisi data lewat API dan cara menanganinya masing-masing. Kode status HTTP mana yang layak dicoba ulang, dan mana yang tidak?

Tiga risiko utama adalah pembatasan laju permintaan, perubahan struktur respons, dan gangguan jaringan atau server. Penanganannya berupa cache dan jeda permintaan, validasi skema, serta retry dengan exponential backoff. Status 429 dan 5xx layak dicoba ulang karena umumnya sementara. Status 400, 401, 403, dan 404 tidak layak diulang tanpa memperbaiki permintaan atau hak akses.

5.2.3 Apa perbedaan MCAR, MAR, dan MNAR? Berikan satu contoh MNAR pada data transaksi dan jelaskan mengapa pengisian nilai justru berbahaya dalam kasus itu.

MCAR berarti kehilangan tidak berkaitan dengan variabel apa pun. MAR berarti kehilangan dapat dijelaskan oleh variabel lain yang teramati. MNAR berarti peluang hilang berkaitan dengan nilai yang tidak tercatat. Contohnya, pelanggan sengaja tidak mencatat tip ketika nilainya sangat kecil. Imputasi median dapat menaikkan kelompok tersebut dan menghilangkan pola perilaku yang sebenarnya.

5.2.4 Mengapa metode Z-score kurang sesuai untuk data yang distribusinya sangat miring ke kanan? Metode apa yang lebih baik, dan mengapa?

Z-score bergantung pada rerata dan simpangan baku yang mudah tertarik oleh ekor kanan dan nilai ekstrem. IQR atau metode berbasis persentil lebih sesuai karena menggunakan ukuran yang lebih tahan terhadap kemencengan distribusi.

5.2.5 Apa yang terjadi pada jumlah baris bila tabel referensi pada join many-to-one ternyata memiliki kunci ganda? Sebutkan dua cara mencegahnya.

Jika tabel referensi memiliki kunci ganda, satu transaksi dapat berpasangan dengan beberapa baris referensi sehingga jumlah baris bertambah. Pencegahannya adalah memeriksa `is_unique` sebelum join dan memakai `validate='many_to_one'` agar proses berhenti ketika asumsi kunci unik dilanggar.

5.2.6 Jelaskan data leakage dalam konteks scaling fitur, dan sebutkan urutan langkah yang benar untuk mencegahnya.

Leakage terjadi ketika parameter scaling mempelajari informasi dari data uji. Urutan yang benar adalah membagi data menjadi latih dan uji, melakukan fit hanya pada data latih, lalu melakukan transform pada data latih dan data uji dengan scaler yang sama.

5.2.7 Apa arti pipeline yang idempoten, dan sebutkan dua teknik konkret untuk mencapainya.

Pipeline idempoten menghasilkan keadaan akhir yang sama ketika dijalankan berulang dengan input dan parameter yang sama. Teknik konkret pada notebook adalah memeriksa penanda `_SUCCESS.json` sebelum memproses periode dan menulis keluaran melalui lokasi sementara sebelum penggantian atomik.

5.2.8 Mengapa mempartisi berdasarkan kolom yang nilainya sangat beragam (kardinalitas tinggi) merugikan? Apa gejala yang muncul?

Kolom berkardinalitas tinggi menghasilkan sangat banyak direktori dan berkas kecil. Akibatnya, pencarian metadata dan pembukaan berkas meningkat, sedangkan pemrosesan paralel menjadi tidak efisien. Gejalanya berupa ribuan berkas kecil, listing direktori lambat, dan pembacaan tetap lambat walaupun data yang dibutuhkan sedikit.

5.2.9 Anda diminta melakukan scraping data ulasan pengguna sebuah situs e-commerce untuk penelitian. Sebutkan empat hal yang wajib Anda periksa lebih dulu, dan satu kondisi yang membuat Anda harus menolak.

Hal yang wajib diperiksa adalah ketentuan layanan, `robots.txt`, perlindungan data pribadi dan hak cipta, serta batas laju permintaan. Scraping harus ditolak apabila mengharuskan pembobolan autentikasi, melewati CAPTCHA atau kontrol akses, atau secara jelas dilarang oleh ketentuan situs dan tidak ada izin pemilik data.

BAB VI

PENUTUP

6.1 Kesimpulan

Praktikum berhasil mengintegrasikan berkas Parquet dan CSV, API cuaca, serta basis data SQLite dalam satu alur pra-pemrosesan. Dataset utama memuat 3.066.766 baris pada 10 kolom terpilih, sedangkan data bersih berjumlah 2.986.910 baris. Dengan demikian, 97,3961% data dipertahankan dan susut kumulatif hanya 2,6039%.

Masalah kualitas terbesar adalah 71.743 nilai `passenger_count` yang hilang atau 2,339%. Nilai tersebut diisi menggunakan median per jam dan ditandai melalui indikator imputasi. Outlier tarif tidak langsung dibuang; IQR menandai 371.616 baris dan Z-score menandai 87.248 baris, sehingga perbedaan 284.368 baris menunjukkan sensitivitas metode terhadap distribusi yang miring ke kanan.

Pipeline pandas menghasilkan 2.986.910 baris dalam 7,30 detik, sedangkan aksi count Spark menghasilkan jumlah yang sama dalam 44,68 detik. Pandas lebih cepat pada runtime dan ukuran data ini, sementara Spark tetap memperlihatkan BroadcastHashJoin untuk tabel zona. Hasil tersebut tidak mendukung klaim bahwa Spark selalu lebih cepat; manfaatnya bergantung pada skala data, jumlah sumber daya, partisi, dan biaya komunikasi.

Pipeline utama bersifat idempoten dan menghasilkan artefak kualitas, lineage, kinerja, serta Parquet berpartisi. Notebook tugas telah menyediakan pipeline dua bulan dan sumber keempat, tetapi kesimpulan komparatif Januari-Juli harus dilengkapi setelah seluruh sel tugas dijalankan melalui Restart session and run all dan output aslinya ditempel pada placeholder laporan.

DAFTAR PUSTAKA

- Chambers, B., dan Zaharia, M. (2018). Spark: The Definitive Guide. O'Reilly Media.
- Kimball, R., dan Ross, M. (2013). The Data Warehouse Toolkit (Edisi ke-3). Wiley.
- Little, R. J. A., dan Rubin, D. B. (2019). Statistical Analysis with Missing Data (Edisi ke-3). Wiley.
- McKinney, W. (2022). Python for Data Analysis (Edisi ke-3). O'Reilly Media.
- NYC Taxi & Limousine Commission. TLC Trip Record Data dan Taxi Zone Lookup. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- Open-Meteo. Historical Weather API Documentation. <https://open-meteo.com/en/docs/historical-weather-api>
- pandas Development Team. Merge, join, concatenate and compare. https://pandas.pydata.org/docs/user_guide/merging.html
- Reis, J., dan Housley, M. (2022). Fundamentals of Data Engineering. O'Reilly Media.
- scikit-learn Developers. Preprocessing data dan Common pitfalls. https://scikit-learn.org/stable/common_pitfalls.html
- Wickham, H. (2014). Tidy Data. Journal of Statistical Software, 59(10), 1-23.
- Nager.Date. Public Holiday API. <https://date.nager.at/Api>

LAMPIRAN

Lampiran 1 Tautan Akses Praktikum

Repository Praktikum 3:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum03

Notebook Praktikum 3:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/blob/main/Praktikum03/BD_A_P03_2411531001_Habibi_Putra_Rizqullah.ipynb

Notebook Tugas Praktikum 3:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/blob/main/Praktikum03/BD_A_TugasP03_2411531001_Habibi_Putra_Rizqullah_%281%29.ipynb

Folder Output Praktikum:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum03/Output%20Praktikum

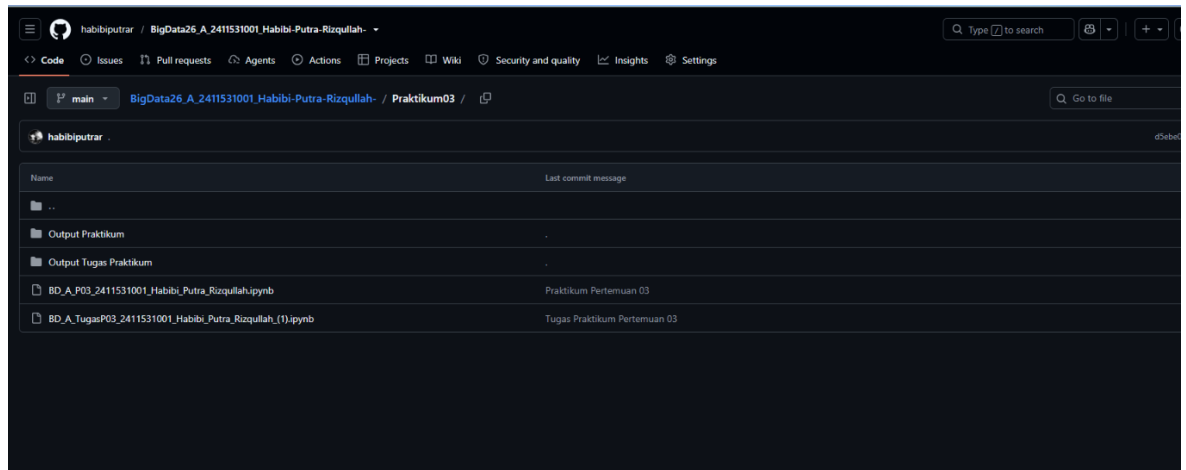
Folder Output Tugas Praktikum:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum03/Output%20Tugas%20Praktikum

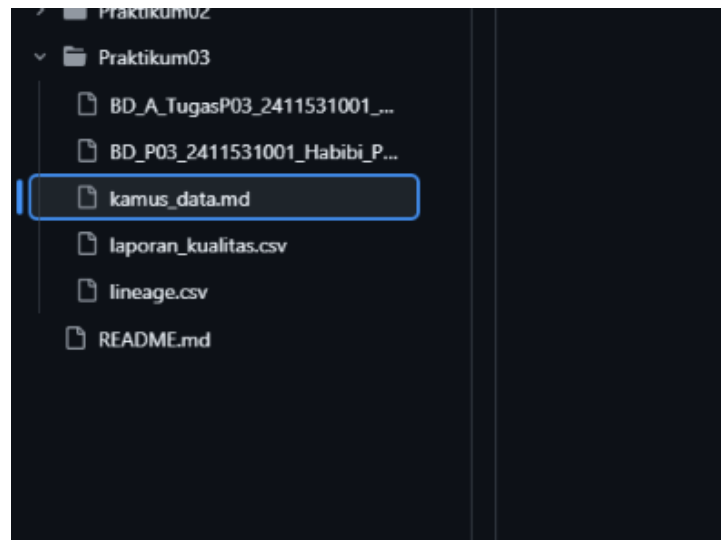
Google Drive Praktikum 3:

https://drive.google.com/drive/folders/108V3OLKvsBIJCvmeTSSrfkQqZrdgvnWu?usp=drive_link

Lampiran 2 Dokumentasi Struktur Repository



Gambar L.1 Struktur folder Praktikum03 pada repository GitHub



Gambar L.2 Daftar notebook dan artefak pendukung pada repository